

Master's thesis

Master's Programme in Computer Science

Performance, Safety, and Engineering Trade-offs of Rust and C++ in Nokia RAN Control-Plane Systems

Ha Vien

August 24, 2026

Faculty of Science

University of Helsinki

Contact information

P. O. Box 68 (Pietari Kalmin katu 5)
00014 University of Helsinki,Finland

Email address: info@cs.helsinki.fi
URL: http://www.cs.helsinki.fi/

HELSINGIN YLIOPISTO – HELSINGFORS UNIVERSITET – UNIVERSITY OF HELSINKI

Tiedekunta — Fakultet — Faculty	Koulutusohjelma — Utbildningsprogram — Study programme
Faculty of Science	Master's Programme in Computer Science

Tekijä — Författare — Author

Ha Vien

Työn nimi — Arbetets titel — Title

Performance, Safety, and Engineering Trade-offs of Rust and C++ in Nokia RAN Control-Plane Systems

Ohjaajat — Handledare — Supervisors

Prof. Keijo Heljanko, Christoph Luneburg, Petteri Kotimaa

Työn laji — Arbetets art — Level	Aika — Datum — Month and year	Sivumäärä — Sidoantal — Number of pages
Master's thesis	August 24, 2026	!! Run three times to appear !!

Tiivistelmä — Referat — Abstract

Context. Radio Access Network (RAN) control planes are written mostly in C++ for performance, at a well-known cost in memory safety and concurrency. Rust promises comparable performance with compile-time safety, but its fit for this latency-critical setting is unproven.

Objective. The thesis compares modern C++ and Rust on a UE Context Management service modelled on a real Nokia gNB-CU-CP component. Average-throughput comparisons leave four dimensions open: tail latency and jitter under overload, the C++ defect classes Rust prevents at compile time, the overhead of its safety model, and the cost of incremental adoption.

Methods. One language-agnostic specification is frozen first, then implemented twice with identical algorithms, concurrency model, allocator, and build settings. A synthetic workload is swept to overload, both versions pass the same parity tests, and the evaluation adds fault injection and an FFI prototype.

Results. Safe Rust rejects all five injected memory-safety and data-race defect classes at compile time. In C++ the same five build cleanly and show up only at runtime, if a sanitiser catches them. Five further classes fail in both languages. On two of them C++ tooling reports what Rust does not. The seam to C++ leaves the core `unsafe`-free and costs at most 10.91% of one operation. Performance changes at saturation. Below it median latency matches, yet an end-to-end call spends only 0.250% on language work. Cross-thread handoff takes the rest, so the sweep reflects the design more than the language. Timed alone, Rust's operation is 1.6× faster and the process is a few MiB larger. Above saturation the order flips: C++ holds 1.8× the throughput of Rust in 20/20 interleaved repeats with both allocators. The far tail below saturation cannot be settled on a virtualised host.

Conclusions. The main cost of adopting Rust is engineering effort and the learning curve, not run-time performance at nominal load. The gain is freedom from data races, not from concurrency defects in general. Overload narrows that claim: a service that must degrade gracefully depends on the standard library's synchronisation primitives, not on the safety model, and a nominal-load test would not expose the gap.

ACM Computing Classification System (CCS)
Software and its engineering → Software notations and tools → General programming languages
Computer systems organization → Real-time systems → Real-time system specification
Networks → Network types → Mobile networks

Avainsanat — Nyckelord — Keywords

Rust, C++, RAN control plane, tail latency, memory safety, fault injection, incremental adoption

Säilytyspaikka — Förvaringsställe — Where deposited

Helsinki University Library

Muita tietoja — övriga uppgifter — Additional information

Software study track

Contents

List of Figures

List of Tables

1 Introduction

Mobile networks are moving from specialised hardware to software on ordinary servers. Functions that once ran on fixed-purpose, vertically integrated equipment are now re-implemented as programs on general-purpose machines. The Radio Access Network (RAN), long the most hardware-bound part of the stack, is part of that shift.

In 5G, the base station (gNB) is split into a central unit and one or more distributed units. The central unit is split again into control- and user-plane functions [**3gpp38401**, **3gpp38300**]. Open-RAN pushes the same idea further: the RAN becomes virtualised, multi-vendor software linked by open interfaces [**polese2023oran**]. The control plane that results, the gNB-CU-CP, is a large, stateful, highly concurrent program. It tracks every connected device, runs RRC signalling [**3gpp38331**], and coordinates mobility and handovers, all within strict latency limits.

Load at a busy site is high. A single site can run thousands of connection-management and mobility procedures per second, each with a budget of a few milliseconds [**3gpp38913**]. A deployed control plane may track millions of active devices at once. As the RAN moves to cloud-native, disaggregated deployments, the correctness, reliability, and timing of this software increasingly set the quality of the whole network. An outage or a tail-latency spike here is not a local defect. Every device the site serves is affected [**dean2013tail**, **barroso2017microseconds**].

This control-plane software is, today, written mostly in C++. The choice is a rational one: C++ offers the low-level control, mature tooling, and performance that latency-critical systems demand. But that performance comes with a well-known cost. C++ places the burden of memory and concurrency management on the programmer, and decades of experience show that this burden is routinely mishandled, producing use-after-free errors, double-frees, data races, and related defects that are expensive to find, dangerous in production, and responsible for a large share of security vulnerabilities in industry practice [**msrc2019memory**, **google2022android**]. Rust has emerged as the first mainstream systems language to promise an escape from this trade-off: memory safety and data-race freedom enforced at compile time, with performance intended to be competitive with C++ [**klabnik2023rust**, **jung2018rustbelt**]. Whether that promise holds for a telecom-grade, latency-critical control plane, where predictability under overload matters as much as raw speed, is not well understood. This thesis, carried out at Nokia, addresses the question directly: it develops two functionally equivalent implementations of a representative RAN subsystem, a UE Context Management service modelled on a real Nokia gNB-CU-CP component, in modern C++ and in Rust, and compares them under a realistic synthetic signalling workload.

Three terms recur throughout. The *gNB-CU-CP* is the control-plane software this work is about. *UE Context Management (UECM)* is the specific subsystem reimplemented and measured. *Tail latency*, the p99 and p99.9 percentiles rather than the mean, is the property that, more than average speed, decides whether such a system meets its deadlines [**dean2013tail**, **barroso2017microseconds**]. Appendix **??** fixes these and

the rest of the domain vocabulary for readers outside telecom.

1.1 Motivation and industrial context

Three trends make this comparison timely. The first is the cloud-native evolution of the RAN itself. Moving control-plane functions onto general-purpose servers raises, rather than lowers, the premium on software quality: the software is longer-lived, evolves continuously, runs at higher concurrency, and must meet its deadlines without the determinism that purpose-built hardware once provided. This shift is accelerated by Open-RAN disaggregation, which turns the base station into virtualised, multi-vendor software connected by open interfaces [**polese2023oran**], so that the control plane a vendor ships increasingly competes on software-engineering quality (reliability, predictable timing, security, and the ease with which a large codebase can be evolved safely) and no longer on purpose-built hardware. In this setting a latency regression or a memory-corruption defect is not a localised bug but a network-wide reliability and security risk, and a language-selection question stops being academic and becomes a concrete engineering decision.

The second trend is that AI is moving from a tool used to tune the network into a workload the network is built to carry, and the next generation is being specified that way from the start. IMT-2030, the ITU framework for 6G, lists artificial intelligence and communication among its six usage scenarios and ubiquitous intelligence among its four design principles [**itu2023imt2030**]. The architecture forming around that is AI-RAN, in which telecommunications and AI workloads are orchestrated together on shared, accelerated infrastructure, so the sites that run the radio also host distributed edge AI platforms [**polese2026airan**]. The AI-RAN Alliance states the operational requirement directly: one common infrastructure has to run both kinds of workload concurrently, and their service-level agreements may differ [**airan2025orchestrator**]. That changes what a language choice has to buy for a control plane. Its software no longer has the host to itself. It contends for CPU, memory, and cache with inference work whose arrival pattern it does not control, and a co-tenant that takes capacity away lowers the load at which the control plane saturates. Saturation is where the two languages separate most sharply in the measurements of Chapter **??**. Memory footprint and behaviour past that point are becoming selection criteria in their own right, and neither is settled by the architecture above the code.

The third trend is an industry-wide shift toward memory-safe languages. Large software organisations report that the majority of their serious security vulnerabilities stem from memory-safety errors, and that adopting memory-safe languages measurably reduces them [**msrc2019memory**, **google2022android**, **vanderstoep2025android**, **google2024securebydesign**]. The case no longer rests on vendor reporting alone: a group spanning academia and industry has argued that memory safety now needs standardised terminology and requirements if it is to be specified and procured at all [**watson2025memorysafety**]. This concern has risen to the level of national policy: cybersecurity agencies now urge software manufacturers to publish "memory-safe roadmaps" and to migrate away from memory-unsafe languages such as C and C++ [**cisa2023memorysafe**, **oncd2024building**]. In the market this subsystem is sold into it has also become a regulatory matter. The EU Cyber Resilience Act places secure-

by-design and lifecycle vulnerability-handling obligations on manufacturers of products with digital elements, with the vulnerability-reporting obligations applying from September 2026 and most of the remainder from December 2027 [**eu2024cra**]. It names no programming language, and it would be wrong to read it as requiring one. What it changes is that the recurring defect classes of a memory-unsafe language become a compliance exposure as well as an engineering one. Rust in particular has moved from a niche language into operating-system kernels, cloud infrastructure, and increasingly into telecommunications. Production hypervisors such as AWS Firecracker, which underpins its serverless platform, are already written in Rust [**agache2020firecracker**]. Yet the evidence base for Rust's suitability rests largely on general-purpose software and on average-case performance benchmarks. The specific question a telecom vendor must answer, whether Rust can meet the tail-latency and predictability requirements of a RAN control plane, whether it genuinely prevents the defect classes that plague the existing C++ code, and what it costs to adopt it incrementally inside a large C++ codebase, has not been answered with direct, controlled evidence. Providing such evidence for a representative subsystem is the motivation for this work.

Industrial setting. This thesis is carried out at Nokia, one of the small number of vendors that build, maintain, and evolve production RAN software end to end. That market is concentrated in very few hands: the five largest suppliers took 96% of worldwide RAN revenue over the first three quarters of 2025, with concentration by Herfindahl–Hirschman Index at a ten-year high [**delloro2025ran3q**, **delloro2026ran**]. The subsystem it studies, UE Context Management (UECM), is modelled on a real gNB-CU-CP control-plane component rather than a textbook example.

The gNB-CU-CP is not a minor component. In the 3GPP disaggregated architecture it is the control-plane half of the central unit: it hosts RRC and manages the UE context lifecycle, delegating user-plane bearer handling to the CU-UP over the E1 interface [**3gpp38401**, **3gpp38300**, **3gpp38331**, **3gpp38463**]. It does so under the millisecond-scale procedure budgets the 5G system places on control signalling [**3gpp38913**], and at the very high event rates and connected-device counts a deployed network sustains. Because it is written in C++, it inherits both that language's performance and its memory-safety and concurrency hazards. That is precisely where the three trends above converge.

Software of this class is shipped, evolved, and operated across live operator networks. A language-selection decision here is therefore a concrete production concern rather than a hypothetical one, which is what makes a controlled comparison on a representative subsystem valuable to a vendor.

UECM is a deliberate choice of study object within that setting. It is small enough to reimplement faithfully and to reason about in full, yet it exercises exactly the properties that make RAN control-plane code hard: per-UE mutable state, a strict operation lifecycle, high concurrency across many UEs, and hard tail-latency expectations. Conclusions drawn from it therefore transfer to the broader class of stateful, concurrent control-plane services. They also remain reproducible by readers outside Nokia, because the implementation is driven entirely by a public, frozen specification rather than by proprietary

code.

A decision with a generational horizon. A language choice in software of this class is not revisited often, and it outlives the generation it was made for. The next generation is being specified now: 3GPP has allocated Release 20 to the 6G studies and Release 21 to the normative work, against a calendar set by the ITU radiocommunication sector that asks for technology proposals in early 2029 and the submitted specifications by mid-2030 [**itu2023imt2030**, **3gpp2024road6g**]. That work explicitly covers migration from and interworking with 5G, so a control plane written for the current generation is not retired when the next one arrives but maintained alongside it. The practical question for a vendor is therefore not only which language suits a 5G control plane today, but which one it can still afford to be maintaining a decade from now.

Nothing in the next generation's stated objectives relaxes the requirements that motivate this comparison. The IMT-2030 framework names security and resilience as one of four design principles common to all of its usage scenarios, states that the system is expected to be secure by design, and defines resilience as the ability to keep operating correctly during and after a disruptive event. It also carries the ultra-reliable low-latency scenario forward in strengthened form, as hyper-reliable and low-latency communication [**itu2023imt2030**]. Freedom from whole classes of defect and predictable behaviour when the system is disturbed are thus being written into the objectives of the next generation rather than left to implementers, and both depend on the language and its libraries as much as on the architecture above them.

The policy clock falls inside the same window, and the roadmap expectation noted above carries a date keyed to precisely this horizon. CISA and the FBI classify the development of new product lines in a memory-unsafe language as a bad practice that significantly elevates risk where the product serves critical infrastructure and a memory-safe alternative is available, and they exempt an existing product from the roadmap expectation only when its announced support ends before January 2030 [**cisa2025badpractices**]. Mobile-network infrastructure sits squarely inside that scope, and the Cyber Resilience Act deadlines noted above fall earlier still [**eu2024cra**]. A vendor that defers the language question until the next generation is specified will not thereby have avoided making it. It will have made it by default, in the incumbent language, for another decade of deployments.

What this thesis contributes to that decision is bounded, and the boundary is worth stating plainly. The measurements here are taken on a subsystem modelled on a current-generation component and evaluated against requirements that are already published. No claim is made about requirements that have not yet been written. What crosses the generational boundary is not a latency figure but the language-level comparison itself: which defect classes a compiler can rule out, what the safety model costs, and what incremental adoption inside a large C++ codebase involves. Those questions have the same answers whichever generation the specification above them names, and they are cheapest to answer before the next codebase is started rather than after.

A concrete failure mode. To make the stakes concrete, consider a mobility storm in which many UEs hand over at once. A single mishandled pointer, a UE context freed on release while an in-flight signalling message still references it, produces a use-after-free. In C++ this compiles cleanly and may pass ordinary testing, only to corrupt an unrelated UE's state or crash the worker thread under precisely the high-concurrency conditions the system exists to handle. The same latent race can equally show up as an unpredictable tail-latency spike instead of an outright crash. In safe Rust the same design does not compile: ownership and borrowing reject the dangling reference at build time. This one scenario captures why the thesis measures both *safety* (which defects are prevented, and when) and *predictability* (tail latency under overload) rather than average throughput alone.

1.2 Problem statement

Two properties of this domain make the language choice consequential. First, manual memory and concurrency management in C++ exposes the software to defects such as use-after-free, double-free, data races, and iterator invalidation, which are silent at compile time and costly to find and fix at telecom scale. Second, what governs whether RAN deadlines are met is not average throughput but worst-case and tail behaviour under load and overload [**dean2013tail**], a property that is easy to regress and hard to measure correctly. Rust promises to remove the first class of defect at compile time while claiming performance competitive on the second, but that claim rests mostly on general-purpose benchmarks rather than on stateful, concurrent, latency-critical telecom workloads.

Bringing these threads together, the concrete problem this thesis addresses is the following. Telecom vendors are under growing pressure, both technical and increasingly regulatory, to adopt memory-safe languages, yet for the RAN control plane they lack *direct, controlled* evidence on which to base that decision. It is not established whether a Rust implementation of a stateful, concurrent control-plane subsystem can match modern C++ in *tail latency and predictability under overload* (as opposed to average throughput), whether Rust's compile-time guarantees actually eliminate the specific defect classes that affect the existing C++ code, and what incremental adoption *costs* inside a large C++ codebase. The available evidence is largely general-purpose and average-case [**balasubramanian2017rust**, **qin2020rust**], leaving these three questions (predictability, prevented-defect scope, and migration cost) open for a representative subsystem. Answering them under controlled, reproducible conditions is the problem this thesis sets out to solve.

1.3 Research questions and hypotheses

The thesis is organised around four research questions, ordered to put predictability and migration ahead of raw average performance:

RQ1 How do Rust and modern C++ compare in latency predictability (median, p99 and p99.9 tail latency, and the maximum) for a RAN control-plane workload under increasing load and overload?

RQ2 Which classes of memory-safety and concurrency defects that occur in C++ are eliminated at compile time in the Rust implementation, and at what stage and cost are the remaining ones detected?

RQ3 What is the overhead, if any, of Rust's safety model (bounds checks, ownership-driven copies, abstraction cost) on throughput, CPU utilisation, and memory behaviour?

RQ4 What are the practical implications (effort, FFI interoperability, and risk) of adopting Rust incrementally within an existing C++ RAN software ecosystem?

The questions are approached with three prior expectations, stated here so that the evaluation can confirm or refute them instead of merely describing results after the fact. **H1:** for this workload the two languages are performance-equivalent in the operationally relevant region below saturation, with any difference confined to the far tail. **H2:** safe Rust prevents at compile time the memory-safety and concurrency defect classes that C++ leaves to best-effort, after-the-fact detection. **H3:** the run-time cost of C++-to-Rust interoperability is negligible relative to the cost of the operations invoked, so that the true cost of incremental adoption is engineering effort rather than performance. H2 is stated so that it can fail. The defect catalogue behind RQ2 spans classes safe Rust is expected to prevent and classes it is expected not to, so the study can locate the boundary of the guarantee instead of only confirming it. These hypotheses are revisited against the evidence in Section **??**.

1.4 Contributions, results, and limitations

The thesis makes five contributions. Two are artifacts: a reproducible benchmarking framework for telecom control-plane workloads, and a frozen shared specification with two parity-tested implementations of a RAN control-plane subsystem, one in C++ and one in Rust. Two are results: a quantitative comparison that weighs tail latency and predictability alongside average throughput, and a defect-class taxonomy from controlled fault injection that marks where Rust's compile-time guarantee stops. The fifth is a set of engineering and migration guidelines for language selection in cloud-native RAN evolution (Section **??**).

The findings divide three ways. On *safety* (RQ2), every injected memory-safety and data-race defect class is prevented at compile time in safe Rust. The same defects all compile in C++, and each is caught, if at all, only at run time and only under the right sanitiser, or on one class under none of them. Seven boundary classes were chosen deliberately, and neither language prevents them. On two of those the C++ tooling is ahead, and on one the defect that safe Rust rules out returns as soon as the program writes `unsafe`. On *migration* (RQ4), the FFI seam is small and keeps the core free of `unsafe`. Measured against the same operation called natively it costs at most 10.91% of one operation, so the real adoption cost is API design and not run-time overhead. Three costs beyond the seam are measured for the same reason, and two of them come back against the received wisdom: most of the Rust build's extra cost is compiling third-party code and not the

compiler's own work, and the binary-size ordering reverses once the two toolchains are held to one linkage policy. Both of these results are final.

On *performance* (RQ1, RQ3) the answer divides at saturation, and drawing that division correctly is part of the contribution. Below saturation the two implementations are equivalent end to end in central behaviour, by a margin no deployment would notice. The paired test does resolve a median difference of 1.6 μs in Rust's favour under both allocators, with an interval reaching 2 μs, so the equivalence is practical and not statistical. The far tail in the same regime cannot be resolved on the available host at all. Neither result is as informative as it looks. An end-to-end operation here is only 0.250% language work and the rest is a cross-thread handoff, so the sweep bounds the design and not the language. Measured on its own, the operation is 1.6× faster in Rust, while the Rust process holds a few MiB more resident memory under both allocators. Past saturation the ordering reverses. The C++ build sustains 1.8× the throughput of the Rust build, and uses more CPU for each completed operation.

Four limitations bound all of this: a synthetic workload, one simplified subsystem, single-author implementation bias, and a virtualised measurement host. Chapter **??** treats each in detail.

The scope is drawn deliberately to protect the internal validity of the language comparison. In scope are the UE Context Management functionality of Chapter **??**, the shared-nothing concurrency design, the load sweep to overload, the fault-injection catalogue, and the FFI prototype. Explicitly out of scope are the full gNB system and its remaining control-plane domains, PHY/L2 processing and the data-heavy user plane, real captured traffic traces (as opposed to a spec-justified synthetic workload), managed-language alternatives (excluded on the tail-latency grounds argued in Chapter **??**), and production deployment and operational concerns such as upgrade, observability, and multi-tenancy. These exclusions trade external validity for the controlled conditions a fair language comparison needs, and each is revisited as a limitation in Chapter **??**.

1.5 Significance and beneficiaries

The results are intended to be directly actionable. For RAN vendors and cloud-native network operators, the thesis supplies controlled evidence on the question that gates any language decision. That question is not "is Rust fast on average?" but "does Rust hold its tail latency and predictability under overload, and what does incremental adoption cost?", answered for a subsystem representative of production control-plane code. For the standards and policy discussion around memory-safe roadmaps [**cisa2023memorysafe**, **oncd2024building**], it grounds a general mandate in a concrete telecom data point. For the research community, the frozen specification, the parity-tested implementations, the reproducible benchmarking framework, and the defect-class taxonomy form a reusable baseline for further language and systems studies. And for practising engineers, the fault catalogue and the migration prototype double as worked examples of what Rust's guarantees do and do not buy in this domain.

2 Background: Systems Languages for Latency-Critical Software

2.1 RAN control-plane systems and the gNB-CU-CP

The NG-RAN architecture splits the gNB into a central unit (gNB-CU) and distributed units, with the control plane (gNB-CU-CP) handling RRC and connection management for each UE [**3gpp38401**, **3gpp38300**, **3gpp38331**]. Such control-plane systems must satisfy demanding non-functional requirements: very high event rates (UE mobility, signalling storms), strict and predictable latency bounds that the 5G service and RAN specifications quantify as KPIs [**3gpp38913**, **3gpp22261**], long-lived software with continuously evolving complexity, and heavily concurrent workloads. In these systems the worst-case and tail behaviour, rather than the average, determines whether deadlines are met, which motivates the predictability focus of this thesis.

Functionally, the gNB is disaggregated along two independent axes. The first is a *central/distributed* split: a central unit (gNB-CU) hosts the higher layers of the protocol stack, while one or more distributed units (gNB-DU) host the lower, more time-critical layers close to the radio. The second is a *control/user-plane* split inside the central unit, separating the control-plane function (gNB-CU-CP) from the user-plane function (gNB-CU-UP). These parts communicate over standardised interfaces: NG towards the 5G core, F1 between the CU and the DU, and E1 between the CU-CP and the CU-UP [**3gpp38401**, **3gpp38300**, **3gpp38463**]. Open-RAN initiatives standardise these interfaces further so that each part can be supplied by a different vendor and run as software on commodity hardware [**polese2023oran**]. Figure **??** places the subsystem studied in this thesis, UE Context Management (UECM), inside the gNB-CU-CP: it owns the per-UE control-plane state and mediates the RRC signalling and mobility procedures, which is why its correctness and timing behaviour are representative of the control plane as a whole.

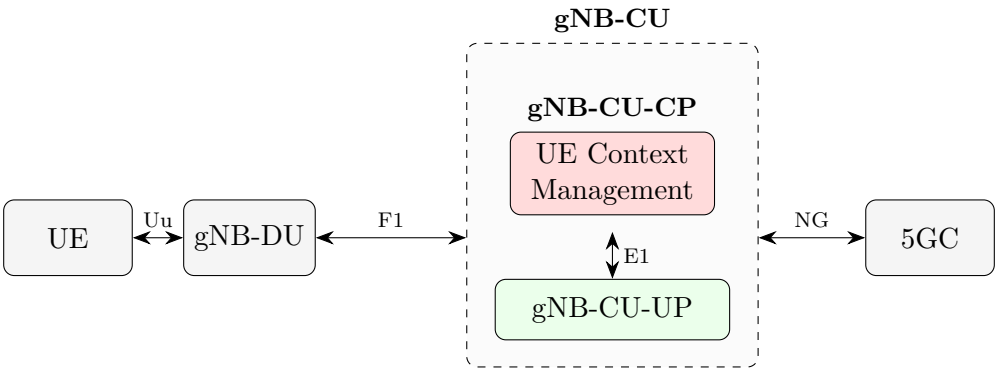


Figure 2.1: Where the studied subsystem sits in the 5G NG-RAN. The gNB is split into a distributed unit (gNB-DU) and a central unit (gNB-CU), and the central unit is split again into control-plane (gNB-CU-CP) and user-plane (gNB-CU-UP) functions, connected by the standardised F1, E1, and NG interfaces. UE Context Management (highlighted) is the per-UE control-plane state manager inside the gNB-CU-CP that this thesis reimplements and measures.

2.2 Latency, throughput, and predictability in real-time software

Latency-critical systems are judged less by how much work they do on average than by how *predictably* they do it. Three metrics must be distinguished. *Throughput* is the sustained rate of completed operations, *latency* is the time an individual operation takes, and *jitter* is the variability of that latency. A system can have excellent mean latency and still miss deadlines often. The meaningful figures are therefore the high-percentile *tail* latencies, the 99th (p99) and 99.9th (p99.9) percentiles, rather than the mean. At the scale of a network, tail latency dominates user-visible behaviour: when a single user action fans out into many internal operations, the slowest of them sets the response time, so rare per-operation delays become common per-request delays [**dean2013tail**]. Modern data-centre and networking software increasingly operates in a regime where the relevant latencies are measured in microseconds, a range in which conventional operating-system and software overheads are no longer negligible [**barroso2017microseconds**]. Fine-grained measurement studies show that, even at moderate utilisation, hardware, operating-system, and application-level effects such as background interference, scheduling, interrupt routing, and power-saving states inflate the tail far beyond what queueing models predict [**li2014tales**]. Figure **??** sets the two effects side by side: the distance between the median and the high percentiles, and the fan-out that turns a delay which is rare per operation into one that is common per procedure.

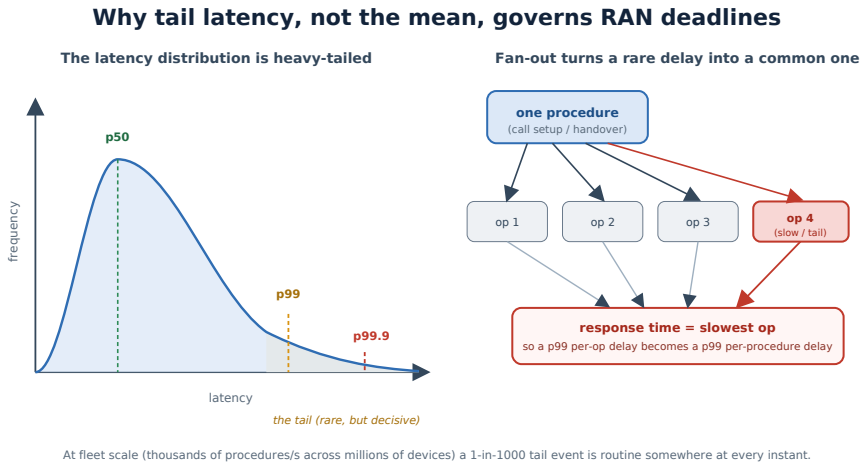


Figure 2.2: Why the tail, not the mean, governs RAN deadlines. On the left, most operations finish quickly and a few take far longer, so the p99 and p99.9 percentiles sit well to the right of the median. On the right, a procedure such as a call setup runs several operations and finishes only when the slowest one does, so a delay hitting one operation in a hundred reaches a much larger share of procedures. At fleet scale a one-in-a-thousand delay is happening somewhere at every instant.

Real-time systems are conventionally graded by the consequence of a missed deadline: in *hard* real-time a miss is a failure, in *firm* real-time a late result is useless but not catastrophic, and in *soft* real-time a miss merely degrades quality. A RAN control plane is best seen as soft-to-firm real-time: an occasional late signalling response degrades service rather than endangering it, but at fleet scale and under signalling storms such degradations accumulate into a network-wide reliability problem. This is compounded

by queueing behaviour: as offered load approaches service capacity, queueing delay grows sharply and the latency tail explodes near saturation. Characterising a system therefore requires driving it not just at nominal load but *into overload*, which is why the evaluation in this thesis sweeps load up to and beyond the saturation point.

Measuring tail latency correctly is itself a pitfall. A naive closed-loop load generator that waits for each response before issuing the next request suffers *coordinated omission*: when the system stalls, the generator stalls with it and simply fails to record the many requests that would have arrived during the stall, systematically hiding the very tail it is meant to expose. Figure **??** contrasts the two generator models. The distinction beneath this is a modelling one, not an implementation detail. In a closed system a new request is triggered only by the completion of a previous one, whereas in an open system arrivals are independent of completions, and the two produce systematically different response-time behaviour on the same service process, with the closed model reporting the lower figures [**schroeder2006openclosed**]. Sound measurement therefore requires an open-loop model with an intended schedule, correction for coordinated omission, high-dynamic-range latency histograms that retain resolution across many orders of magnitude, and statistically rigorous reporting with repeated runs and confidence intervals rather than single point estimates [**georges2007rigorous**]. These techniques are applied in Chapter **??**.

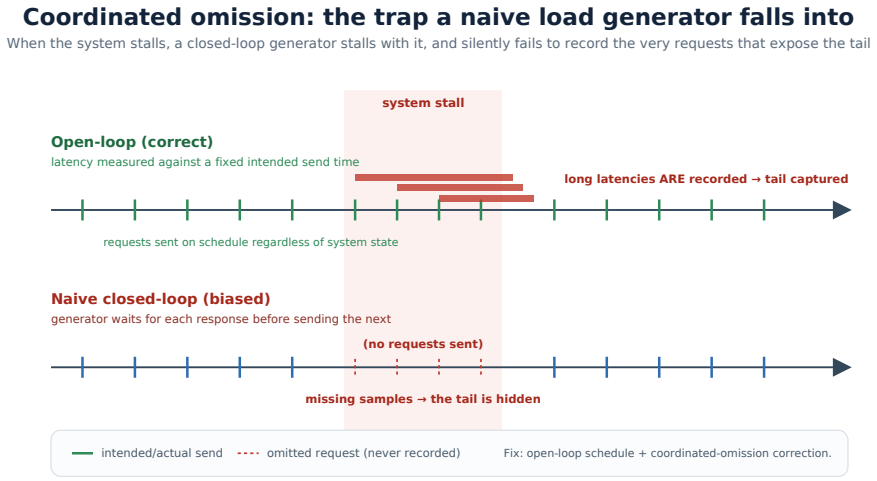


Figure 2.3: Coordinated omission. The open-loop generator at the top sends on a fixed timetable even while the system is stalled, and times each request from the moment it was due, so stalled requests are recorded with their full delay and the tail shows up. The closed-loop generator at the bottom sends the next request only after the previous reply arrives. When the system stalls it stops sending too, so the slowest requests are never issued and never measured, and the tail vanishes from the data.

2.3 C++ for latency-critical systems

C++ is the default language for RAN control-plane software for good reasons. It offers zero-overhead abstractions, direct control over memory layout and allocation, deterministic destruction through the Resource Acquisition Is Initialization (RAII) idiom, and mature compilers, libraries, and profiling tools, all without a garbage collector that could inject unpredictable pauses. Modern C++ (C++11 and later) adds smart pointers

(`std::unique_ptr`, `std::shared_ptr`) that automate ownership, move semantics that avoid needless copies, and a formal memory model with `std::atomic` for lock-based and lock-free concurrency. These features make it possible to write fast and, with discipline, reasonably safe systems code.

The difficulty is that the compiler does not enforce this safety. The programmer does. RAII and smart pointers reduce lifetime errors without eliminating them. A raw pointer or reference can outlive the object it points to, a container mutation can invalidate an iterator, an object can be used after it has been moved from, and nothing in the type system stops a mutable object being shared and written by several threads at once. The result is a familiar catalogue of defects, use-after-free, double-free, buffer overflow, data races, deadlocks, and iterator invalidation, that are easy to introduce and expensive to find at telecom scale. Memory-corruption errors of this kind have been a persistent security problem for decades, and deployed mitigations remain only partial defences [**szekeres2013sok**]. Their prevalence is well documented in industry. Memory-safety issues account for roughly 70 % of the security vulnerabilities reported at Microsoft [**msrc2019memory**], and Google reports a sharp drop in such vulnerabilities as memory-safe languages are adopted in Android, to below a fifth of that platform's total vulnerabilities by 2025 [**google2022android**, **vanderstoep2025android**]. This specific catalogue is developed in detail in Chapter **??** and forms the basis for the fault-injection study in Chapter **??**.

C++ practice mitigates these defects with tooling rather than with language guarantees, and the tooling is inherently partial. Dynamic sanitisers such as AddressSanitizer (ASan), ThreadSanitizer (TSan), and UndefinedBehaviorSanitizer detect many errors, but only on the code paths that a given test actually executes with the right inputs, and they impose runtime and memory overheads that make them unsuitable for production. TSan in particular reports only those data races that happen to occur during a run, so a race that manifests only under a rare interleaving can pass unnoticed. Its deadlock detector inherits that limitation from the technique it implements, which infers deadlock potentials from the lock-acquisition graph of a single trace [**bensalem2005deadlock**], and it ships as an experimental component covering only the POSIX mutex families [**tsandeadlock**]. Static analysers help but suffer false negatives and cannot reason about the whole program. Detection is therefore best-effort and after the fact, which is precisely the situation a language with compile-time guarantees is meant to change.

That describes C++ as it can be deployed today, and the language is not standing still. Since 2023 the standards committee has pursued *profiles*: named sets of rules covering type, bounds, lifetime, and arithmetic safety that a conforming implementation must enforce wherever they are enabled [**sutter2025profiles**]. That direction was chosen over *Safe C++*, a proposal for a safe subset with borrow checking and its own standard library, which is Rust's model expressed in C++ [**baxter2024safecpp**]. The comparison drawn here nonetheless stays live. Profiles are not yet standardised, and in January 2026 their own authors were still calling for implementations and treating the design of the initial profiles as work still to be done [**vandevoorde2026profiles**]. The concurrency profile, the one that would bear on the defect classes catalogued in Chapter **??**, is placed after that initial set and not within it: "more profiles should be defined and implemented,

notably a concurrency profile" [**vandevoorde2026profiles**]. A rule enforced only when it is switched on is also a different kind of thing from a property the type system carries unconditionally, especially where the enforcement is a run-time check and not a compile-time rejection. The implementations compared here are therefore built against C++23 as specified and shipped, and Section **??** records what that pins the comparison to.

2.4 The Rust ownership and safety model

Rust enforces memory safety and data-race freedom at compile time through its ownership and borrowing model [**klabnik2023rust**], and ships with modern tooling in Cargo and integrated testing. The type system statically forbids mutation of shared state and confines the unavoidable exceptions to `unsafe` code kept behind safe APIs [**jung2021safe**].

These guarantees rest on more than assertion. The soundness of the safe fragment of the language, including the role of `unsafe` as an explicitly marked escape hatch, has been established formally by the RustBelt project [**jung2018rustbelt**], and the aliasing discipline that even `unsafe` code must respect has been given a precise operational semantics [**jung2020stacked**]. Empirical evidence supports the model in practice: in real codebases `unsafe` is used sparingly and in localised ways [**qin2020rust**, **astrauskas2020unsafe**], and a study of all Rust memory-safety CVEs found that every one of them but a single compiler bug involved `unsafe` code [**xu2021rustcve**]. The caveat is that unsafety propagates through dependencies. Fewer than a third of libraries use `unsafe` directly, but a much larger fraction cannot be fully statically checked because it hides somewhere in their call chain [**evans2020rust**], which is a further argument for keeping the safety-critical core of a subsystem `unsafe`-free, as this thesis does.

One property matters most in a latency-critical setting. This safety comes from the type system and the borrow checker, not from runtime instrumentation, and is intended to be *zero-cost*. Systems research argues Rust attains safety without the overhead of garbage collection or runtime checks, and even enables efficiency techniques that are impractical in C [**balasubramanian2017rust**]. Whether that zero-cost claim holds for a telecom control-plane workload is precisely the property this thesis evaluates.

Concretely, the borrow checker rejects at compile time the very use-after-free sketched in the introduction: a program that frees a context while a reference to it is still live does not build. Figure **??** puts that defect in its UE Context Management form beside the C++ version, which compiles and is caught, if at all, only at runtime.

The same discipline extends to concurrency. Rust encodes thread-safety in the type system through the `Send` and `Sync` marker traits: a value may cross a thread boundary only if its type is `Send`, and may be shared by reference across threads only if it is `Sync`. Sharing a value that permits unsynchronised mutation is therefore a compile error, not a latent race. This is the property Rust markets as "fearless concurrency." Error handling is likewise explicit: fallible operations return a `Result` value that the caller must handle, rather than raising an exception that unwinds through hidden control flow. Both the cost and the control flow of the failure path therefore stay visible, which is an advantage in latency-sensitive code where surprises in the tail are what matter.

Rust is not the only memory-safe language, and the reason the comparison here is drawn

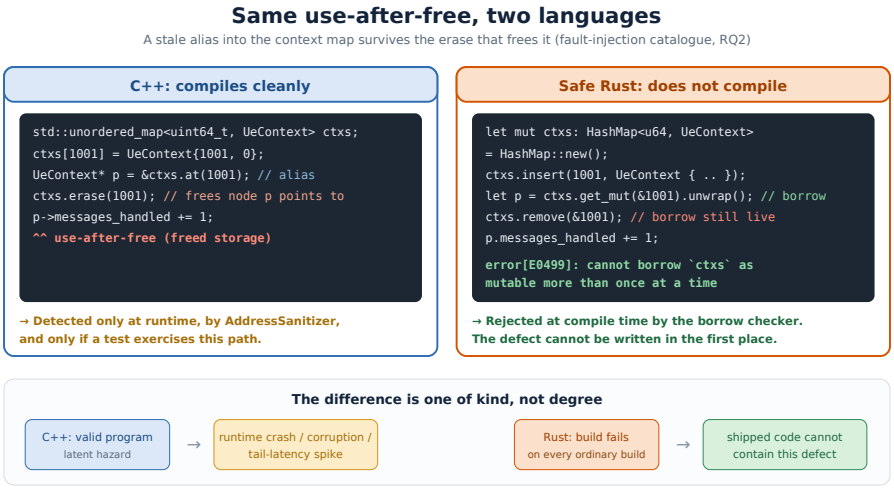


Figure 2.4: The same use-after-free defect in C++ and in safe Rust. The stale alias into the context map survives the `erase`/`remove` that frees it: in C++ this compiles and is caught only at runtime (and only under AddressSanitizer, on a path a test exercises), whereas in safe Rust the borrow checker rejects it at compile time (`E0499`), so the defect cannot be written at all. The difference is one of kind, not degree.

against C++ rather than against a managed runtime belongs with this model. Java, C# and Go achieve memory safety through a runtime and garbage collection, which is an excellent trade for much software and a poor one for a tail-latency-critical control plane: collection pauses are a recognised source of tail latency in large systems [**dean2013tail**], and a managed runtime gives up the control over memory layout and scheduling this domain relies on. C, at the other extreme, is as memory-unsafe as C++ while offering fewer abstractions for building large systems. Rust is the first widely supported language to combine compile-time memory and thread safety with the absence of a garbage collector and throughput and memory use alongside C and C++ [**balasubramanian2017rust**, **jung2021safe**, **bugden2022rust**]. That is why managed languages are out of scope and why the meaningful comparison for RAN control-plane software is C++ against Rust.

Table **??** sets the two languages' mechanisms side by side. The distinction it is built around is not which hazards each language names, since both name much the same ones, but *when* and *by what* each is ruled out, and the last two rows mark where the answer stops being "by the compiler" for either of them.

Table 2.1: How each language enforces the properties this thesis measures. The rows above the rule are the ones on which the two differ in kind rather than in degree: a compiler that rejects a program rules the defect out of every execution of it, whereas a sanitiser rules it out only of the executions an instrumented test happens to take. The rows below the rule are the boundary developed in Chapter **??** and measured in Chapter **??**, where neither language offers a static guarantee.

Concern	Modern C++	Rust
Object lifetime	RAII and smart pointers, applied by convention. A raw pointer or reference may outlive its referent	Ownership: each value has one owner and is dropped exactly once
Aliasing	Unrestricted: any number of mutable aliases may coexist	Borrowing: either one mutable reference or many shared ones, never both
Bounds	Unchecked on `operator[]`, checked only through `at()`	Checked by default, elided where the index can be proved in range
Thread safety	Not expressed in the type system: sharing a mutable object across threads compiles	`Send` and `Sync` marker traits. Unsynchronised sharing is a type error
Error signalling	Exceptions or error codes: the failure path can be invisible at the call site	`Result` values the caller must handle. The failure path is explicit
Escape hatch	The whole language	`unsafe` blocks, delimited and mechanically countable
Stage of detection	Run time, under the right sanitiser, on a path a test exercises	Compile time, on every ordinary build, whatever the test coverage
Lock-acquisition order	Not checked	Not checked
Atomicity of a composed operation	Not checked	Not checked

2.5 Concurrency models for control-plane software

How concurrency is structured shapes both correctness and tail latency. The dominant model is *shared-memory* concurrency, in which threads mutate common state guarded by mutexes and atomics. It is flexible, but it is the source of races, deadlocks, lock contention, and priority inversion, and its worst case is hard to bound. The alternative is *message-passing* or *shared-nothing* concurrency, in which state is partitioned so that each piece is owned by a single thread and all interaction happens through messages, as in the actor model. It trades flexibility for scalability and much simpler reasoning, and it maps onto per-UE partitioning naturally, since each UE's state is largely independent. The choice also moves the quantity measured here: thread-per-core structuring changes application tail latency on its own [**enberg2019threadpercore**], so the model is held fixed across both implementations instead of left to each language's idiom. Figure **??** contrasts the two.

A third structure, an asynchronous runtime over a work-stealing scheduler, is idiomatic

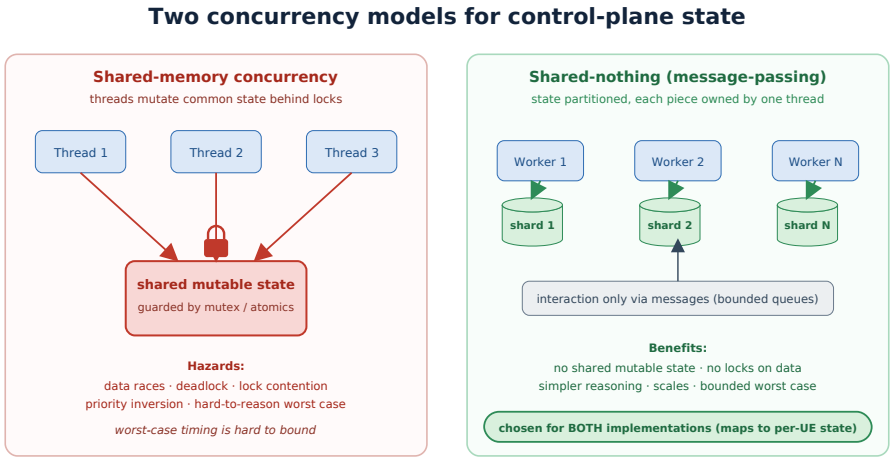


Figure 2.5: Two concurrency models for control-plane state. Shared-memory concurrency (left) lets threads mutate common state behind locks and is the source of data races, deadlock, and lock contention, with a worst case that is hard to bound. Shared-nothing message-passing (right) partitions state so each shard is owned by a single worker and all interaction goes through bounded queues, which removes locks on context data and maps onto per-UE partitioning. It is the model adopted for both implementations.

in both languages and used in neither. Its timing behaviour belongs to the runtime, not to the language hosting it: a real-time study of asynchronous Rust finds one application meeting or missing its deadlines depending on the runtime and thread-priority mapping it is given, and the configuration that meets them is not the idiomatic work-stealing one [**skoudlil2025asyncrust**]. Adopting such a runtime would import the variable this study exists to exclude, and Section **??** returns to a comparison in which it was not excluded.

Queueing between producers and consumers is a second, independent choice. Unbounded queues mask overload and let latency and memory grow without limit. *Bounded* queues, whether they block for backpressure or shed, bound worst-case memory and make overload visible, which predictable tail latency requires. Both choices fix the architecture used by both implementations in Chapter **??**: a shared-nothing, thread-per-shard design in which UE state is divided into independent slices called *shards*, each owned by exactly one worker thread, so no two threads ever touch the same context. Requests reach a shard through a bounded multi-producer, single-consumer queue that blocks when full.

2.6 Interoperability between C++ and Rust

Adopting a new language in a large existing system is necessarily incremental: a Rust component must coexist with and be callable from a substantial C++ codebase. The common denominator for that is the C application binary interface (ABI). Rust can expose functions with `extern "C"` linkage and hand out *opaque handles* to Rust-owned objects, which C++ calls through a C header. Tooling automates parts of this: `bindgen` (Rust bindings from C headers), `cbindgen` (C headers from Rust), and the `cxx` crate (a safer bidirectional bridge). Interoperability is not free. Crossing the boundary steps outside the checked language, so the seam must contain `unsafe` code, values beyond the simplest need marshalling, and the foreign interface has to be designed deliberately. Seam size,

the `unsafe` it concentrates, and per-call crossing cost are the quantities RQ4 measures in Chapter **??**.

One distinction bounds what those measurements can establish. Concentrating `unsafe` in the seam says what the compiler checks, not what can happen at run time. A Rust component linked into a C++ host shares one address space with it, so a defect on the C++ side can corrupt Rust-owned data however much of the Rust side the borrow checker verified. This has been shown experimentally, and the proposed remedies isolate the two heaps in hardware or in the runtime instead of trusting the discipline of the interface [**rivera2021galeed**, **kirth2022pkrusafe**]. The defect classes specific to Rust/C boundaries have been catalogued separately from those of either language [**li2022crosslang**], and the binding generators themselves carry bugs [**cai2026interop**]. The consequence is stated once here and honoured throughout: an `unsafe`-free core means the core's own defects are ruled out at compile time, and it does not mean the core is protected from its host.

What has moved recently is the reach of the tooling, not the guarantee. A stateless model checker for Rust now explores the state space of a program together with its foreign dependencies, which puts data races arising from the interaction of the two languages within reach of a checker even though they sit outside what the borrow checker sees [**pearce2026rustmc**]. That capability arrived after the prototype measured here was built, and Chapter **??** records it as the next step for the seam.

2.7 Research gap

The preceding sections set up the gap. The RAN control plane is a soft-to-firm real-time, highly concurrent domain where tail behaviour under overload, not average throughput, governs the quality the network delivers. C++ supplies the control and performance the domain demands, but leaves whole classes of memory and concurrency defects to best-effort detection after the fact. Rust promises to prevent those classes at compile time at no run-time cost, and offers a credible incremental adoption path through the C ABI.

The domain is not unstudied, and the volume of prior work is easy to underestimate because almost none of it is indexed. At least five vendor-hosted master's theses, all carried out at Ericsson, have compared candidate languages for RAN and telecom software. One weighs runtime cost against development efficiency across C++, Rust, and Go [**karlback2021ran**]. One asks whether Rust could replace C in RAN node management [**olsson2023ranmgmt**]. One builds a thread-per-core packet processor on asynchronous Rust and measures it against the DPDK application it mirrors [**olsson2023asyncpackets**]. One measures median and 99th-percentile latency for control-plane procedures across five asynchronous programming frameworks [**arvidsson2025async**]. The last builds matched C++ and Rust implementations of an ASN.1 decompiler and compares them statistically [**emberg2023asn1**]. All five are directly relevant, and two report differences unfavourable to Rust.

Three of them vary the language together with the runtime and the concurrency model, so a measured difference cannot be attributed to the language rather than to the framework built on it [**karlback2021ran**, **olsson2023asyncpackets**, **arvidsson2025async**].

The effect sizes reach a factor of five, which is an indication of how much of the measurement the runtime owns. The remaining two hold the design closer to fixed but report execution time and peak memory rather than tail behaviour under rising load [**olsson2023ranmgmt**, **emberg2023asn1**], and the closest of them, Emberg and Wadsten, measures a stateless transformation rather than a stateful, concurrent subsystem. Figure **??** sets the neighbouring studies against the properties a controlled answer would require, and the pattern it shows is that each holds some of them and none holds all.

The concentration of that evidence is itself part of the gap. All five studies come from one vendor, and their conclusions have never been tested against another organisation's software, targets or conventions, so what is known about this question is what one company's frameworks report about it. The published record on the other side is emptier: there is no study of Rust among the indexed publications of the vendor hosting this work, and the nearest peer-reviewed position from that side argues for making memory-error sanitisation practical rather than for replacing it with a compile-time guarantee [**gjomemo2016mesc**], which is the strategy this thesis tests against. Section **??** compares the five with this study directly.

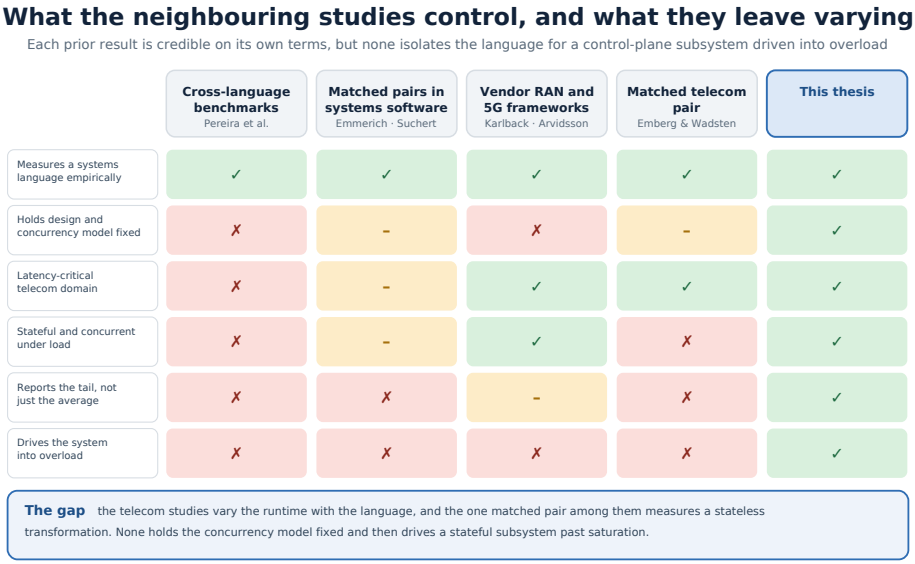


Figure 2.6: What the neighbouring studies control, and what they leave varying. Each row is a property that a controlled answer to the research questions would need. Each column is a study. The general cross-language comparisons measure the right kind of object but the wrong regime, reporting averages on general-purpose workloads. Matched pairs in systems software hold the design fixed, but outside this domain and below saturation. The vendor RAN and 5G framework studies reach the right domain, and one of them reports the tail, but in all of them the language varies together with the runtime and the concurrency model. The one matched telecom pair, Emberg and Wadsten, controls the design but measures a stateless transformation. Holding the design and the concurrency model fixed for a stateful, concurrent subsystem and then driving it past saturation is what this thesis adds. The contribution is the control, not a new measurement technique.

What is therefore missing from the literature is *direct, controlled* evidence for a representative control-plane subsystem: whether Rust actually holds its tail latency and predictability under overload against modern C++ *when the concurrency model, algorithms, and data structures are held fixed*, exactly which defect classes it eliminates and at what

stage the remainder are caught, what its safety model costs in throughput and resources, and what incremental adoption costs in practice. Closing this gap is the purpose of the four research questions. The next chapter grounds them in the specific subsystem, defect classes, and timing properties at stake, and Chapter **??** sets out the methodology by which they are pursued.

3 Problem Context: UE Context Management and Predictability

3.1 The UE Context Management subsystem

The subsystem under study is a UE Context Management service. It is modelled on a real Nokia gNB-CU-CP service domain that owns the UE context lifecycle, RRC procedure execution [**3gpp38331**], and mobility/handover. Those are the very functions the 3GPP architecture assigns to the control-plane part of a disaggregated gNB: the CU-CP hosts the RRC and the control-plane part of PDCP and manages the UE context, delegating user-plane bearer handling to the CU-UP over the E1 interface [**3gpp38401**, **3gpp38300**, **3gpp38463**].

Figure **??** shows the UE-context lifecycle state machine that both implementations encode identically. This state machine is exactly what the mixed workload of Section **??** walks each UE through.

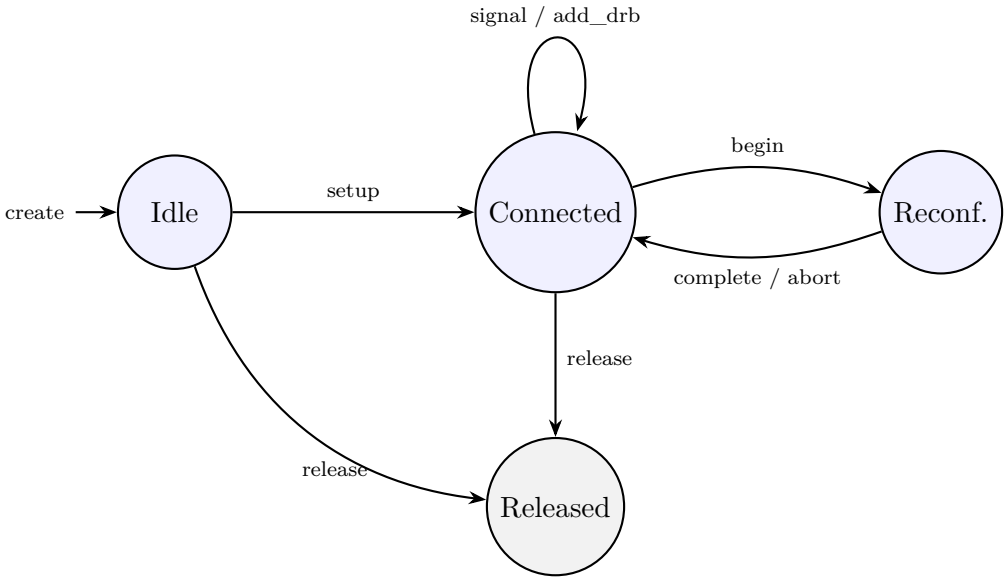


Figure 3.1: UE-context lifecycle state machine. `create` allocates a context in `Idle` and `setup` connects it. `signal` and `add_drb` are handled in place while `Connected`. A reconfiguration or handover moves through `Reconfiguring` and back on `complete` or `abort`, and `release` tears the context down. Operations invalid for the current state return an `OpError` instead of changing state.

Why this subsystem. UECM works as a study object because it is both representative and small enough to control. It is core RAN layer-3 functionality, and it carries the properties that make production control-plane code difficult: per-UE mutable state, a strict operation lifecycle, high concurrency across UEs, and the latency and reliability targets the 5G system sets for control-plane procedures [**3gpp38913**, **3gpp22261**]. Conclusions about how a language handles those properties therefore transfer to other stateful, concurrent control-plane services. The subsystem is also small enough to specify precisely,

implement faithfully in two languages, and reason about in full, which is what makes a parity-tested comparison possible at all. A larger subsystem would buy external validity at the cost of the internal validity a fair language comparison depends on. UECM sits where both stay within reach.

Operation set. The interface exercised by the study is deliberately small and lifecycle-oriented. It can create and release a context, set up a connection (`Idle→Connected`), handle an in-session signalling message, add a data radio bearer, begin, complete, or abort a reconfiguration or handover, and look up a context's state under concurrent load. Every operation is defined for every state of the machine in Figure **??**: an operation that is illegal in the current state returns a typed error and leaves the context unchanged, rather than throwing or corrupting state. This closed, well-typed operation set is what lets the same test vectors and the same workload drive both implementations along identical paths (Section **??**).

3.2 Memory-safety and concurrency defect classes

The defects that affect C++ control-plane code of this kind fall into two families. The *memory-safety* family concerns the lifetime and bounds of objects: a *use-after-free* accesses storage that has already been released, a *double-free* releases the same storage twice, and a *buffer overflow* reads or writes past the end of an allocation. The *concurrency* family concerns unsynchronised or ill-ordered access from multiple threads: a *data race* is a pair of unsynchronised accesses to the same location of which at least one is a write, a *deadlock* is a cyclic wait for locks, and *iterator invalidation* is the use of an iterator or reference into a container after a mutation has moved or freed the underlying storage. What these share is that they are all *silent* in C++: each is accepted by the compiler and, on the paths ordinary tests happen to exercise, may appear to work, surfacing only later as corruption, a crash, or an unpredictable latency spike. These are the same memory-corruption classes that decades of security research have struggled to contain in C and C++ [**szekeres2013sok**].

Each class has a natural incarnation in UE Context Management, which is why the subsystem is a good vehicle for studying them:

- **Use-after-free.** A signalling message holds a raw pointer or reference to a UE context that a concurrent `release` has already erased from the context map. The in-flight handler then dereferences freed storage.
- **Double-free.** A release path runs twice for the same UE, for example because a retransmitted release is not made idempotent, freeing the context's owned resources a second time.
- **Iterator invalidation.** Code iterates a context's data-radio-bearer list while adding or removing a bearer, so the iterator refers into storage the container has since reallocated.

- **Data race.** Two workers update the same context field, an activity counter or a state flag, without synchronisation, so the accesses race.
- **Unguarded shared mutation.** A context is shared by reference across threads and mutated without a lock or other discipline, a race whose root cause is the absence of any ownership rule for the shared state.
- **Use-after-move.** Admission moves a freshly built context into the store and then answers the request from the local variable it has just moved from, so the acknowledgement carries an empty identity. This class is carried for what the C++ tooling does *not* say about it. Nothing is corrupted and nothing is undefined, because a moved-from object is in a valid but unspecified state, so the defect has no runtime symptom for a sanitiser to find.
- **Thread-unsafe handle shared across workers.** A refcounted context handle whose counter is not atomic, the idiom a hot path reaches for when the atomic control block of a shared pointer costs more than it is worth, is passed to two shard workers. The constraint that makes the handle correct is written in a comment, because C++ has no way to state thread affinity in a type.

These seven classes lie squarely within the guarantees Rust claims to provide, which makes them the natural test of whether that claim holds. Testing only them, however, would build the expected answer into the choice of test cases, so seven further classes are carried into the study for the opposite reason, because there is good reason to expect that safe Rust does *not* prevent them:

- **Lock-order deadlock.** A handover locks the source UE context and then the target while a concurrent handover in the opposite direction locks target then source, and the two wait for each other indefinitely. Nothing about object lifetimes or aliasing is violated. The fault is in the order of acquisition.
- **Lost update.** Two workers increment a context's activity counter through a read-modify-write split across two separate critical sections. Every individual access is correctly synchronised, so this is a *race condition* rather than a *data race*: the atomicity violation lives in the composition of the operations, not in the memory accesses themselves, and it corrupts a value silently instead of crashing.
- **Leak through a reference cycle.** A context and the peer it is being handed over to hold owning references to each other, so neither reference count reaches zero and neither is ever destroyed. Reference counting cannot collect a cycle in either language, and leaking is safe in Rust by explicit design. This is the cleanest statement of a boundary in the catalogue, because both languages fail identically.
- **Integer overflow in a release build.** A bearer's TEID stride computation exceeds the width of its type. In C++ signed overflow is *undefined*, so the program has no meaning. In Rust it is *defined* to wrap, because the `opt-level = 2` profile this study compares leaves `overflow-checks` off, so the program has a meaning

and the meaning is wrong. The pattern is not hypothetical: CVE-2026-64467 in `rust_binder` is a stride error of this shape.

- **Worker failure blast radius.** One shard worker fails part-way through a procedure. The concurrency model of Section **??** makes a shard a failure domain by construction, and neither language's type system enforces that it behaves as one: in C++ an exception escaping the thread callable terminates the whole process, and in Rust the panic is confined to its thread but poisons the state it held, so the damage propagates along the data instead. Directly relevant to a carrier-grade availability argument, and addressed by nothing in either type system.

- **Double-free across the seam.** A C++ host holds an opaque handle to a Rust context and releases it twice, which is the retransmitted release of the double-free class above, now expressed the way a foreign-function boundary has to express it. Safe Rust rejects the double release because ownership tracks the value. A seam cannot keep that, since handing a raw pointer to a C caller is exactly the operation that ends the compiler's knowledge of it. The class therefore appears on both sides of the boundary, which is the sharpest available statement of what the boundary is.

- **Forgotten lock guard.** Bearer setup takes the context lock and a validation failure returns without releasing it, so the next worker to touch that context waits indefinitely. Rust removes the ordinary form of this bug, because a guard releases when it is dropped and there is no hand-written unlock to skip. It does not remove the bug, because a guard is a value and leaking a value is safe. Where the reference cycle leaks memory, this leaks the context itself.

Together these fourteen form the catalogue injected in Chapter **??**, and the split between the two groups is what lets the study report a boundary rather than a verdict. Buffer overflow is described above for completeness but is not injected, because neither implementation performs raw pointer arithmetic on the context store, so an injected instance would be an artificial construct and not a plausible bug in this subsystem. Table **??** collects the catalogue with the outcome expected of each case *before* the study was run, which is the form in which the runner records it: an expectation written down in advance can be violated in either direction and reported as a discrepancy, whereas one formed after the fact cannot.

Table 3.1: The injected defect catalogue: each class in the form it would plausibly take in UE Context Management, and the outcome recorded for it in advance of the study. The last column is a prediction rather than a result. Table **??** reports what was measured, and the fault runner flags a mismatch in either direction instead of absorbing it into the conclusion. Buffer overflow is listed for completeness and not injected: neither implementation does pointer arithmetic on the context store.

Defect class	Incarnation in UE Context Management	How it fails	Expected in safe Rust
Memory-safety family: object lifetime and bounds			
Use-after-free	An in-flight signalling handler dereferences a context that a concurrent `release` has already erased from the map	Corruption or crash	Prevented
Double-free	A retransmitted release is not made idempotent, so the release path frees the context's resources a second time	Corruption or crash	Prevented
Use-after-move	Admission answers the request from the context it has just moved into the store	Silently empty identity	Prevented
Buffer overflow	No plausible incarnation here: the context store is reached only through container APIs	n/a	Not injected
Concurrency family: unsynchronised or ill-ordered access			
Iterator invalidation	A bearer is added while the context's data-radio-bearer list is being iterated	Corruption or crash	Prevented
Data race	Two workers update the same context field, a counter or a state flag, without synchronisation	Torn or lost value	Prevented
Unguarded shared mutation	A context is shared across threads and mutated with no ownership rule governing it	Torn or lost value	Prevented
Thread-unsafe handle shared	A context handle whose refcount is not atomic is passed to two shard workers	Corrupted refcount	Prevented
Boundary group: included because safe Rust is expected not *to prevent them*			
Lock-order deadlock	Opposing handovers lock source-then-target and target-then-source	Hang	Not prevented
Lost update	A counter's read-modify-write is split across two critical sections, so each access is synchronised but their composition is not atomic	Silently wrong value	Not prevented
Reference-cycle leak	A context and its handover peer hold owning references to each other, so neither reference count reaches zero	Unbounded growth	Not prevented
Integer overflow (release build)	A bearer's TEID stride computation exceeds the width of its type	Silently wrong value	Not prevented
Worker-failure blast radius	One shard worker fails part-way through a procedure while holding shard state	Outage beyond one shard	Not prevented
Double-free across the seam	A C++ host releases an opaque handle to a Rust context twice	Corruption or crash	Not prevented
Forgotten lock guard	Bearer setup returns on a validation failure without releasing the context lock	Hang	Not prevented

3.3 Tail latency and predictability in RAN

Section **??** set out why the tail rather than the mean governs a latency-critical system, and what measuring it correctly demands. What that section left general, this one makes specific, because two properties of the RAN setting are what turn a methodological pref-

erence into the governing metric.

Those properties are fan-out and scale, drawn in Figure **??**. One user-visible action, a call setup or a handover, becomes many dependent control-plane operations whose slowest member gates completion, and a busy site drives thousands of such procedures per second across millions of connected devices. Put together, the p99.9 of a single UE Context Management operation is not a curiosity at the margin of a distribution. It is a rate of degraded procedures the network sustains continuously.

The scale at which those percentiles matter is not hypothetical. A measurement study of a disaggregated 5G RAN times the CU-CP-to-DU UE Context Setup procedure over F1-C, and the CU-CP-to-CU-UP bearer context setup over E1, under different interface-protection schemes, and finds the choice worth hundreds of microseconds: on F1-C, IPsec adds on the order of 120 μs on average to UE context setup, against roughly 430 μs for DTLS [**michaelides2026ranlatency**]. Overheads of that size are treated as consequential for precisely the procedure family this thesis reimplements, which places these operations in a regime where tens of microseconds are an engineering quantity rather than noise.

The study therefore characterises the full latency distribution, focusing on the high percentiles (p99 and p99.9) and the maximum, and on how these degrade as offered load approaches and then exceeds saturation (Chapter **??**). Stopping at nominal load would miss the regime that decides service quality, since queueing delay grows super-linearly near capacity. It is also the regime in which a difference between two implementations of one design, if there is one, has room to become visible.

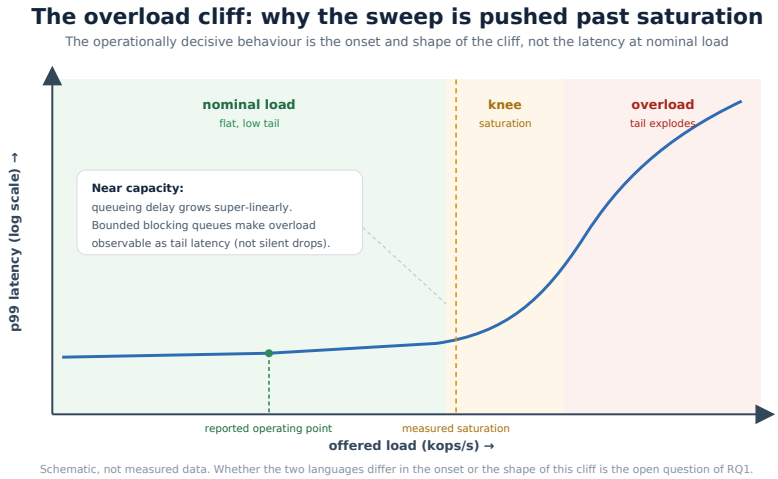


Figure 3.2: The overload cliff. Tail latency stays low and flat while offered load is below service capacity, then climbs steeply once the system is driven past saturation and requests queue behind the bounded, blocking queues. The operationally decisive behaviour is the onset and shape of this cliff, which is why the load sweep is pushed beyond the saturation point rather than stopping at nominal load. The figure is a schematic of the phenomenon and not a plot of this study's data: whether the two languages differ in where the cliff begins or how steeply it rises is exactly what RQ1 asks, and Section **??** reports that the development host cannot resolve it.

4 Research Methodology

The previous chapter set out what the study must achieve. This one sets out how. It states the overall research strategy, the method chosen for each research question and why it is appropriate, the controls that make a language comparison fair, the statistical treatment of the measurements, and the reproducibility and ethical commitments under which the work is carried out.

4.1 Research approach

The study is experimental with a constructive (design-science) element. The artifact is a pair of functionally equivalent implementations of the subsystem, one in C++ and one in Rust, plus the framework that benchmarks them. Both are built from a single frozen specification and then evaluated under controlled experiments. Evidence is triangulated across three methods. Performance measurement under a load sweep answers RQ1 and RQ3, a controlled fault-injection study answers RQ2, and an FFI migration case study, part measured and part qualitative, answers RQ4. The three are independent. A weakness in one of them, such as the environment sensitivity of tail-latency measurement, therefore does not carry into the conclusions as a whole. Every measurement reports variance over repeated runs, and Section **??** sets out the controls that keep the language the only variable.

That design follows from the shape of the questions. Each asks how one outcome (tail latency, defects prevented, resource use, migration cost) depends on one variable, the implementation language. The language therefore has to be the only systematic difference between the two objects under test. This rules out the commonest form of language comparison in the literature, where two independently written programs are measured and the gap is attributed to the language. Such a gap also carries the design, the libraries and the skill of their authors.

Building both sides from one specification removes that confound. The comparison is then between two encodings of the same design, and the artifact becomes both the contribution and the instrument that measures it. Where a question can be answered with numbers, the treatment is quantitative. Where it cannot, as with the kind of guarantee in RQ2 and the API-design effort in RQ4, it is qualitative but still systematic.

The gap this design addresses (Section **??**) is only as good as the search that looked for it, so that search is on record. Appendix **??** gives the databases, the term combinations, the citation chaining and the filtering rules. Two of its findings bear on the design and not just on the bibliography.

The vendor-hosted theses closest to this study (Section **??**) appear in no citation database. Chaining therefore never reaches them, and they had to be found by searching the institutional repositories directly. The same searches establish an absence on the other side: no Finnish thesis on Rust in a radio-access or telecom context, and no indexed publication mentioning Rust from the vendor hosting this work. The domain-specific literature on

the language question is therefore Swedish and Ericsson-hosted, and whether it carries over here is untested. Those theses are grey literature, so they are cited as evidence of what a vendor has attempted and found, not as peer-reviewed results.

The search returned no study, indexed or grey, that measures the tail-latency behaviour of a Rust and a C++ implementation of the same stateful, concurrent control-plane subsystem under overload.

Table **??** maps each research question to the method, artifact, data and metric that answer it. All four chains end in the same place: Section **??** judges each question and Section **??** disposes of its hypothesis. RQ1 and RQ3 share H1, worded to cover performance equivalence in both the latency and the resource sense. That wording turns out to be the hypothesis's main weakness. RQ2 is insensitive to the environment, and RQ4 is so except along one axis: build time is a timing measurement like any other, and is therefore read as a ratio between two toolchains measured alternately on one machine. RQ1 and RQ3 depend on the measurement host and are reported against what it can carry (Section **??**).

Table 4.1: Research design: how each research question is answered. Every question is tied to a concrete method, a published artifact, the data it produces, and the metric on which the conclusion rests. RQ1 and RQ3 carry two methods rather than one: the sweep measures the round-trip a deployment would experience, and the component decomposition measures the part of it the language is responsible for. Section **??** states why one cannot stand in for the other. RQ1 and RQ3 are tested against H1, RQ2 against H2, and RQ4 against H3.

RQ	Method	Artifact / data	Primary metric
RQ1	Load sweep to overload, interleaved repeats	Benchmark harness, latency histograms	median, p99/p99.9 tail latency, and maximum vs. offered load
RQ3	Same sweep, resource capture	`getrusage` samples per run	peak RSS, CPU utilisation, throughput
RQ1, RQ3	Component decomposition: the core called directly, then through the service	Matched microbenchmark pair, interleaved repeats	per-operation cost split into the operation and the cross-thread handoff
RQ2	Controlled fault injection	Defect catalogue, ASan/TSan/UBSan, `rustc`, Loom/Shuttle and Miri logs	defect-class taxonomy (compile-time / runtime / undetected)
RQ4	FFI migration prototype	C++ host + Rust C-ABI seam	seam size, `unsafe` scope, per-call interop cost

4.2 Methods for the four research questions

Table **??** pairs each question with a method.

4.2.1 Performance measurement under a load sweep (RQ1, RQ3)

RQ1 and RQ3 are answered by driving both implementations with one deterministic synthetic workload across a sweep of offered load, from light load through saturation into overload. Two design decisions make the measurement sound. The load generator follows an *open-loop* schedule: each request has a fixed intended send time, and latency is measured against that time and not against the moment of admission. That corrects for coordinated omission as Chapter **??** defines it. The sweep is pushed past saturation deliberately, because the overload cliff appears only once offered load exceeds service capacity (Section **??**). Latency goes into high-dynamic-range histograms configured identically on both sides, and resource use (peak resident set size, CPU utilisation, throughput) is sampled through the operating system.

Measuring the component as well as the round-trip. A round-trip bundles two costs of different character. One is the operation the language runs inside a single thread: a hash lookup, a state-machine guard, a field update, all compiler-generated. The other is the cross-thread handoff around it: an enqueue, a futex wake, a context switch, a completion, none of it language-specific. Only the first can plausibly differ between C++ and Rust, and no end-to-end measurement separates them.

The method therefore works at two levels. The operation processor is called directly, with no queue in the path, then through the full threaded service. The difference between the two is the handoff. Both levels are batch means over a long loop and not per-call medians, because a clock read costs tens of nanoseconds, the same order as the operation itself, and per-call timestamps would measure the instrument. The languages are interleaved across repeats and compared with the same exact paired test as the sweep, so the levels differ in what they measure and not in how they are analysed.

Reporting both separates two questions the research questions did not distinguish as originally worded: what a deployment of this shape would experience, and what the language is doing. Section **??** finds the two more than two orders of magnitude apart. The distinction therefore decides whether the result can be interpreted at all. The second level was added after a first pass of the campaign, and Section **??** records that revision.

Isolating the contended synchronisation primitives. The overload comparison in Section **??** attributes its gap to two forced-idiom entries where the standard libraries diverge. They are the bounded blocking queue and the one-shot completion handle. That attribution is argued from the shape of the full-service measurement, so a third matched microbenchmark drives those two primitives alone, with no `ShardCore` work in the path, under the same interleaved repeats and paired test. Section **??** gives its configuration and reports whether the isolated primitives reproduce the ordering the full service showed.

4.2.2 Controlled fault injection (RQ2)

RQ2 asks which defect classes a language prevents. Measuring defect-free code cannot answer that: the absence of a bug in a passing test says nothing about whether the language would have caught it. The method is therefore adversarial, following the established practice of software fault injection, where faults are introduced deliberately to see how a system behaves in their presence [**natella2016fault**].

A catalogue of representative C++ defects is written *deliberately*, drawn from the classes that affect this subsystem (Chapter **??**). Each is a small self-contained program flavoured with the domain: a stale pointer into the context map, a double release, a mutation during iteration, an unsynchronised shared update. Each is built and run under AddressSanitizer, ThreadSanitizer and UndefinedBehaviorSanitizer, recording which tool detects it, if any, and at what stage.

The same construct is then written in safe Rust and compiled. The compiler's outcome is recorded, and where it compiles, so is the program's behaviour. The boundary classes give the compiler nothing to say, so the Rust side also goes to two model checkers, Loom and Shuttle, and for the class that lives inside an `unsafe` block to the Miri interpreter. These answer the "at what stage and at what cost" half of RQ2 for them, and each is paired with a control in which the defect is fixed, so that finding the defect and reporting everything are not the same observation.

Whether ThreadSanitizer reports the lock-order inversion depends on the schedule, so a single run would record whichever outcome the machine happened to give. The runner executes that case twelve times and reports the rate, so intermittent detection is not mistaken for reliable detection.

Why the C++ side is tooled with sanitisers only. No static analyser is run against the C++ catalogue, and that is a decision. RQ2 does not ask which tool finds the most defects. It asks at what *stage*, and with what force, each language rules a defect out, and the taxonomy's three outcomes (compile-time rejection, runtime detection, silent miss) are the categories that question needs. An opt-in pass belongs to none of them: a warning a build can be configured to ignore is a different kind of object from a compilation that cannot succeed, and that difference is what the method measures.

The cost is a C++ side narrower than the best a C++ project could manage, since a well-configured analyser could plausibly reach the small intra-procedural memory-safety cases, though not the concurrency ones. The study therefore reports the tooling it ran, the sanitiser-based practice of Section **??**, and not a ceiling on what C++ tooling can achieve.

The injected fault is held fixed while the language varies, so any difference in outcome comes from the language's guarantees and not from the specific bug. What the method establishes is the *kind* of protection each language offers, not a performance number.

One design decision determines whether the taxonomy can be trusted. A catalogue drawn only from classes the language is known to prevent could confirm the hypothesis but never disturb it, and would describe the author's choice of cases as much as the language. The

catalogue is therefore built in two groups: classes inside Rust's stated guarantee, and classes just outside it, where ownership and the `Send`/`Sync` discipline give no protection. Each case carries a recorded expectation and the runner reports a violation in either direction, so an unexpected outcome surfaces as a discrepancy instead of being absorbed into the narrative. What this buys is a boundary and not a score: the study can state which classes the language removes and which comparable ones it leaves in place, which is what a migration decision needs.

4.2.3 Foreign-function-interface migration case study (RQ4)

RQ4 concerns the practical cost of adopting Rust inside an existing C++ system, an engineering question as much as a performance one. It is answered with a prototype built in the realistic direction of migration: a C++ host calling a Rust component across a C application binary interface. Three quantities are measured on it. The *size* of the seam counts its entry points, lines of code and dependencies. The *scope* of `unsafe` records where the checked language is left and whether the safety-critical core is affected. The *per-call cost* is the price of crossing the boundary, measured against the same operation invoked natively so the comparison isolates what the seam adds. The first two are read off the code, the third is a matched pair of microbenchmarks. Pairing them separates the run-time cost of interoperability from its true cost, which lies in API design and human learning.

Interoperability is one of the three terms RQ4 names, and a prototype answers it alone. Three further quantities cover the other two, chosen because a build engineer reaches them before anything the seam can show. *Build time* is measured clean and incrementally, for the core library alone and for every default target, with the same job count and interleaved repeats on both sides so that one host's load falls on both toolchains equally. *Static footprint* is measured on the conformance runner, the one executable both languages build from the same specification, and at both linkage policies instead of at each toolchain's default, because the two defaults differ in whether the standard library is linked in and a single pair of numbers would report that choice as a language difference. *Third-party surface* counts the transitive crates each configuration resolves and the `unsafe` sites inside them, which makes the dependency-free claim of Section **??** checkable. The first is host-sensitive and the other two are not.

4.3 Fairness as a controlled-variable strategy

Every research question attributes a difference to the language, so validity depends on holding everything else equal. Fairness is therefore the study's central control, enforced at several levels.

The *specification* is frozen before any code is written. It fixes the public API, the state machine, the data structures and their algorithmic strategy, and the sharding and concurrency model (Section **??**). The *run-time substrate* is equalised by linking the same allocator on both sides and matching build-optimisation levels as closely as the toolchains allow. The *workload* comes from a driver identical in structure across languages that, for a given seed and active-UE population, drives both implementations through the same se-

quence of operations. Any latency difference therefore reflects how each language executes the same work, not a difference in what work is done.

Where the two languages genuinely cannot express the same construct, the deviation is recorded in a forced-idiom log. What remains uncontrolled, principally single-author skill asymmetry, is acknowledged as a threat to validity (Chapter **??**) and not claimed to be eliminated.

Table **??** states the strategy variable by variable, and its third column matters most. A controlled experiment is credible in proportion to how honestly it accounts for what it failed to control. Three rows record a residual and not a clean fix: the languages cannot always express the same construct, they do not ship the same hash-map layout, and one person wrote both implementations. The forced-idiom log handles the first, and the last row names the one variable left with no control at all.

Table 4.2: The controlled-variable strategy: what is held fixed so that the language remains the only systematic difference, and what each control does not reach. The third column is not a list of oversights but the study's own account of its limits. Each entry is picked up again as a threat to validity in Section **??**. Only the last row names a variable for which there is no control at all.

Variable	How it is held fixed	What the control does not reach
Design	One specification, frozen before any code is written, fixing the public API, the state machine, the data structures and their algorithmic strategy, and how state is divided into shards (§**??**)	Where the two languages cannot express the same construct, the divergence is logged, not removed (Table **??**)
Concurrency model	Thread-per-shard over bounded blocking MPSC queues on both sides, with no asynchronous runtime on either	Nothing outstanding
Hashing	SplitMix64 for shard selection and FNV-1a inside the shard, pinned identically, so neither standard library's default hasher becomes an accidental advantage	The container itself is still each language's own hash map, and the two differ in layout (§**??**)
Allocator	The same `jemalloc` linked on both sides, with a system-allocator build kept on each as a control	Nothing outstanding
Optimisation	`-O2` against `opt-level = 2`	Nominal parity is not identical code generation
Workload	One deterministic driver. For a given seed and active-UE population both implementations execute the same operation sequence, checked by per-kind counts	Synthetic and spec-justified rather than replayed from production traffic
Instrumentation	The same HDR histogram algorithm and configuration, the same open-loop schedule, the same repeat and interleaving discipline	Nothing outstanding
Behaviour	A language-neutral conformance suite whose canonical transcripts are diffed op for op (§**??**)	Parity bounds behaviour, not performance
Author	No control	One person wrote both implementations and is not equally expert in both

4.4 Statistical treatment of measurements

Performance measurements are non-deterministic, so single-run numbers are avoided throughout. Each configuration (language × allocator × load level) is measured over many independent repeats, and the four variants are *interleaved* within each repeat so that slow drift, thermal effects, and background activity perturb them equally instead of biasing one. Results are summarised by the median with a nonparametric bootstrap 95% confidence interval [**efron1993bootstrap**] rather than by the mean with a standard deviation, because latency distributions are heavy-tailed and skewed and a distribution-free interval on the median makes no normality assumption. This follows established guidance on statistically rigorous performance evaluation, which warns that reporting a single number invites conclusions that run-to-run variation does not support [**georges2007rigorous**]. Tail percentiles (p99, p99.9, p99.99) are read from the HDR histograms, whose logarithmic bucketing retains three significant figures of resolution across the whole range so that high percentiles are not quantised away. Intervals are reported in full, not as a symmetric half-width, because a bootstrap interval on the median of a heavy-tailed sample is markedly asymmetric and a half-width wide enough to cover its upper bound would place the lower bound below zero.

Two of these quantities need their own note, because how far into the tail a campaign can see is a property of how many operations it recorded. At the representative load a single run retires on the order of 170 thousand measured operations, so its p99.99 rests on roughly seventeen observations: a per-run p99.99 is dominated by sampling noise even on a perfectly quiet host. Reading the far tail at all therefore requires persisting each run's complete HDR bucket contents alongside the summary percentiles, so that the repeats can be *pooled* into one distribution of several million operations first. The harness does that now, and the design keeps pooling and averaging apart, because they are different statistics. A pooled figure would take the operation as the unit of analysis and resolve the far tail, at the cost of carrying no across-repeat interval, while the median-of-repeats figures reported with intervals keep the repeat as the unit and remain the only basis for a paired comparison. Jitter would likewise be measured directly rather than inferred from the latency distribution, as the absolute difference between consecutive latencies within a producer. That is the packet-delay-variation quantity of RFC 3393 [**rfc3393**], and it matches the sense in which this thesis uses the term: variation from one operation to the next, not spread of the distribution as a whole. Neither is reported in Chapter **??**, because the campaign analysed there predates the instrumentation (Section **??**).

Neither p99.99 nor jitter would enter the family of paired tests described below in any case. That follows the same rule as the rest of the design rather than being a consequence of the missing data: both are computed from precisely the operations that interference lands on, which is the class of statistic pairing does not rescue, so a paired interval on them could not be read as a language effect. They belong in a descriptive column, and the family stays the size the design committed to.

Interval estimates alone, however, are a weak basis for a comparison. Two intervals can overlap while one variant is ahead in every single repeat, and they can separate on a difference that does not reproduce. The study therefore also tests the comparison directly,

and the interleaved design is what makes this possible: because the four variants are measured back to back inside each repeat, the runs form a matched design in which the interference common to a block cancels in the pairwise difference. Language is compared against language with the allocator held fixed, using an exact two-sided paired permutation test on the repeat-wise differences. The null hypothesis is that a pair differs only by run-to-run noise, under which the sign of each paired difference is exchangeable. For the repeat counts used here every one of the 2^n sign assignments is enumerated rather than sampled, so the resulting p-value is exact and, like the bootstrap, assumes nothing about the shape of the distribution. Because the same comparison is made for several metrics and both allocators, the family of tests is corrected with the Holm step-down procedure [**holm1979simple**], which controls the family-wise error rate without assuming independence between the tests.

Two decisions about how that family is read are taken here rather than after the results, because either could otherwise be made to suit whatever the data turned out to be.

The first is which comparisons enter it, and the answer turns on the operating point rather than on the metric. Below saturation every variant completes work as fast as it is offered, so achieved throughput there is pinned to the offered load by construction and cannot express a language effect whatever the languages do. It is reported with its interval instead of tested. Above saturation that reasoning inverts, because what each variant still sustains is pinned to nothing and becomes the one quantity that can separate them, so a second operating point is fixed in the overload region and sustained throughput and CPU utilisation are tested at it (Section **??**). Two commitments go with that. The overload comparisons join the same Holm family rather than forming a second one, so the division into two operating points cannot be read as a split chosen once the p-values were known. And no latency percentile is tested above saturation at all, because a producer that cannot keep up with a fixed intended schedule falls behind cumulatively, which makes the percentiles there a lower bound and not an estimate. Appendix **??** sets out the reasoning behind both.

The second is what to conclude when the interval and the p-value disagree. At modest repeat counts they routinely will: a paired interval can sit wholly on one side of zero while the corrected p-value stays far above any conventional threshold. Which instrument to believe is not a free choice, and it does not depend on whether the metric happens to be interesting. It depends on whether pairing removes the interference that particular measurement is exposed to. Pairing cancels whatever an interleaved block shared. It cannot cancel interference that landed on one run of a pair and not on the other. A run-level summary such as peak resident set size is unmoved by a handful of descheduled operations, and so is a central order statistic such as the median of tens of thousands of per-operation samples. For those, a paired difference that keeps its sign across repeats is a difference in the software. A high percentile is the opposite case: p99 and p99.9 are computed from precisely the operations that were disturbed, so an interval excluding zero is as consistent with one run of a pair having been unlucky as with a real effect. Table **??** is therefore blocked by that distinction, and intervals in its second block are not read as findings however far from zero they sit. Applying the rule to the block that suits the argument and quietly dropping it for the other is the specific failure this pre-commitment

exists to prevent.

The resolving power the design can reach is bounded by the number of repeats, and is stated rather than left implicit. With 20 repeats and 14 comparisons the exact test's two-sided floor is 0.0000019, and 0.000027 after correction, so a null reported here reflects the data and not the arithmetic. Quoting both is what lets a reader tell a null result from an unresolvable one. Choosing the repeat count so that the corrected floor sits below the conventional 5% is therefore part of the measurement protocol, alongside pinning the clock and isolating the cores (Section **??**). Appendix **??** derives the two floors.

4.5 Reproducibility and research artifacts

The study is designed so that its results can be regenerated by a third party without access to Nokia's proprietary code. Three things make that possible. The implementations are driven entirely by the public frozen specification. The benchmarking harness, the fault catalogue and its runner, and the FFI prototype are published together with the thesis. And every figure and table in the evaluation is produced by a single script directly from the raw measurement logs, so that rerunning the campaign on a different host regenerates the artifacts verbatim.

The execution environment is captured automatically alongside each result set: CPU model, kernel, compiler and toolchain versions, and the frequency and scheduling settings of the testbed. The conditions of a measurement therefore travel with its data. That matters for one distinction in particular. Separating environment-sensitive results (the performance sweep) from environment-insensitive ones (parity testing, fault injection, and the FFI structure) is what lets the thesis state, per result, which of the two a reader is looking at. Nothing on the path requires proprietary source, which is what makes third-party reproduction possible at all rather than merely permitted.

5 Design and Implementation

5.1 Shared specification: the fairness baseline

Before any code was written, one specification was frozen, and nothing in it names a language. It fixes the public API, the UE state machine, the data structures and the algorithm behind them (a sharded hash map keyed by UE identifier, for instance), the granularity of locking and sharding, and the concurrency model. Both implementations follow it. That is what keeps the comparison a comparison of two languages, and not of two designs or two levels of skill. The specification is a deliverable in its own right, and it is the main control against implementation bias.

5.2 Conceptual model and concurrency design

Two implementations are built against the shared specification, and they do the same job: a modern C++ version (C++23, STL, RAII, standard threading) and a Rust version (stable toolchain, ownership-based memory model, safe concurrency). Both use one concurrency model, described as *shared-nothing, thread-per-shard*, which is how real RAN L3 divides UE state. Async and thread-based models are never mixed, and that is deliberate.

The UE-identifier space is split into N_{shards} shards, and each shard belongs to exactly one worker thread. Eight is the default. The measurement campaign pins seven instead, so that every worker and every producer owns one of the pinned cores, as Section **??** records. Only the owning worker reads or changes its shard's contexts. Workers therefore share no mutable state, and context data needs no locks. Callers (the load-generator threads) never touch context state directly. They submit request messages to a shard's bounded multi-producer, single-consumer (MPSC) queue, then wait on a single-use completion handle that the worker writes the result into. The queue is *bounded and blocking*. When a shard's queue is full, submission blocks until space frees up. That is what produces realistic queueing latency under overload, and it is also what makes tail latency observable at all, because a policy that dropped requests instead of blocking would hide the overload. Requests to the same shard run strictly first-in-first-out.

This structure is what gives the safe-concurrency comparison something to measure. The machinery under test is the queue, the completion handoff, and the ownership transfer of each request, not lock contention. One piece of shared mutable state is allowed: an atomic counter that stamps a per-context activity sequence number. It is write-only bookkeeping, and no control flow branches on its value. Latency is measured across the whole path, from intended send time to completion, so it includes queueing delay under load and covers the same cross-thread round-trip on both sides. Section **??** later splits that figure into the part the language executes and the part the operating system does. Figure **??** shows the data flow, and Figure **??** traces one operation through it as a sequence.

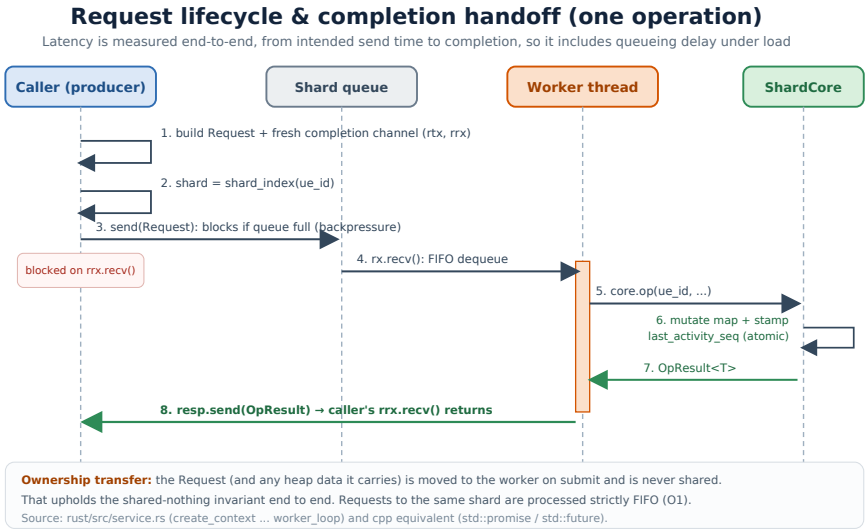


Figure 5.1: The end-to-end path of a single operation and the completion handoff. The caller builds a request with a single-use completion handle, routes it by `shard_index(ue_id)` to the owning shard's bounded queue (blocking if it is full), and waits. The shard's single worker dequeues in FIFO order, executes the operation against its exclusively owned store, and writes the result back to the completion handle. Ownership of the request is transferred to the worker on submit and never shared. Latency is measured across the whole path.

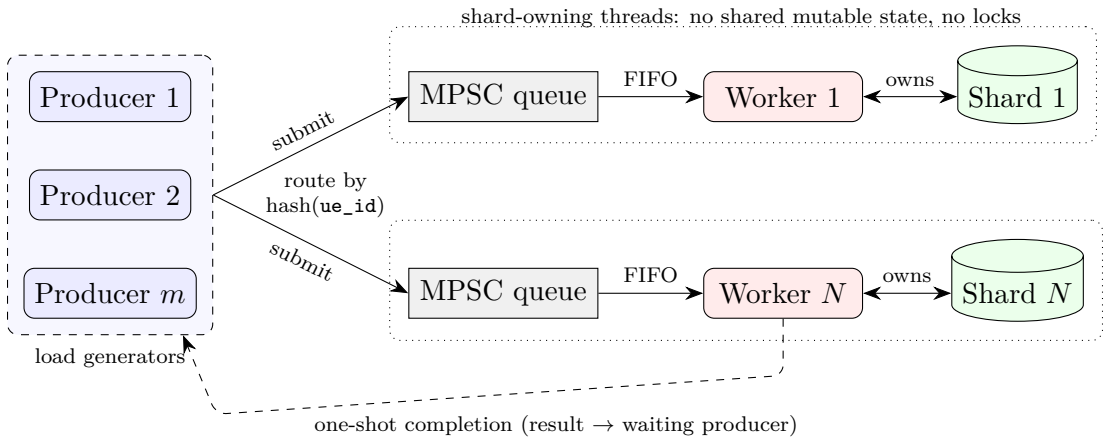


Figure 5.2: Shared-nothing, thread-per-shard architecture. Producers submit request messages to a shard's bounded, blocking MPSC queue (selected by a pinned hash of the `ue_id`) and wait on a single-use completion handle. Each shard is read and written by exactly one worker thread, so there are no locks on context state. Blocking on a full queue is what makes overload observable as tail latency.

5.3 Architectural choices and technology stack

The C++ side uses CMake with Ninja, the standard library, and `std::thread`. The Rust side uses Cargo and the stable toolchain. Both link the same memory allocator, which keeps the comparison about the two languages and not about two allocators: C++ links the system `jemalloc`, and Rust installs a `jemalloc` global allocator (from `tikv-jemallocator`). Each side also keeps a system-allocator build as a control. Latency passes through the same high-dynamic-range (HDR) histogram algorithm [**tene_hdrhistogram**] on both sides, configured the same way (range [1 ns, 3.6 s], three significant figures). Rust uses the standard `hdrhistogram` crate. C++ uses a header-only port of the same bucketing algorithm, checked against a known distribution. Build optimisation is matched as closely as the two toolchains allow, so the Rust release profile is pinned to `opt-level = 2` against the C++ `-O2` build.

5.4 Implementation details

Both implementations follow the frozen specification field for field, and both are small enough to read end to end. The C++ core is 674 lines across eight headers and two translation units. The Rust core is 762 lines across eight modules. Dropping blank lines and comments leaves 517 and 611. Both counts describe the build the campaign actually measures. A build flag switches on the two attribution probes of Section **??**, and the conditional blocks that select them are left out of the count, because counting them would describe neither the compared build nor the probe.

The Rust version is therefore the larger of the two, by 13% in raw lines and 18% in code lines. That belongs in the text and not in a footnote, for two reasons. Line count is one of the software-quality proxies this study set out to collect. And the difference is not stray verbosity: it is the net of several forced idioms pulling in opposite directions, which Section **??** takes apart file by file. The rest of this section works outward from the data model to the concurrency machinery, and reports what each language-forced divergence cost as it comes up.

5.4.1 The UE context and the state machine

Each UE context stores its identifier, lifecycle state (a four-value enumeration `Idle`, `Connected`, `Reconfiguring`, `Released`), serving cell, RRC transaction id, a fixed 32-byte dummy security context, a dynamic list of up to eight data radio bearers, and two counters (messages handled and an activity sequence number). The data radio bearer list is deliberately a growable container (`std::vector` / `Vec`) and not a fixed array, so that the allocator study (RQ3) observes realistic per-context allocation. Reads return an immutable snapshot (a `UeContextView`) copied out of the worker, not a reference into live shard state. That keeps the shared-nothing guarantee intact and keeps the two implementations semantically identical. Figure **??** gives the record and the read path in full.

Identical field declarations do not, however, produce identical objects. Measured on the evaluation host, a `UeContext` takes 96 bytes in C++ and only 88 bytes in Rust. Layout

policy accounts for all of it. C++ guarantees that non-static data members sit in declaration order, so the one-byte `state` enumeration between the eight-byte identifier and the four-byte cell id forces 3 bytes of padding, and the 32-byte security key, which ends at offset 50, forces a further 6 before the eight-byte-aligned bearer vector. Rust carries no `#[repr(C)]` annotation here, so it is free to reorder fields, and it does. It packs the small scalars together and both holes disappear.

The specification does not try to erase that difference, and the reason is worth stating. Pinning the Rust layout with `#[repr(C)]` would penalise Rust for nothing but obedience to a C++ padding rule. Hand-reordering the C++ declarations would be an optimisation the specification asks of neither side. The gap is therefore recorded as a language-level effect and then bounded. At the 5 000 active UEs the sweep uses, the 8-byte gap comes to roughly 40 kB. Resident set sizes are measured in megabytes, so this does not matter to the RQ3 comparison.

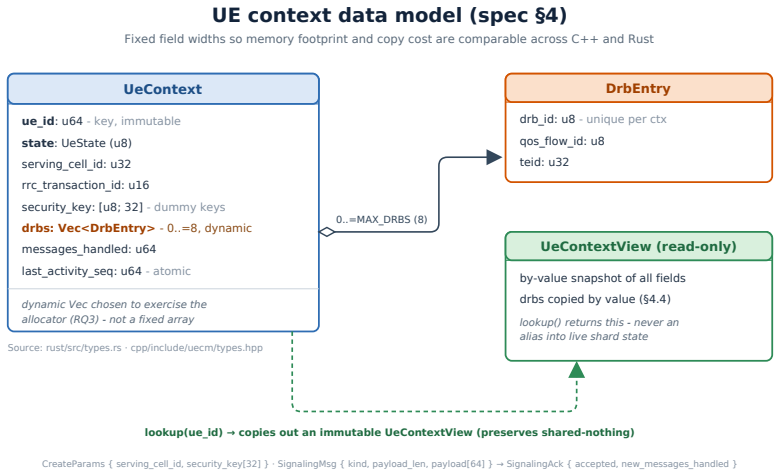


Figure 5.3: The UE-context data model shared by both implementations. Field widths are fixed so that memory footprint and copy cost are comparable across languages. The data-radio-bearer list is a dynamic `Vec`/`std::vector` on purpose, to exercise the allocator for RQ3. `lookup` returns an immutable `UeContextView` copied out by value rather than an alias into live shard state, preserving the shared-nothing guarantee.

That record is driven by the lifecycle of Figure **??**. Each operation is a single guarded transition, and Table **??** gives the implemented table in full. Two properties hold everywhere, and they are what make the two implementations comparable op for op. First, every operation checks before it changes anything. The guard sequence validates the identifier, then locates the context, then tests the state, and it returns the matching `OpError` before touching any field. A rejected operation therefore leaves the context bit-identical to what it was. Second, no operation can half-succeed, because each performs at most one mutation once the last guard has passed.

Two behaviours need saying out loud, because the table hides them. The `Released` state is never actually stored. `release_context` erases the entry from the shard map outright, so `Released` is a passing label in the specification and not a value an implementation can be caught resting in. A second release therefore returns `NotFound` and not `InvalidState`. The other case is `abort_reconfigure`. It restores the *state* to `Connected`, but it does not restore the serving cell that `begin_reconfigure` overwrote with the handover target.

That is a deliberate simplification, since the model saves no pre-handover cell to roll back to. It is still a semantic commitment and not an oversight, and both implementations make it the same way, which is what the parity suite of Section ?? checks.

Table 5.1: The implemented state-transition table, identical in both languages. Every operation additionally returns `InvalidArgument` for a zero `ue_id`, and a rejected operation leaves the context unmodified. `Released` is never stored: release erases the entry.

Operation	Required state	Resulting state	Errors
`create_context`	context absent	`Idle`	`AlreadyExists`, `CapacityExceeded`
`setup_connection`	`Idle`	`Connected`	`NotFound`, `InvalidState`
`handle_signalling`	`Connected`	`Connected`	`NotFound`, `InvalidState`
`add_drb`	`Connected`	`Connected`	`NotFound`, `InvalidState`, `CapacityExceeded`, `AlreadyExists`
`begin_reconfigure`	`Connected`	`Reconfiguring`	`NotFound`, `InvalidState`
`complete_reconfigure`	`Reconfiguring`	`Connected`	`NotFound`, `InvalidState`
`abort_reconfigure`	`Reconfiguring`	`Connected`	`NotFound`, `InvalidState`
`release_context`	any live state	entry erased	`NotFound`
`lookup`	any live state	unchanged	`NotFound`

5.4.2 Sharding, the store, and the bounded queue

Both sides pin their hashing, so that neither standard library's hasher can become an accidental language advantage. Shard assignment runs the `ue_id` through the SplitMix64 finalizer, three xor-shift and multiply rounds with the constants `0xBF58476D1CE4E5B9` and `0x94D049BB133111EB`, and then reduces modulo the shard count. The finalizer matters because the workload draws sequential identifiers, and a naive modulo would map those onto shards in a fixed repeating pattern. Scrambling the identifier first makes the offered load per shard even in both languages by construction, not by luck.

The per-shard container is each language's standard hash map (`std::unordered_map` / `std::collections::HashMap`), but the default hashers are bypassed. They differ too much to leave in place. C++'s `std::hash<uint64_t>` is usually the identity function, while Rust's default `SipHash-1-3` is cryptographically strong and correspondingly slower. Keeping both would have measured a standard-library choice, not a language. Both maps are therefore keyed through a pinned FNV-1a 64-bit hash over the eight little-endian bytes of the identifier, and a test checks that the Rust `BuildHasher` produces a digest bit-identical to the reference `fnv1a_64`.

Both maps reserve an initial capacity of 4 096 entries at start-up so that steady-state measurements are not disturbed by rehashing. The remaining configuration is likewise shared: the default eight shards, a queue bound of 1 024 requests per shard, at most eight bearers and a 64-byte signalling payload per context, and a ceiling of 200 000 contexts per shard. Only the shard count is varied from these defaults, and only by the campaign, which sets it to match the cores it is given.

The bounded blocking queue is where the two standard libraries part company most sharply. Rust supplies the primitive the specification asks for outright. `sync_channel(capacity)` is a bounded multi-producer, single-consumer channel whose

`send` blocks when the queue is full and whose `recv` blocks when it is empty, which is exactly the required backpressure in one call. The C++ standard library, through C++23, offers no bounded blocking queue at all. The same semantics are therefore assembled by hand from a `mutex`, two `condition_variable`s and a `std::queue`, in a class of 45 lines.

The point of the comparison is not that the C++ code is badly written. It is short, it uses the predicate form of `wait` so that spurious wakeups are handled correctly, and it releases the lock before notifying. The point is that it exists at all. Every one of those lines is a place where a concurrency defect could enter. And the correctness argument for it rests with the author, not with the compiler or the library: that the two condition variables are never confused, that the predicates match the notifications, that no wakeup is lost. This is the same asymmetry the fault-injection study of Chapter **??** measures, showing up here in ordinary code with no defect in it instead of in a bug put there on purpose.

5.4.3 Requests, responses, and the service lifecycle

The message type carried through that queue is the second forced divergence. Rust models a request as an `enum` with one variant per operation. Each variant carries exactly its own arguments plus a `Sender` parameterised by exactly that operation's result type. C++ has no payload-carrying enumeration in the standard idiom used here, so a request is a tagged struct instead. It holds an `OpKind` discriminant alongside *every* argument field any operation might need, paired with a single `std::promise`. That promise's value type must therefore be a `std::variant` over all four possible result types.

This has three measurable consequences. The first is size. A `Request` is 160 bytes in C++ against 96 in Rust, because the C++ struct always carries the create parameters, the 68-byte signalling message, the bearer entry, and the reconfiguration parameters, whether the operation in flight uses them or not. The Rust enum is sized by its largest variant alone. With the default eight shards holding up to 1 024 requests each, full occupancy means roughly 1.3 MB of queue-resident request storage against 0.8 MB. The second consequence is a copy cost paid on every submission, because the unused fields are default-initialised and moved along with the used ones.

The third matters most to the argument of this thesis, and it is a type-safety gap. The C++ caller has to recover its typed result from the variant with `std::get<OpResult<T>>`, which throws `std::bad_variant_access` if the worker ever writes an alternative the caller did not expect. That exception cannot fire in the code as it stands, because each `OpKind` is handled by exactly one branch and that branch stores the matching alternative. But the invariant behind it is upheld by reading two files, not by the compiler. Rust settles the same invariant by construction. The sender embedded in each variant is already typed, so there is no response union, nothing to unwrap, and no runtime dispatch on the way back.

Starting the service allocates the shards, spawns one worker thread per shard, and hands each worker exclusive ownership of its store. The request path is then the one described in Section **??**. Shutdown is the more delicate half. It has to stop workers that are blocked on an empty queue without abandoning requests that are still in flight. Both

implementations solve it the same way, with an in-band `Stop` sentinel. One `Stop` request is pushed onto each shard's queue, and because the queue is strictly FIFO, every request already queued ahead of it runs to completion before the worker sees the sentinel and returns. No separate closed flag, timeout, or drain API is needed, and no completion handle is left unfulfilled. The worker threads are then joined, so shutdown returns only once every shard has actually stopped.

Both languages tie this to object destruction, and the mechanism is one of the few places where the two are near-identical: the C++ destructor calls `shutdown()`, and Rust's `Drop` implementation does the same, so a service that goes out of scope cannot leave detached threads behind. Both guard against double invocation with a `running` flag, making shutdown idempotent.

The window between those two calls is a precondition on every operation, and it is written as one. An operation submitted before `start()` or after `shutdown()` is a caller error and not a runtime condition, so it is reported out of band and does not widen the closed `OpError` set of Section **??**. C++ throws `std::logic_error` and Rust panics. Saying so explicitly was not a formality. The specification forbade the call from the start, neither implementation checked it, and in that gap the two languages did not fail alike. The episode is reported with the safety results in Section **??**, because a defect found in code written to be correct carries evidence an injected one cannot.

One more piece of ownership discipline holds the shared-nothing invariant together end to end. A request is moved into the queue and not copied, and it carries its completion handle with it, so the producer and the worker never both hold a usable reference to the same request. C++ enforces this by making `Request` move-only, a property it picks up automatically from the move-only `std::promise` it contains. In Rust it follows from ordinary move semantics, and the compiler rejects any attempt to use the request after submission. The only shared mutable state in either implementation is the atomic activity counter. C++ holds it by reference, Rust as a cloned `Arc<AtomicU64>`, and both increment it with relaxed ordering because nothing branches on its value.

5.4.4 Forced idioms, and what the two cores cost

Where the two languages must diverge to satisfy the same requirement, the deviation is recorded in the specification's forced-idiom log so that it can be weighed rather than silently absorbed into the results. Table **??** collects the log in full, and the preceding subsections have already quantified the two entries with measurable consequences.

Table 5.2: The forced-idiom log: every point at which the two implementations diverge in order to satisfy the same specification requirement, with the measured consequence where there is one.

Concern	C++	Rust
Result type	`std::expected<T, OpError>`	`Result<T, OpError>`
Request	tagged struct: discriminant plus every argument field (160 B)	`enum`, one variant per operation, carrying only its own arguments (96 B)
Response	`std::variant` of four result types, unwrapped by the caller with `std::get`	none needed: each variant carries a sender of its own result type
Bounded queue	hand-written `mutex` plus two `condition_variable`s (51 lines)	`sync_channel(capacity)`
Completion	`std::promise` / `std::future`	a fresh `mpsc::channel()` per request
Per-shard map	`std::unordered_map` with an `FnvHash` functor	`HashMap` with an `FnvBuildHasher`
Activity counter	`std::atomic<uint64_t>` held by reference	`Arc<AtomicU64>` cloned into each worker
Teardown	destructor calls `shutdown()`	`Drop` calls `shutdown()`
Optimisation level	`-O2`	`opt-level = 2`

The log also explains the line-count difference reported at the head of this section, and it says more than the net figure does, because the contributions do not all point the same way. Table **??** breaks the 88-line gap down by concern. Rust is much *shorter* in exactly one place, the bounded queue, where it needs no code at all against 51 lines of C++. It is longer almost everywhere else, and three entries account for most of that excess.

The largest is the service module, for the reason given above. Because the C++ response is a single `variant`, all ten public operations funnel through one shared `submit` helper, and each typed wrapper shrinks to two or three lines. Rust's per-variant typed sender admits no such helper without generics, so every operation repeats the same sequence: create a completion channel, construct its variant, send, receive. The next entry shows the trade-off most clearly, and it is hashing. C++ meets `std::unordered_map`'s hasher requirement with a five-line functor wrapping `fnv1a_64`. Rust has to supply the whole trait apparatus, every line of it boilerplate, and what it buys is a statically dispatched, type-checked hasher instead of a callable of the conventional shape. The third entry is the module tree and public re-export list, work C++ gets for free from its header structure.

None of this is a defect on either side, and none of it shows up in the performance figures. It is the kind of engineering cost the forced-idiom log exists to expose. Rust spends source lines on trait machinery and repetition at the call site, and what it gets back are guarantees the compiler can check. It then wins much of that spend back wherever its standard library happens to supply a primitive that C++'s does not.

Counting lines, however, answers only half the question a reviewer of either codebase would ask, because it treats a line of boilerplate and a line of branching as the same thing. Table **??** therefore adds cyclomatic complexity, measured by `lizard` [**lizard**] over both cores. One tool applied to both languages is what makes the figures comparable, because it uses the same definition of a decision point on each side. Each language's own

tooling would count somewhat different things. The two measures then point in opposite directions. Rust is 18% longer than C++ by code lines, but its total cyclomatic complexity is 15% *lower*, 81 against 95, spread over slightly more functions (40 against 37). Both cores are undemanding by the usual thresholds, and both have a median per-function complexity of one, so a small number of functions at the top carry the aggregate.

That top is where the difference sits, and it traces back to a divergence already described, not to anything spread thin. The most complex function in the C++ core is `worker_loop` at 13, more than twice its Rust counterpart's 6. It switches on the `OpKind` discriminant and then stores the matching alternative into the response `variant`. The tagged-struct design of Section **??** therefore gathers the whole dispatch, and the correctness argument that goes with it, into one branch-heavy function. Rust's per-variant typed sender removes the return-path dispatch altogether, which is why its worker has nothing comparable to unwrap. The trade is sharper than the line count alone suggested. The extra lines Rust spends buy back branching in the one place where branching is hardest to review. And the `std::bad_variant_access` invariant that Section **??** described as upheld by reading two files lives inside exactly the function this measurement flags.

Table 5.3: Size and cyclomatic-complexity proxies for the two cores, measured with `lizard` [**lizard**] applied identically to both languages. Benchmark drivers, tests, conformance runners, the FFI seam, and the probe-conditional blocks of Section **??** are excluded. Only the compared code the parity contract constrains is counted. Complexity is one plus the number of decision points in a function. The line count is NLOC as `lizard` defines it, which also discounts brace-only and preprocessor lines. It is the same measurement quoted at the head of Section **??**.

Measure	C++	Rust
Files	10	8
Functions	37	40
Lines of code (blank and comment lines excluded)	517	611
Total cyclomatic complexity	95	81
Mean complexity per function	2.6	2.0
Median complexity per function	1	1
Most complex function	13 (`worker_loop`)	6 (`add_drb`)

Table 5.4: Where the 88-line difference between the two cores comes from, by concern (raw lines, including comments. C++ figures merge header and translation unit. Probe-conditional blocks excluded as in Section **??**). Rust is shorter only where its standard library supplies a primitive C++ lacks.

Concern	C++	Rust	Δ
Service (queues, workers, API)	234	278	+44
Hashing (functor vs. trait pair)	36	70	+34
Module tree and re-exports	0	26	+26
Data model	73	85	+12
Message types	48	60	+12
Configuration	18	29	+11
Shard core (state machine)	182	188	+6
Error model	32	26	−6
Bounded queue	51	0	−51
Total	674	762	+88

One property of the Rust core is not a cost at all, and it closes this section because it

is what the costs above bought. The Rust library contains *no* `unsafe` code, and that is checked mechanically over the crate. Safe Rust therefore guarantees data-race freedom for this core at compile time, through the `Send` and `Sync` traits. The shard store is owned by a single worker and never shared, the request type is moved across the thread boundary, and the one piece of genuinely shared state, the activity counter, is an atomic. None of this needs a runtime sanitiser to establish, and none of it needs test coverage to hold. The C++ implementation does no manual memory management beyond STL containers and no explicit locking outside the queue class, and it was checked data-race free under ThreadSanitizer.

The two statements are not, however, evidence of the same kind, and that difference is the thesis's central safety claim in miniature. The Rust property is a theorem about the program, discharged by the compiler on every ordinary build and covering every execution. The C++ property is an observation about the executions a particular instrumented binary happened to take, and it would have to be re-established after any change, on a build most projects do not run by default. Quantifying exactly how far apart those two kinds of assurance are is the purpose of the fault-injection study in Chapter **??**.

5.5 Public API and operation semantics

Both implementations expose the same small, lifecycle-oriented API against the shared specification, so the surface a caller sees is identical apart from each language's naming conventions. A service is created with a fixed shard count and a per-shard queue bound. Callers then invoke context operations, each identified by the target `ue_id`: create, setup, handle-signalling, add-bearer, reconfigure begin/complete/abort, release, and state lookup. Every operation is *total* and *typed*. It returns either a success value or a typed `OpError` drawn from a closed five-value enumeration (`NotFound`, `AlreadyExists`, `InvalidState`, `CapacityExceeded` and `InvalidArgument`). An operation that is illegal for the context's current state (Figure **??**) returns the matching error and leaves the context untouched. It does not mutate the context, and it does not signal failure out of band.

That choice matters for fairness as much as for correctness. Neither implementation uses exceptions or error codes as a hidden control-flow channel, so the cost and the control flow of the failure path are visible and identical on both sides. It also means the same conformance vectors (Section **??**) can assert op-for-op agreement on both the returned value and the resulting state.

5.6 Parity verification

The whole comparison rests on the two versions being functionally identical, so parity is tested and not assumed. A language-neutral conformance suite writes test vectors as sequences of operations with their expected results and an expected post-state summary. A runner in each language executes the vectors and emits a transcript in one fixed format, and the two transcripts are diffed for op-for-op agreement. The seeded suite covers the full error model, the reconfiguration begin/complete/abort paths, data-radio-bearer fill and overflow, and shard-assignment determinism. Both implementations currently pass and

agree. The benchmark workloads add a second parity signal, independent of the first. For a given seed and active-UE population, the mixed workload's per-operation-kind counts are checked to be identical across languages, which confirms that both implementations are driven along exactly the same code paths.

Two things bound what that establishes, and both are answered rather than conceded. A cross-diff of two implementations cannot tell "both correct" apart from "both wrong in the same way", and that is the failure mode a shared specification and a single author make most likely. The expected results in the hand-written vectors also came from the specification by way of the person who wrote the implementations. A third implementation of the same written specification therefore acts as an independent oracle. It is written in Python, shares no code with either language, and has to reproduce all 223 hand-derived expectations before it is allowed to generate any of its own. It then generates randomised operation sequences under several profiles: realistic lifecycles, adversarial sequences weighted towards illegal transitions, sequences that sit on the capacity limits, and sequences that stress shard assignment. The model computes every expected result. A campaign of roughly 793 000 generated operations produced no divergence at all. The two implementations agree with each other, and with an independent reading of the specification, on sequences nobody chose. The generator is seeded and deterministic, so any future divergence is a permanently reproducible regression and not an anecdote.

Three further checks bound the suite from other directions. The specification's transition matrix is walked exhaustively and not sampled: every operation applied to a context in every state it can occupy. Each case asserts the exact error, the resulting state on success, and the absence of any field change on failure, including the error precedences the specification leaves unordered. The C++ implementation is then run under AddressSanitizer, ThreadSanitizer and UndefinedBehaviorSanitizer across that whole suite, and reports nothing. That matters for fairness. Safe Rust carries data-race and memory-safety freedom by construction, so without the sanitiser run the C++ arm's equivalent property would rest on not having failed yet instead of on having been checked. Miri, run over the Rust state-machine suite, likewise reports no undefined behaviour. Third, the suite's reach is measured and not asserted, because a negative result from a sanitiser is only as broad as the code that ran. Line coverage of the C++ library is 94.6% and of the Rust library 98.2%, and both implementations' core operation processors and service facades are fully covered. The two figures are not strictly comparable, because `gcov` and `llvm-cov` do not resolve an executable line alike. On the finer-grained regions `llvm-cov` counts, the Rust figure is 99.3%. Each bounds the claims made about its own implementation.

6 Experimental Setup and Evaluation

6.1 Testbed configuration

Two execution environments must be distinguished, because the thesis deliberately reports results of two different evidential strengths and the distinction rests entirely on where they were produced.

The environment used. All performance results reported in this chapter were measured on one dedicated dual-socket server: two Xeon E5-2680 v4 processors, 16 vCPUs on a single NUMA node presented as one thread per core, running Rocky Linux 8.10 with `g++ 15.1.1` and `rustc 1.97`. The benchmark is pinned to cores 2–15, so that every worker and every producer owns a core and none contends with the operating system. Table **??** lists the host against the protocol described below. Every run captures its own environment record (CPU model and topology, kernel, compiler and toolchain versions, and the frequency, turbo, and multithreading settings) alongside the measurement data, so the conditions of a measurement travel with it.

One property of this host has to be stated plainly, because it bounds every tail claim below: it is a virtual machine. The environment record identifies a KVM guest, and three consequences follow. First, the CPU frequency governor and turbo state are host-controlled. They are not merely unset but unobservable from inside the guest, so frequency stability is assumed and not enforced. Second, the record shows non-zero CPU steal, on the order of ten milliseconds of vCPU time per second. That is small in aggregate, but it is the same order as the tail latencies being measured, and nothing inside the guest can remove it. Third, the topology is not what it appears to be. The guest is shown two sockets of eight single-threaded cores, which is not the shape of an E5-2680 v4, a fourteen-core part with simultaneous multithreading. That topology is therefore synthetic. Whether the sixteen vCPUs sit on distinct physical cores or share thread siblings is a host decision, and it cannot be read from inside the guest. Siblings would compete for one core's execution resources. The cores were also not removed from the scheduler with `isolcpus`, so a benchmark thread can still lose its core to other work.

This is a much better instrument than the shared development laptop an earlier pass of this campaign used. The cores are uniform and server-class, nothing else of consequence runs on the machine, and the run-to-run spread at p99 fell by more than an order of magnitude between the two. It is still not the bare-metal host the protocol below specifies.

The protocol the performance campaign still lacks. Testbed-grade tail figures need a dedicated bare-metal host. The protocol for one is specified, scripted, and published, so it can be run without further design work (`bench/TESTBED.md`), and its requirements sit in the right-hand column of Table **??**. What running it would add is narrower

than it once was. Three of the four things it was going to buy have arrived without it: the saturation knee, the shape of the overload cliff (Section **??**), and the replication of the resource comparisons across the allocator control (Section **??**). What is left is the far tail *below* saturation. There the confidence interval on a single variant's p99 is still several times the spread between all four variants, and the likely cause is steal and scheduler preemption, not anything either language does. That one result is therefore reported as unresolved and not as a null, and the machine has to fix it, not the analysis.

Measurement discipline common to both. Whatever the host, each sweep point is repeated 20 times and reported as a median with a bootstrap 95% confidence interval, never as a single run. The four language×allocator variants are interleaved within each repeat, so thermal or background drift hits them equally instead of biasing one. That interleaving is what makes a contended host usable for a *comparative* conclusion even though it ruins the absolute numbers, because interference is then shared and not assigned. Throughput is counted over the measured phase alone, with the warm-up iterations left out of both the numerator and the denominator, so achieved throughput can be compared directly with offered load.

The division Table **??** draws is not between important and unimportant results. It is between results that are properties of the compilers and the source, which this host settles as firmly as any host would, and results that are properties of the machine, which it cannot settle at all.

6.2 Workload and evaluation measurements

A synthetic but spec-justified telecom workload is generated by a deterministic driver whose structure both implementations share, so the two versions are exercised along identical code paths. Two workloads are provided. The *signal* workload preloads a population of connected UEs and then hammers the `handle_signalling` hot path, which isolates steady-state per-message latency. The *mixed* workload is a pinned RAN lifecycle generator. It walks each UE through a realistic procedure sequence, context creation, connection setup, signalling, bearer setup, reconfiguration/handover (begin/complete/abort), and release, using a fixed SplitMix64 draw order and a per-slot expected-state model so that every operation it emits is valid. The fixed draw order means the per-operation-kind counts are identical in C++ and Rust for a given active-UE population and seed, which doubles as the parity signal described in Section **??**. The control parameters are the target event rate and the number of active UEs, swept from nominal load through saturation and overload. The send schedule is fixed and the histogram is corrected, so a stall introduces no coordinated-omission bias [**schroeder2006openclosed**, **tene_hdrhistogram**]. Three measurement campaigns are run: a load sweep for performance, the fault-injection study, and an FFI migration prototype. Figure **??** shows the two workloads and the procedure sequence the mixed one walks.

What "spec-justified" claims, and what it does not. The term is used precisely, because the distinction bounds what the workload can support. The *sequence* of opera-

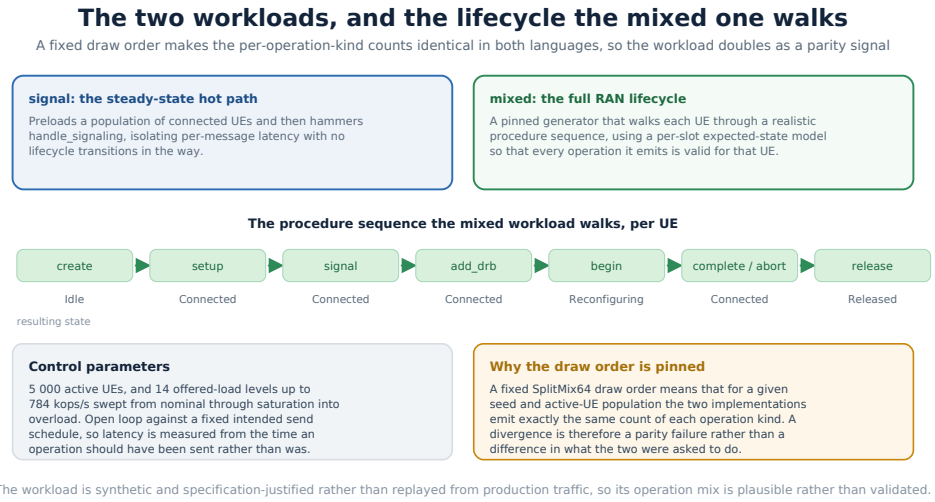


Figure 6.1: The two workloads. The *signal* workload isolates the steady-state hot path. The *mixed* workload walks each UE through the full RAN procedure sequence, with a per-slot expected-state model ensuring that every operation it emits is legal for the state its context is actually in. Because the SplitMix64 draw order is pinned, a given seed and active-UE population produce identical per-operation-kind counts in both languages, so a divergence is a parity failure rather than a difference in what the two implementations were asked to do.

tions is justified. It is the RRC connection lifecycle the 3GPP specifications define for a UE, from establishment through in-session signalling and bearer setup to reconfiguration and release [**3gpp38401**, **3gpp38331**]. The per-slot expected-state model then guarantees that every operation the driver emits is legal for the state its context is actually in, so the workload never drives the service down a path the specification forbids. The *proportions* are not justified in the same way. How often a real cell hands over, against how often it establishes a bearer, depends on deployment, mobility, and time of day, and no public figure fixes it. The mix used here is therefore plausible and not validated. Section **??** reports that as a threat to external validity, and trace replay heads the future-work list for exactly this reason.

For a *comparative* question the distinction costs less than it would for an absolute one, and saying why is part of arguing that the data suits the task. Both implementations execute the same sequence in the same order for a given seed and active-UE population, and the per-operation-kind counts verify it. An unrepresentative mix therefore shifts both languages equally and cannot manufacture a difference between them. An unvalidated operation mix bounds how far the numbers travel. It does not compromise the fairness of the comparison they are used to make.

Table **??** gathers the configuration of the service, the workload and the campaign in one place. Those values are otherwise spread across the specification, Chapter **??**, and this chapter. That is convenient while reading each in turn, and unhelpful to anyone trying to reproduce the campaign or to judge whether a parameter was chosen to suit the result. Every entry in the campaign block is either fixed by the specification or generated from the measurement logs, so none of it can drift out of step with the data reported below. Two derived values are used throughout what follows. The representative operating point is not chosen by eye. It is the busiest swept load at which all four build variants still

hold their p99 below 5 ms, which puts it at 56 kops/s, against a measured saturation of 335 kops/s.

6.3 Evaluation data

Three campaigns produce three datasets, and they differ in kind and not only in content. What follows describes each one as captured. A reader judging whether the analysis is sound needs to know what was recorded and at what granularity, not just what was reported. Every measured quantity this chapter quotes reaches the page as a generated macro. `bench/refresh_thesis.sh` rebuilds all of them, along with every figure and table body, from the raw logs, so a rerun moves the prose and cannot leave a headline number behind. The software-quality proxies are not gathered here but reported where they read most clearly: beside the forced-idiom log they explain (Table **??**), and beside the FFI seam, the only place either implementation contains `unsafe` (Table **??**).

The performance sweep. Each of the 1120 runs of the factorial design in Table **??** contributes one record, written as newline-delimited JSON. A campaign is therefore an append-only log and not a summary table. Nothing is aggregated at capture time, and every statistic in this chapter is recomputed from the raw records. A record carries the run's configuration (language, allocator, workload, shard and producer counts, target rate), its outcome (operations completed, errors, wall-clock duration, achieved throughput), the latency distribution read out of the HDR histogram (mean, p50, p90, p99, p99.9, maximum, in nanoseconds), the resource counters taken through `getrusage` (peak resident set size, user and system CPU seconds, CPU utilisation), and the per-operation-kind counts. Keeping the kind counts in every record is what makes a divergence between the two languages show up as a parity failure (Section **??**) instead of being mistaken for a performance difference.

The component decomposition and contended primitives. The second measurement level of Section **??** yields a much smaller dataset of a different shape. For each of 15 interleaved repeats it records the batch-mean cost in both languages, once for the operation called directly and once for the same operation called through the threaded service. A third dataset, of the same paired shape over 20 repeats, records sustained throughput through the bounded queue and completion primitives alone (Section **??**). The FFI comparison of Section **??** has the same structure over 15 repeats. All three are paired by construction, which is what the exact paired test of Section **??** requires.

The fault-injection catalogue. This dataset is categorical and not numeric. Each of the 14 defect cases yields four tool logs: the AddressSanitizer, ThreadSanitizer and UndefinedBehaviorSanitizer output for the C++ program, and the `rustc` output for the safe-Rust counterpart. Each case also yields one row in a machine-readable taxonomy, recording the three sanitiser outcomes, the detection stage, the compiler result with the diagnostic codes that fired, and the expectation registered before the study. The two original boundary cases yield a second table from the model checkers, and the `unsafe`

one a third from the interpreter. Carrying the expectation in the data instead of in the prose is what lets the runner report a mismatch in either direction. The table and figure in Section **??** are generated from that file, not transcribed from it.

What travels with the data. Every campaign writes an environment record beside its results. It captures host and kernel, distribution, compiler and toolchain versions, CPU model and topology, and the frequency governor, turbo, and simultaneous-multithreading settings, *including* the fields a given host does not expose. That last detail matters more than it sounds. The record identifies the host as a KVM guest, and the governor and turbo state are stored as unavailable instead of being left out. The limitation documented in Section **??** is therefore a property of the dataset itself and not only of the prose describing it, and a reader can check it without taking it on trust.

The performance dataset is complete and self-consistent as an analysis and is used below to answer RQ1 and RQ3 provisionally. Rerunning the same scripts over a campaign captured on a bare-metal host replaces these numbers with testbed-grade ones without any change to the analysis.

6.4 Evaluation results

The safety and migration results do not depend on the measurement host at all. The performance and resource results do, and Section **??** states what that host can and cannot carry. Every figure and table below is measured on it unless stated otherwise.

Resolved and unresolved outcomes are reported side by side. Saying at the outset which is which keeps a null result from being read as an absent one. Six findings are resolved: the defect-class taxonomy and its boundary (Section **??**), the cost of the FFI seam (Section **??**), the memory difference between the two builds (Section **??**), the cost of the bare operation (Section **??**), the equivalence of the four variants at the median below saturation, and their divergence past it (Section **??**). That last one is the largest and most reproducible language effect in the study.

The far tail *below* saturation is the one unresolved outcome, and the host is the reason. Run-to-run spread at p99 exceeds every difference between the variants, and no number of repeats on a virtualised machine with non-zero CPU steal closes that gap (Section **??**). Only the bare-metal protocol of Section **??** will answer it. The overload divergence is resolved as a measurement, though not fully as a mechanism. Section **??** tests whether the bounded queue and completion primitives alone explain it. They do not reproduce the full-service ordering at significance, which leaves work somewhere in the full handoff path beyond the isolated primitives.

6.4.1 Performance (RQ1, RQ3)

The hypothesis under test is H1: that the two languages are performance-equivalent below saturation, with any difference confined to the far tail. This section reports where, if anywhere, each wins and by how much, across the sweep up to and beyond saturation.

Everything in it is generated from the raw logs by `bench/plot_figures.py`: figures, ta-

bles, and the numbers quoted in the prose alike. The section can therefore be regenerated verbatim if the campaign is rerun on the bare-metal testbed. The absolute figures belong to the host of Section **??**. Each point aggregates 20 interleaved repeats. Tabulated values and shaded bands are medians with nonparametric bootstrap 95% confidence intervals.

Table **??** summarises the per-operation latency distribution at a sustained offered load of 56 kops/s. That operating point is not chosen by eye: it is the highest swept load at which every one of the four build variants still holds its p99 below 5 ms, that is, the busiest point at which all four are simultaneously healthy, which makes it the fairest place to compare them.

The table has to be read at two different levels of confidence, and separating them is the honest reading of this campaign. At the *median* the four variants sit within 2 μs of one another, or 2.1%, and every pair of confidence intervals overlaps. The table on its own therefore separates nothing. That is a statement about unpaired intervals and not about the languages. The paired test below does resolve a small and consistent difference at the median, and the distance between those two readings is exactly why Section **??** tests the comparison instead of inspecting the intervals.

Above the median the situation is different in kind. The point is not that the languages differ but that this host cannot say whether they do. The p99 values differ by 293 μs across variants, while the widest 95% confidence interval on a single variant's own p99 spans 1480 μs, many times the between-variant difference. When the uncertainty about one variant exceeds the gap between all of them, no ordering can be read off. Reporting one would be reporting the scheduler. The same holds with greater force at p99.9 and at the maximum.

This is the concrete form of the limitation stated in Section **??**, and its cost is bounded. It does not touch the throughput and saturation behaviour, the resource results below, or anything in the RQ2 and RQ4 chapters. What it does mean is that one half of RQ1 gets only a negative answer from this campaign, namely how the two languages behave in the high percentiles while the service is still meeting its offered load. That tail is not yet measurable. The other half is untouched, and Section **??** resolves it decisively on this same host. Closing the gap needs the bare-metal protocol, not more repeats here, because the spread is systematic interference and not sampling noise. Section **??** also shows that the quantity a language could affect lies more than three orders of magnitude below the one this measurement resolves.

One qualification about the metrics belongs here and not only in the validity discussion. RQ1 names the median, p99, p99.9, and the maximum because those four are what this campaign delivers. The harness also records inter-operation jitter and p99.99, and it keeps the histograms the far tail would have to be pooled from, but the campaign reported here predates that instrumentation. Predictability is therefore evidenced at p99 and p99.9 and no further. That is a gap in the evidence and not in the design, and it closes with the bare-metal campaign the tail claims already wait on.

Table 6.1: Per-operation latency at a sustained offered load of 56 kops/s (mixed RAN-lifecycle workload, 7 shards). Values in microseconds: the median over 20 repeats above its nonparametric bootstrap 95% confidence interval. The interval is printed in full rather than as a symmetric half-width because it is strongly asymmetric at these percentiles, and a half-width wide enough to cover the upper bound would place the lower one below zero.

Variant	p50	p99	p99.9	max
C++ (system alloc)	79.9	4667	6222	7817
	[79.6, 80.7]	[4272, 4895]	[5501, 7242]	[6572, 8886]
C++ (jemalloc)	80.3	4506	5949	7472
	[79.4, 80.7]	[4289, 5216]	[5540, 7160]	[6470, 8826]
Rust (system alloc)	79.0	4530	6013	7932
	[78.7, 79.7]	[4197, 5677]	[5612, 7905]	[6527, 9327]
Rust (jemalloc)	78.6	4375	5624	6951
	[78.1, 79.7]	[4173, 4661]	[5476, 6425]	[6363, 8143]

Testing the comparison rather than eyeballing it. Overlapping intervals are a weak instrument in both directions, so the comparison is also tested directly, by the procedure fixed in Section **??** and reported in Table **??**. What licenses the pairing is a property of this campaign and not of the statistic. The four variants are interleaved inside every repeat, so the i-th run of each was measured in the same block under the same ambient conditions. Whatever interference that block carried is common to the pair, and it cancels in their difference. That is the second thing the interleaving discipline buys, beyond the fairness argument it was adopted for.

Table 6.2: Exact paired permutation test of C++ against Rust at 56 kops/s, with the allocator held fixed. The effect is the median of the 20 paired differences (Rust − C++), shown above its bootstrap 95% confidence interval. The test statistic is their mean. p is exact over all 2^{20} sign assignments, and the final column applies the Holm correction across the whole family of 14 tests. The attainable floors are 0.0000019 uncorrected and 0.000027 after correction. The rows are blocked by the criterion set out in Section **??**: pairing rescues the statistics in the upper block, so a consistent interval there is evidence, whereas the high percentiles below are computed from exactly the operations interference falls on, so an interval excluding zero is not read as a finding. Achieved throughput is not tested, for the reason given in the same section, and appears in Table **??** instead. Measured on the virtualised Xeon host of Section **??**.

Metric	Allocator	Effect (Rust − C++)	p	Holm
Robust to interference on single operations				
p50 latency (µs)	system	−1.1 [−2.0, −0.4]	0.002	0.012
p50 latency (µs)	jemalloc	−1.6 [−2.0, −0.5]	0.000086	0.00069
Peak RSS (MiB)	system	+1.9 [+1.8, +2.0]	0.0000019	0.000027
Peak RSS (MiB)	jemalloc	+3.4 [+3.3, +3.5]	0.727	1.000
CPU utilisation (%)	system	−3.4 [−5.0, −2.4]	0.0000076	0.000069
CPU utilisation (%)	jemalloc	−1.9 [−3.1, −0.8]	0.005	0.029
Determined by it: high percentiles				
p99 latency (µs)	system	+12 [−356, +866]	0.433	1.000
p99 latency (µs)	jemalloc	+111 [−389, +304]	0.462	1.000
p99.9 latency (µs)	system	−133 [−508, +1495]	0.460	1.000
p99.9 latency (µs)	jemalloc	−141 [−1028, +508]	0.407	1.000

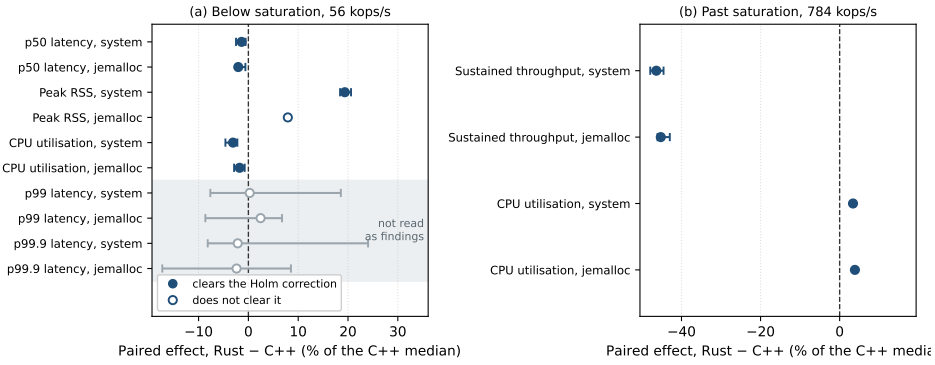


Figure 6.2: Every test in Tables **??** and **??** as a paired effect with its bootstrap 95% interval, normalised by the C++ median so that microseconds, mebibytes, percentage points and kops/s can share an axis. Filled markers are the 9 of 14 tests that clear the Holm correction. The two allocators of a metric are drawn adjacent, so an effect that reversed sign between them would appear as a pair straddling the zero line. The shaded rows are the high percentiles, computed from exactly the operations interference falls on, which Section **??** fixes in advance as rows whose intervals are not read as findings. The metrics keep their own units in the tables.

Figure **??** draws the same family on one axis, and three things it contains are easier to

see there than in a table of fourteen rows: which effects survive the correction, whether an effect keeps its sign when the allocator changes, and how much wider the tail intervals are than the rest.

Five things follow. First, 9 of the 14 tests in the family survive the Holm correction. At this operating point they are the two median-latency rows, peak resident set size under the system allocator, and both CPU rows. The four remaining survivors are the overload comparisons of Section **??**. Peak resident set size under jemalloc is the one upper-block row that does not clear the correction, despite an interval nowhere near zero, and Section **??** takes up why.

Second, the reading rule fixed in Section **??** has to be applied before any of those are interpreted. 10 paired intervals exclude zero. In the tail block the ones that do are: none. The rule is stated in advance, and not at this point, precisely because that outcome is not guaranteed. An earlier seven-repeat pass of the same campaign did produce tail intervals excluding zero, and more repeats then removed them. That is the behaviour the rule anticipates.

Third, the allocator control supplies a second check, independent of the first. Every comparison is run twice under conditions that should not change a language effect, so an effect that really is the language ought to keep its sign whichever allocator is linked. Here p50 latency, p99 latency, p99.9 latency, Peak RSS and CPU utilisation all do, and 0 reverse. The control confirming rather than vetoing is no evidence that it was unnecessary. On an earlier campaign, run before the two `jemalloc` builds were held to one configuration, the CPU effect cleared the correction under one allocator and reversed sign under the other, and it was reported as unresolved on exactly this rule. A control earns its place by what it would have caught, not by what it happens to catch.

Fourth, the 5 rows the correction does not clear are not all saying the same thing, and Table **??** is what separates them. A test that fails to resolve a difference still bounds it: there is a tightest margin within which the two variants can be called equivalent, and it is a property of the data rather than of the verdict. Read that way the upper block and the tail block come apart sharply. At the median the bound is $2.0\,\mu$s, so the difference the test does resolve is also small in absolute terms and the two readings agree. In the tail block the bounds run out to $1298\,\mu$s, which is this host's inability to see stated as a quantity instead of as a null. The distinction matters because the two are routinely reported the same way. A null with a tight bound is a result. A null with a bound wider than any difference a deployment would care about is the campaign declining to answer, and saying which is which is the least this table can do for the half of RQ1 the host cannot resolve.

Table 6.3: How large each effect could be. For every comparison at 56 kops/s, the observed effect against the tightest margin within which the two variants could be called equivalent, and the verdict Table **??** reaches. The two questions are deliberately kept in separate tables: one asks whether a difference is there, this one asks how large it could be if it is, and a row can matter in one and not the other. Latency margins are in microseconds, peak resident set size in mebibytes, CPU utilisation in percentage points.

Metric	Allocator	|Effect|	Equivalent within	Verdict
p50 latency (µs)	system	1.1	2.0	significant
p50 latency (µs)	jemalloc	1.6	1.8	significant
p99 latency (µs)	system	12	662	not resolved
p99 latency (µs)	jemalloc	111	252	not resolved
p99.9 latency (µs)	system	133	1298	not resolved
p99.9 latency (µs)	jemalloc	141	692	not resolved
Peak RSS (MiB)	system	1.9	2.0	significant
Peak RSS (MiB)	jemalloc	3.4	3.5	not resolved
CPU utilisation (%)	system	3.4	4.8	significant
CPU utilisation (%)	jemalloc	1.9	2.9	significant

Fifth, the median is now a resolved difference and not a null: 1.6 μs in Rust's favour, consistent under both allocators. The interval on it reaches 2 μs against a median of roughly 80 μs, and the temptation is to report that as a bound on the language effect. That reading would be wrong, and the next section explains why.

6.4.2 What the end-to-end number is made of

Every latency the sweep reports is an end-to-end round-trip, and Section **??** set out why that bundles two costs of quite different character. One is the operation itself, executed inside a single thread and generated by the compiler. The other is the cross-thread submit-and-complete handoff around it, charged to the operating system. Only the first can plausibly differ between the languages, and the sweep cannot separate the two.

That matters for the bound just quoted, because a bound says only as much as the quantity it is a fraction of. Both costs are therefore measured directly, by the two-level method described there, in both languages and at the sweep's own shard count and context population. `ShardCore` called with no queue in the path gives the operation. `Service` gives the operation plus the handoff. Their difference is the handoff. Table **??** gives the result, and it reframes the whole performance comparison.

Table 6.4: Where an end-to-end operation's cost actually goes, measured at 7 shards and 5 000 active UEs with a single uncontended caller. Each row is the median over 15 interleaved repeats above its bootstrap 95% confidence interval, in the unit its own magnitude calls for: the two terms differ by more than two orders of magnitude, which is the point of the table. p is the exact two-sided paired permutation test of C++ against Rust on the repeat-wise differences. The handoff is obtained as the difference between the two measured rows, so it carries the uncertainty of both.

Component	C++	Rust	p
Operation alone, no queue in the path (ns)	34.5	21.3	0.000061
	[32.4, 35.2]	[21.0, 21.5]	
Cross-thread submit-and-complete handoff (µs)	13.58	13.59	0.439
	[13.38, 13.83]	[12.69, 14.52]	
End-to-end round-trip, uncontended (µs)	13.62	13.61	0.421
	[13.41, 13.87]	[12.71, 14.54]	
Operation's share of the round-trip	0.250 %	0.160 %	

The operation is 34.5 ns in C++ and 21.3 ns in Rust. The handoff is 13.58 μs and 13.59 μs. The work the languages actually perform is therefore at most 0.250% of what the sweep measures. The complementary 99.750% of every latency in Table **??** is the operating system moving a message between two threads.

That shape is not peculiar to this system or to this pair of languages. Zhou et al. revisit a classic decomposition of transaction-processing cost on a shared-nothing database, built like this service with one partition per worker thread. Once the per-operation work has been optimised, they report, what dominates the remaining cost is communication across isolation boundaries and not the operation itself [**zhou2025oltp**]. The table above is that same pattern showing up in a control-plane service, which makes the result below a general property of the design and not an artefact of this measurement.

Two conclusions follow, and they pull in opposite directions.

The first is that the sweep cannot bound the language effect, and the bound quoted above must not be read as though it does. An interval of 2 μs on the paired median difference is 58 times the operation's own cost. A language effect could be several hundred percent of the work being done and still sit comfortably inside it. What the sweep does establish is weaker and still useful: neither language disturbs the scheduler-dominated round-trip, which is what a deployment of this shape would actually experience. What it cannot establish, at any repeat count, is anything about the code the compilers emit. That limit comes from how the measurement is built and not from the host, so unlike the tail result it will not be fixed by moving to bare metal. It is fixed by measuring the component on its own, which is what this section does.

The second is that once the component *is* measured on its own, the languages are not equivalent. Rust runs the bare operation about 1.6× faster, the difference shows up in the great majority of repeats, and the paired test puts it two orders of magnitude below the conventional threshold. This decomposition is a separate experiment from the sweep and no part of its Holm family. It runs 15 repeats instead of 20, so its own attainable floor is 0.000061 and not the 0.000027 quoted in Section **??**, and no correction is applied to it. The same holds for the interoperability comparison of Section **??**. This is the only place in the whole performance campaign where a language difference in *time* is resolved at all, and the sweep is blind to it by more than two orders of magnitude.

Figure ?? draws both conclusions on one page, because each is easier to see than to read off a table of numbers spanning four decades. Its second panel is the argument of this section: the instrument sits entirely above the quantity it would have to constrain.

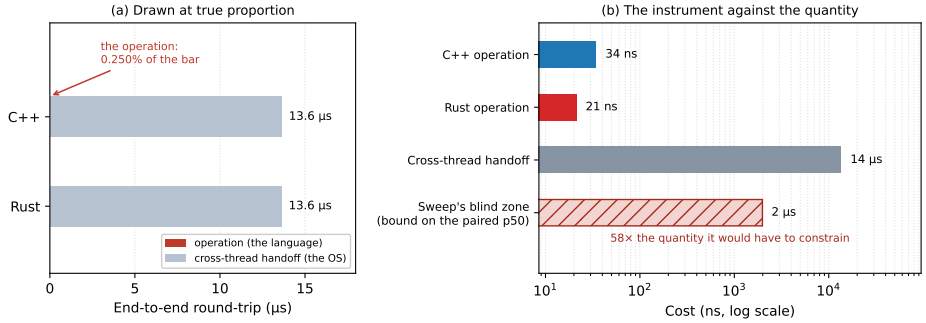


Figure 6.3: Where an end-to-end operation's cost goes, and why the sweep cannot bound a language effect. **(a)** The round-trip at true proportion: the operation the language performs is 0.250% of the bar, the remainder the cross-thread handoff. **(b)** The same quantities on a log axis against the smallest paired difference the sweep can resolve, shown as a blind zone. Both operation measurements fall inside it by a factor of 58, and (b) is also where the campaign's one resolved timing difference lives.

The attribution needs care, because the obvious reading is the wrong one. Nothing here implicates the borrow checker, bounds checking, or ownership. The most likely source is already recorded in the forced-idiom log of Table ??. The per-shard container is each language's standard hash map, and those are different data structures. C++'s `std::unordered_map` has to be node-based to keep its iterator- and reference-stability guarantees, so a lookup chases a pointer out of the bucket array into a separately allocated node. Rust's `HashMap` is an open-addressed table that keeps entries inline and usually resolves a lookup within one cache line. The hash function is pinned to FNV-1a on both sides (Section ??) precisely so that hashing could not be the explanation, which leaves the container layout. That is a real and reportable difference between the two languages as they are actually used, because a developer reaching for the standard hash map gets whichever one their language ships. It is not evidence about Rust's safety model and should not be quoted as such. Confirming the mechanism would need the cache-miss counters Section ?? lists as the clearest instrumentation gap.

Three limits keep the finding the size it is. It is a single-threaded, uncontended microbenchmark, so it says what the operation costs and not what a loaded system does with it. It is measured on the same host as the sweep, so the absolute figures carry the same caveat, though the ratio is far less sensitive to that than either term is. And it is one operation on one data structure. Generalising from it to "Rust is 1.6× faster" would repeat exactly the error this thesis criticises in the average-case literature.

6.4.3 Latency, throughput, and saturation across the sweep

Figure ?? plots p99 latency against offered load, and the shape it shows is the one Figure ?? sketched. Through the tracking region the four variants climb together and gently, from roughly 240 μs at the lightest load to single-digit milliseconds by 335 kops/s. The confidence bands there are wide enough that no curve can be separated from another anywhere in that region. Then, in the single step of the rate ladder that follows, every

variant's p99 rises by more than two orders of magnitude at once, from about 6 ms to several seconds. That is the overload cliff, and how abruptly it arrives is the finding. One increment of offered load separates a service meeting millisecond deadlines from one missing them by three orders of magnitude. A design with bounded, blocking queues has no graceful degradation to offer in between.

Two properties of the far side of the cliff bound what those numbers mean, and they are why Section **??** tests no percentile there. Latency past the knee is accumulated schedule backlog and not service time. And because the measured phase is cut off at a fixed width, the reported percentiles saturate near that width, which makes every one of them above the knee a lower bound. What stays interpretable in this regime is how much work each variant still completes, and Section **??** takes that up.

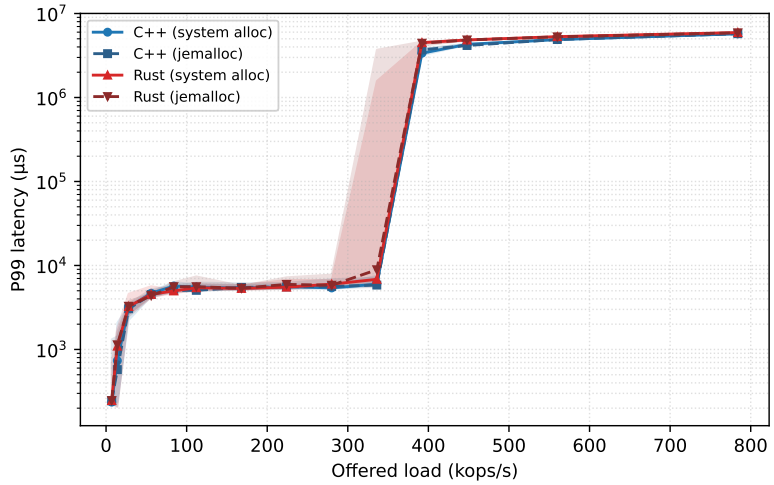


Figure 6.4: Tail latency (p99) versus offered load for the four build variants. Lines are medians over 20 repeats. Shaded bands are bootstrap 95% CIs. Log-scaled y-axis.

Figure **??** shows achieved throughput against offered load, and it is the more informative of the two plots. Throughput is an average over a whole run, which makes it far less sensitive than a percentile to interference landing on individual operations. Every variant tracks the ideal offered-equals-achieved line up to 336 kops/s and saturates at around 335 kops/s. Two things follow immediately. First, achieved throughput matches offered load below the knee to within 0.1%, so the operating point of Table **??** is a genuinely *sustained* load and not a nominal one. The latencies reported there are not those of an already-queueing system. Second, the knee sits at the same place for all four variants, which is what the identical thread-per-shard structure predicts. Where the design stops coping is a property of the design.

What happens *after* the knee is not shared. The four curves separate as soon as the offered rate stops being met, and they separate by language rather than by allocator. Section **??** measures that separation instead of leaving it to the eye.

Figure **??** holds the load fixed instead and sweeps the percentile. The four variants coincide at the median and fan out as the percentile rises. That is the expected shape for a heavy-tailed distribution, and it is why a percentile-aware methodology is used throughout. The fanning is not a language effect, and the right-hand panel shows why.

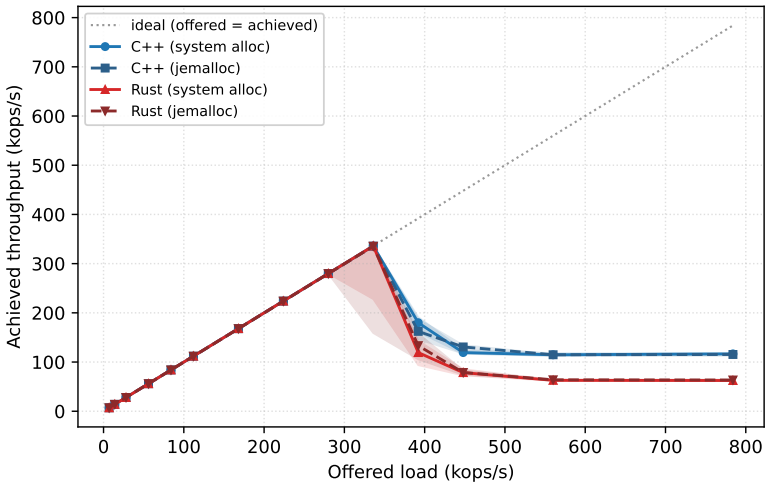


Figure 6.5: Achieved throughput versus offered load. The dotted line marks the ideal offered-equals-achieved regime. Throughput is counted over the measured phase only, so the warm-up iterations bias neither the numerator nor the denominator.

It sets the spread between the four variants at each percentile against the widest interval any single variant carries there. At the median the spread is the larger of the two, which is the regime in which an ordering means something. From p90 upward the interval exceeds the spread several times over. The fan therefore marks where the measurement loses resolution, not where the languages differ.

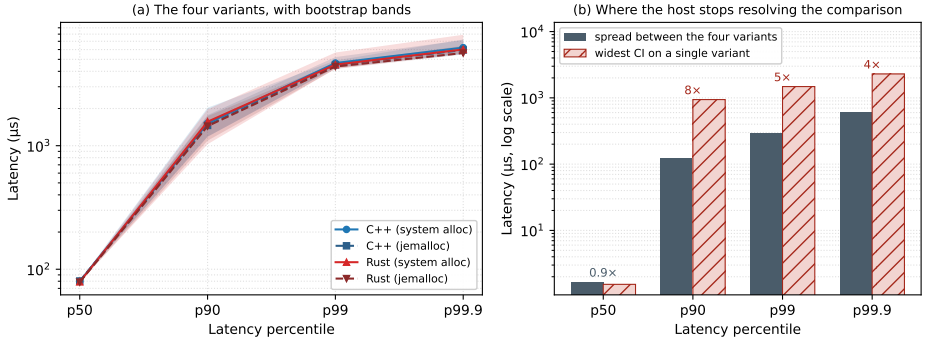


Figure 6.6: Per-operation latency across the percentile range at 56 kops/s. **(a)** The four variants as medians over 20 repeats, each with a bootstrap 95% band. The bands are the point: drawn as bare lines the curves lie on top of one another and read as a claim of equivalence, which is not what a measurement that cannot separate them has shown. **(b)** Why they cannot be separated above the median. The spread between the four variants is set against the widest confidence interval on any one of them, with the ratio annotated above each pair of bars. Once that ratio passes one, the uncertainty about a single variant is larger than the gap between all four and no ordering can be read off.

6.4.4 Behaviour past saturation (RQ1)

RQ1 asks how the two languages compare under increasing load *and overload*, and everything reported so far has been below the knee. This section is the other half. The operating point is 784 kops/s offered, defined as the deepest swept load at which every variant has stopped meeting the offered rate, and the metrics are the two that remain interpretable there (Section **??**). Table **??** gives what each variant still sustains and

Table ?? tests the comparison.

The languages are not equivalent in this regime, and the margin is not marginal. C++ sustains 116 kops/s where Rust sustains 63, a factor of 1.8. The paired difference is at least 50 kops/s under either allocator, it holds in 20/20 repeats, and its Holm-adjusted p sits at the corrected floor of the whole family of 14 tests. This is the largest and most reproducible language effect anywhere in the study, and it is the only one that speaks directly to the question RQ1 was written to ask.

The CPU column is what makes the result interpretable and not merely large, because it rules out the reading that would otherwise be natural. A variant that completes less work might simply be doing less: throttled, blocked, idle. That is not what happens. Rust draws *more* total CPU than C++ here, reproducibly and under both allocators, having flipped the sign it carried at the representative load in Table ??. Divided through by the work that actually completed, its CPU cost per sustained operation is 1.9 times the other's. Neither variant is near the capacity of the machine either, since both sit near six of the fourteen pinned cores. So the gap is not a capacity limit and not idleness. It is effort spent without completing operations, which is the signature of a synchronisation path that spins, or parks and wakes repeatedly, where the other blocks once and sleeps.

Figure ?? puts both halves of that reading side by side, and its grouping carries a third. The separation falls almost entirely inside the allocator groups, which means the effect follows the variable the study manipulates and not the one it controls for.

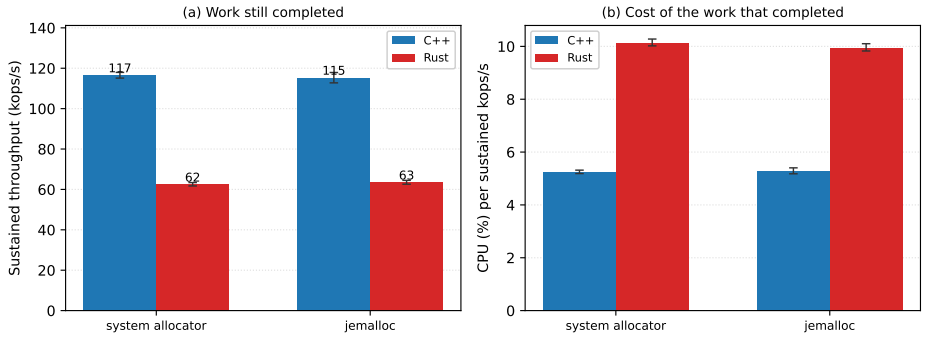


Figure 6.7: Behaviour at 784 kops/s offered, well beyond the 335 kops/s knee. Bars are medians over 20 repeats with bootstrap 95% intervals, grouped so that the two bars inside a group differ only in language. **(a)** What each variant still completes: C++ sustains a factor of 1.8 more than Rust. **(b)** Total CPU divided by the work that actually completed, which is what rules out the reading that the variant delivering less is simply doing less. It spends 1.9 times as much CPU per sustained operation, and the CPU figures are summed across threads.

Where that effort goes is narrowed by measurement and not left open. Past saturation the producers spend almost all their time in two places recorded in Table ??: the bounded blocking queue and the completion handle. Section ?? isolates those primitives, and they do not reproduce the full-service ordering at significance. That rules them out *alone* as the whole explanation for a 1.8× gap. As in Section ??, nothing in the borrow checker, in bounds checking, or in ownership is implicated, and the finding should not be quoted as though it were.

Three things again bound what the finding licenses. It is one design, a shared-nothing service with one request outstanding per producer and a queue bound of 1 024, and the

depth of the collapse depends on all three. It is measured with a harness whose producers block per request, so it describes what this shape of client experiences and not what a pipelined one would. And the absolute figures belong to this host. What travels is the ordering and its cause. Below saturation the language choice did not matter. Above it the standard library's synchronisation primitives did, and the crossover is sharp enough that a comparison stopping at nominal load would have concluded the opposite.

That crossover also bears on the closest prior work, which reports a comparable degradation past a threshold request rate and attributes it to the asynchronous runtime [**arvidsson2025async**]. Section **??** takes up what it means that the same shape appeared here with no such runtime on either side.

Table 6.5: What each variant still sustains at 784 kops/s offered, well beyond the 335 kops/s knee: the median over 20 repeats above its bootstrap 95% confidence interval. Below saturation achieved throughput is pinned to the offered rate by construction and can express nothing about the languages. Here it is the only thing that can. CPU utilisation exceeds 100% because it is summed across threads, so 600% is six busy cores of the fourteen pinned.

Variant	sustained throughput (kops/s)	CPU (%)
C++ (system alloc)	116.6	611.6
	[115.1, 117.9]	[610.0, 612.2]
C++ (jemalloc)	115.4	609.4
	[112.8, 117.9]	[607.8, 611.9]
Rust (system alloc)	62.4	632.7
	[61.7, 63.4]	[630.8, 634.2]
Rust (jemalloc)	63.5	633.1
	[62.6, 64.5]	[631.6, 634.5]

Table 6.6: Exact paired permutation test of C++ against Rust at 784 kops/s, the allocator held fixed, Holm-corrected across the same family of 14 tests as Table **??** rather than as a family of its own. "Repeats" counts how many of the 20 paired differences agree with the sign of the median, which a statistic computed from their mean cannot show. No latency percentile is tested here, for the reason fixed in advance in Section **??**.

Metric	Allocator	Effect (Rust − C++)	Repeats	p	Holm
Sustained throughput (kops/s)	system	−54.0 [−55.8, −51.9]	20/20	0.0000019	0.000027
Sustained throughput (kops/s)	jemalloc	−52.2 [−53.1, −49.6]	20/20	0.0000019	0.000027
CPU utilisation (%)	system	+20.6 [+19.1, +23.3]	20/20	0.0000019	0.000027
CPU utilisation (%)	jemalloc	+23.6 [+20.8, +25.2]	20/20	0.0000019	0.000027

6.4.5 Contended queue and completion primitives (RQ1)

The overload result attributes its gap to the two forced-idiom synchronisation constructs, but that attribution was argued from the shape of the full-service measurement. This subsection isolates those constructs alone.

7 producer threads hammer one bounded queue of capacity 1 024 while a single consumer drains it. Each producer submits a no-op job carrying a one-shot completion handle and blocks until the consumer signals. No `ShardCore` work runs in the path. C++ uses the

hand-written `BoundedQueue` of Section **??** with `std::promise/std::future`. Rust uses `sync_channel` and a fresh `mpsc::channel()` per request. Each timed window lasts two seconds after a warmup. 20 interleaved repeats are summarised as medians with bootstrap intervals and tested with the exact paired permutation procedure of Section **??**. Table **??** reports the result.

Table 6.7: Throughput through the bounded queue and completion primitives alone, with 7 contending producers and queue capacity 1 024. Values are medians over 20 interleaved repeats above bootstrap 95% confidence intervals. The paired test compares C++ against Rust on sustained throughput (kops/s). Measured on the host of Section **??**.

Language	Throughput (kops/s)	Mean ns/op	Paired p
C++	55.0	18178	—
	[53.7, 58.4]	[17110, 18632]	
Rust	58.7	17045	0.226
	[56.8, 61.8]	[16169, 17594]	

The isolated primitives do not reproduce the overload ordering. C++ sustains 55 kops/s and Rust 59 kops/s, a gap of only 1.1× with $p = 0.226$. The full service at 784 kops/s separates by 1.8×, with the corrected p at the family floor. The honest reading is therefore that the overload divergence involves extra work the microbenchmark strips away: the larger tagged `Request` on the C++ side, the worker's variant dispatch, and the map operations that still execute in the overload regime. What the result does establish is still useful. An orders-of-magnitude difference in the primitives alone is ruled out, and the search for the mechanism moves to the rest of the handoff path and not to the safety model.

6.4.6 Resource use (RQ3)

Table **??** reports resource use at the same sustained load. Peak resident set size separates the four variants cleanly, and it does so whichever way the family is read. Its intervals exclude zero, and its effect keeps its sign under both allocators. Figure **??** plots the same measurement. Its middle panel is where the finding of this section sits, with the two effects inside peak RSS drawn as paired differences on a shared axis.

Two effects are present in it, not one, and they are not the same size. Linking `jemalloc` costs 32.9–34.4 MiB in either language. The Rust build costs 1.9–3.4 MiB more than the C++ build under either allocator, with a bootstrap interval of 1.8 to 3.5 MiB. Resource use follows both, and the allocator effect is the larger by an order of magnitude. That ordering comes partly from the fairness controls and is not a fact about `jemalloc` in general. Both builds pin the arena count identically, so that neither language's default could become an accidental advantage, and pinning it high is what makes the arena metadata a large fixed cost on both sides. A deployment tuning `jemalloc` for footprint instead of for comparability would not pay that cost.

The one row in the upper block of Table **??** that does not clear the correction is peak RSS under `jemalloc`, and the reason lies in the statistic and not in the software. The interval there is tight, nowhere near zero, and 18 of the 20 paired repeats agree. What defeats the test is a handful of runs reporting a resident set some 30 MiB below their variant's median while staying indistinguishable in throughput, wall time, and error count. The

permutation statistic is a mean of paired differences, and one such run dominates it. This is the case the reading rule of Section **??** was written in advance to cover, and Section **??** takes up both the rule and the measurement defect behind it.

CPU utilisation behaves differently from either, and the difference depends on the regime rather than being noise. At this below-saturation operating point Rust draws slightly *less* CPU, by at most 3.4 percentage points, consistently under both allocators and clearing the correction under both. Section **??** finds the sign flipped past saturation and the magnitude much larger. Achieved throughput adds nothing here, for the reason Section **??** gives. At a below-saturation point it is pinned to the offered load, and it is reported only to confirm that the point is sustained and not merely nominal, which it is to within 0.1%.

Two qualifications keep the memory finding the size it actually is. It is a difference in fixed cost and not in cost per context. At the 5 000 active UEs used in the sweep, the eight-byte layout difference measured in Section **??** accounts for roughly 40 kB, two orders of magnitude short of the gap. Most of the gap is therefore runtime and allocator-arena overhead, which does not scale with the workload and would matter proportionally less in a larger deployment, not more. The second qualification is that the measurement establishes the difference without explaining it. No hardware counters or allocation profiles were collected, so the split between arena pre-reservation, per-thread caches, and static footprint is not resolved here. Both points are revisited in Chapter **??**.

The RQ3 finding is therefore neither the flat "no resource penalty" that overlapping intervals would suggest, nor a general performance penalty. Rust costs a few megabytes more of fixed footprint, reproducibly and under both allocators, but an order of magnitude less than the allocator choice itself. It is slightly cheaper in CPU while the service is coping, and considerably more expensive once it is not. On the bare operation of Section **??** it is the faster of the two.

Table 6.8: Resource use at a sustained offered load of 56 kops/s: achieved throughput, peak RSS, and total CPU utilisation. The median over 20 repeats above its bootstrap 95% confidence interval. Peak RSS is the one metric here whose intervals separate the four variants cleanly.

Variant	throughput (kops/s)	peak RSS (MiB)	CPU (%)
C++ (system alloc)	55.99	9.7	108.5
	[55.99, 55.99]	[9.6, 9.8]	[107.7, 109.8]
C++ (jemalloc)	55.94	42.6	108.0
	[55.93, 55.95]	[42.4, 42.6]	[106.9, 108.7]
Rust (system alloc)	55.99	11.6	105.0
	[55.99, 55.99]	[11.5, 11.6]	[104.0, 105.3]
Rust (jemalloc)	55.94	45.9	106.3
	[55.94, 55.95]	[45.8, 46.0]	[104.8, 107.2]

6.4.7 Safety via fault injection (RQ2)

This study does not claim that Rust has no bugs. It injects the catalogue of 14 defects set out in Table **??**, each as a small self-contained program flavoured with the subsystem's domain so that it maps onto a plausible bug in the service. The split into two groups is what the method of Section **??** turns on: seven classes from inside the region Rust claims to govern, and seven chosen precisely because safe Rust is expected *not* to prevent them.

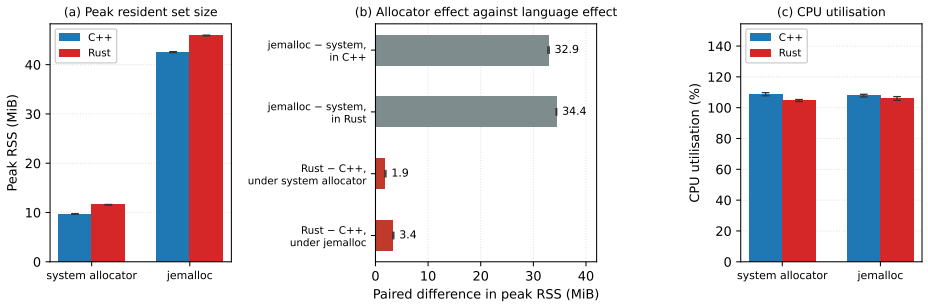


Figure 6.8: Resource use at a sustained offered load of 56 kops/s, grouped by allocator so that the two bars in a group differ only in language and the two groups differ only in allocator. Medians over 20 repeats with bootstrap 95% intervals. **(a)** Peak resident set size. **(b)** The two effects it contains, each as a paired difference over the same repeats: linking `jemalloc` costs 32.9–34.4 MiB, while the Rust build costs 1.9–3.4 MiB more than the C++ one. Both are real, and on a shared axis the allocator effect is an order of magnitude the larger. **(c)** CPU utilisation, summed across threads, where the difference runs the other way and is small.

Without the second group the outcome would be decided by the choice of test cases, confirming the expected result and never able to disturb it. With it, the study measures a boundary instead of demonstrating a claim. The fault runner holds the expectation recorded for each case in advance and reports a violation in either direction. A case that unexpectedly compiled, or unexpectedly failed to, would therefore surface instead of being absorbed into the conclusion.

Procedure. For each C++ defect the study builds and runs the program under AddressSanitizer (ASan), ThreadSanitizer (TSan) and UndefinedBehaviorSanitizer (UBSan), and records which tool, if any, detects it. The third is not a refinement of the other two but a separate axis. Signed overflow is invisible to both ASan and TSan, so without its own column the integer-overflow class would be recorded as undetected in C++ when in fact it is caught. No static analyser is run, for the reason set out in Section **??**. Its output is neither a compile-time rejection nor an observation of an execution, so it belongs to none of the three outcomes the taxonomy records. The C++ columns of Table **??** therefore report the tooling this study ran, not a ceiling on what C++ tooling could achieve. The matching construct is written in safe Rust and compiled. Where it compiles, it is then run under a bounded timeout and its observable behaviour recorded, because "it compiles" alone would not establish whether the defect survives into the running program. The catalogue, the runner, and the raw tool logs are published with the thesis.

Results. The outcomes are shown in Table **??**, measured with `g++ 13.3` (`-std=c++23`, `-O0` so the optimizer cannot elide an injected defect) and `rustc 1.97` (edition 2021). That C++ toolchain is the local development machine's and not the measurement host's `g++ 15.1.1`, and the catalogue was run there deliberately. Whether a program compiles, and what a sanitiser reports when it runs, are properties of the toolchain and not of the machine hosting it. That is the concrete content of calling this result environment-insensitive in Section **??**.

Within its scope the result is unambiguous. Every one of the seven memory-safety and

data-race defects is a compile error in safe Rust, so none can be written at all. In C++ each is valid code whose defect shows up only at runtime, and only if a test actually exercises the buggy path. ASan catches the memory-safety defects. The concurrency defects are caught only by TSan and are invisible to ASan. The overflow in the boundary group is caught only by UBSan and is invisible to both. No single C++ sanitiser covers the catalogue, and the three between them still leave 4 of the 14 classes untouched, with a fifth reported only intermittently. Runtime detection also means building a dedicated instrumented binary and paying the sanitiser's time and memory overhead, while the Rust errors are reported on every ordinary build whatever the test coverage. The parenthesised codes are the `rustc` diagnostics that fire. `E0499` is conflicting mutable borrows, `E0502` a mutable borrow while a shared borrow is live, `E0382` use of a moved value, and `E0373` a closure outliving borrowed data as it escapes to another thread.

Two rows in that group say something the other five cannot, and both were added to test the cost argument rather than to repeat it. The use-after-move is the only in-guarantee class no C++ sanitiser reports at all. Admission answers the request from the context it has just moved into the store, and a moved-from object is valid but unspecified, so nothing is corrupted, nothing is undefined, and all three tools run clean while the acknowledgement goes out under an empty identity. Every other class in this group is caught by at least one of them, which is what makes this one the sharper form of the argument: the instrumented build is not merely expensive, on this class it is empty. The thread-unsafe handle is the other. Its diagnostic is `E0277`, a trait bound rather than a borrow error, because the rejection comes from `Send` rather than from the borrow checker. The C++ handle carries its thread-affinity rule in a comment, which is the only place C++ has to put it. In Rust that rule is part of the type and `thread::spawn` asks for it in its signature, so the compiler refuses without analysing the body at all. One caveat belongs with the first of these. A static analyser configured for it would plausibly flag the use-after-move, and `clang-tidy` ships a check for exactly this pattern. None is run here, for the reason given in Section **??**, so the row records what the sanitisers report and not the limit of what C++ tooling could reach.

All seven boundary cases behave as predicted, and they mark out the edge of the claim. Every one compiles in safe Rust and every one misbehaves. The deadlock case hangs in both languages and had to be killed by the harness timeout. The lost-update case ends in both languages with a silently wrong answer. Against an expected total of 40 000 increments the safe-Rust program recorded 25 734, losing 14 266, and the C++ program under ThreadSanitizer recorded 27 767, losing 12 233. Neither language is meaningfully better off here, which is the point of carrying the class at all. The reference cycle leaks in both, the release-build overflow produces a wrong TEID in both, and a failure inside one shard worker takes down more than that shard in both.

The forgotten guard is the newest liveness case and it separates two things the phrase "RAII prevents this" runs together. Rust does remove the ordinary form of the bug. The C++ version fails because a validation path returns before a hand-written unlock, and there is no hand-written unlock in the Rust version to skip. What Rust does not remove is the class, because a guard is a value and leaking a value is safe, so a guard parked anywhere that outlives the procedure leaves the context locked and the next worker waiting. Both

programs had to be killed by the timeout, and neither language's tooling said anything: no sanitiser reported the C++ version, because a lock that is never released is not a race and not a cycle, and there is nothing for a lock-order detector to close. Read beside the reference-cycle leak the pair is the same lesson twice. There leaking costs memory, here it costs the context.

The seam double-free is the case that does not report a new defect class at all, and it is in the catalogue for that reason. It is the double-free of the in-guarantee group, moved to the other side of the rule by being written the way a foreign-function boundary has to write it. Safe Rust rejects the double release with `E0382`, because ownership tracks the value. Handing a C caller a raw pointer is precisely the operation that ends that tracking, and from there `Box::from_raw` asserts ownership of whatever address it is given. Asserting it twice frees the storage twice, and the program aborts on the allocator's own check. The C++ half of the pair is deliberately unremarkable and unchanged, so the only column that moves is the Rust one. The guarantee is not a property of the language considered alone. It is a property of the language together with the boundary the component is deployed behind, and that boundary is the one Section **??** measures.

Two of the seven are cases where the C++ tooling is *ahead*, which is the outcome the two-sided catalogue was built to be able to produce. LeakSanitizer, which ships inside ASan, reports the C++ reference cycle as an unreachable allocation at exit. The safe-Rust cycle leaks in silence, because leaking is safe in Rust by explicit design and nothing in the default toolchain looks for it. That this belongs to the language, and is not an artefact of the injected example, shows in the work being done to change it. Proposals to add garbage collection to Rust are motivated precisely by shared-ownership structures that reference counting expresses "awkwardly, inefficiently, or both" [**hughes2025gcrust**]. UBSan reports the C++ overflow, because signed overflow is undefined. The Rust overflow is *defined* to wrap at `opt-level = 2`, which leaves `overflow-checks` off, so the program has a meaning and the meaning is wrong. That second contrast needs stating carefully, because it does not reduce to which language is safer. Defined-but-wrong is easier to debug than undefined-and-wrong, and no less incorrect, and neither is what "memory safe" is usually taken to promise.

The deadlock result needs a qualification the earlier reading of it did not have. ThreadSanitizer does flag the inversion in C++, but on some runs and not others: 5 of 12 repeats in the run behind this table, while the program deadlocks on every one. The rate itself is not stable across runs and should not be read as one, since repeated campaigns on this host have returned figures on both sides of half. What reproduces is the intermittency. The mechanism is specific, and it generalises beyond this catalogue. TSan builds its lock-order graph from acquisitions that *succeed*, so the cycle closes only when some thread completes an inversion without blocking. A run in which the first inversion blocks produces one edge and never the second, and the detector stays silent at exactly the moment the bug bites. A positive control that inverts the same two locks without ever blocking is reported every time, which is what establishes that the detector was live and not misconfigured. The honest statement for this class is therefore narrow. The tool catches the inversion the program survives, and it is unreliable on the one that deadlocks, which is a very different proposition for a continuous-integration pipeline than a clean catch.

The distinction all of this exposes is precise, and it is the one the phrase "thread safety" routinely blurs. Rust's concurrency guarantee is the absence of *data races*, meaning unsynchronised concurrent access to the same location. It is not a guarantee about *lock-acquisition order*, nor about the *atomicity of a composed operation*, nor about leaks, wrapping arithmetic, or fault containment, and it is suspended wherever a program writes `unsafe`. The lost update stays the quietest case, because no sanitiser reports it. With every access properly locked there is no race for a race detector to find.

Where the boundary classes are caught, and at what cost. RQ2 asks which defects are prevented, and also at what stage the rest are detected. For the boundary group the sanitiser columns leave that second half largely blank. Two Rust model checkers answer it, and the division of labour between them is the one a production Rust system has been validated with. Loom explores interleavings soundly in the release/acquire memory model, using bounded partial-order reduction to hold down the state space. Shuttle samples them with probabilistic concurrency testing, which scales to cases beyond Loom's reach [**bornholt2021shardstore**]. Both report both of the original boundary defects. Loom reports the deadlock as a closed cycle and fails the lost-update assertion, and Shuttle does the same. Each is paired with a control in which the defect is fixed, by consistent lock order and by holding the read-modify-write in one critical section, and both controls come out clean. The reports therefore come from the defects and not from the tools.

The cost is the part worth recording. Neither defect program runs under either checker as written. Each had to be re-imported against the checker's own synchronisation and thread types, because a tool that does not intercept the primitive cannot schedule it. Each also had to be shrunk to model size. The lost-update model drops from twenty thousand iterations per thread to one, and the deadlock model loses the sleep that widens its window on real hardware. So the answer for the Rust half of the boundary is *detected at test time, after a rewrite*. On the C++ half the sanitiser takes the program unchanged and, for these two classes, finds one unreliably and the other not at all. The two halves point in opposite directions, and neither language's story here is the simple one. The consequence for practice is developed in Chapter **??**.

The seam double-free is answered by a different tool and comes with a sharper limit. Miri interprets the program against the Stacked Borrows aliasing model, and it reports the second release precisely, as a dangling box reconstructed from a freed allocation. A control that releases the same handle once is clean, so the report comes from the defect rather than from anything `Box::from_raw` does in general. The stage for this class is therefore test time again, and this time without a rewrite. The limit is what a migration has to weigh. Miri interprets Rust and cannot execute a C++ host, so the program it checked uses a Rust stand-in for the host. The tool that finds this defect cannot be pointed at the configuration the defect ships in, which is a seam with C++ on the other side of it. Nothing in the toolchain checks that boundary, and the practical answer is the one Section **??** argues from the prototype: keep the seam small enough to review by hand, because review is what is left.

A defect that was not injected. Every outcome above comes from a defect written on purpose, and that is the standing weakness of the method. Whoever assembles a catalogue already knows what safe Rust prevents. One case in this study does not carry that weakness. A design review of the specification found that its concurrency section forbade submitting an operation outside the service's running window (before `start()`, or after `shutdown()`), while neither implementation checked for it. The prohibition was addressed to the caller with no mechanism behind it, and in that gap the two languages did not fail alike. The C++ submit path indexes the shard vector that `shutdown()` has cleared, which AddressSanitizer reports as a heap-use-after-free. Called before `start()` it is a segmentation fault on the zero page. In Rust both calls were already a panic, from the ordinary bounds check on the same indexing operation, and nobody had designed for it.

What makes the case evidentially different is where it came from, not how severe it is. It sat in code written to be correct, in the arm whose safety claim rests on ThreadSanitizer, reachable through the public API by calling documented operations in a documented order. It also survived every check this study runs: 6 C++ test binaries, 27 Rust tests, 223 hand-written and 793 000 generated conformance operations, and three sanitiser configurations reported clean. It survived them for one reason. None of them ever called an operation outside the running window. ASan finds this defect only once a test drives it, which is exactly the property the cost argument above assigns to runtime detection, seen here on the study's own artefact instead of on a planted one.

The direction in which it must *not* be over-read is the flattering one. Rust did not prevent this at compile time, and nothing in the borrow checker knows that a service must be started before it is used. What turned undefined behaviour into a diagnosable failure was a slice bounds check. That is a runtime guarantee, so it is the same kind of assurance the C++ sanitiser offers and not the compile-time kind Table **??** records. The honest claim is that checking by default turned a specification gap into a panic, not that a type system caught a lifecycle error. What is left is still the distinction this chapter keeps returning to: a panic that names its cause on every build, against a heap-use-after-free that needs an instrumented binary and a test that thinks to look. Both implementations now reject the call explicitly, in both directions, and both suites cover it.

Table 6.9: Defect-class taxonomy: measured outcomes for the injected catalogue. C++ under `g++ 13.3` with ASan/TSan/UBSan. Rust under `rustc 1.97`, edition 2021. "Compile error" means the safe-Rust program does not build, so the defect cannot be written. The lower block is the boundary group: defect classes chosen because safe Rust is *not* expected to prevent them, and does not. Two of those seven are the ones on which the C++ tooling is ahead. The body of this table is generated from `faults/results/taxonomy.csv`, so it cannot disagree with the run that produced it.

Defect class	ASan	TSan	UBSan	Safe-Rust outcome
Within the guarantee				
Use-after-free	detected	detected	not detected	compile error (E0499)
Double-free	detected	not detected	not detected	compile error (E0382)
Iterator invalidation	detected	detected	not detected	compile error (E0502)
Data race	not detected	detected	not detected	compile error (E0373†)
Unguarded shared mutation	not detected	detected	not detected	compile error (E0502†)
Use-after-move	not detected	not detected	not detected	compile error (E0382)
Thread-unsafe handle shared	not detected	detected	not detected	compile error (E0277)
Boundary of the guarantee				
Lock-order deadlock‡	not detected	intermittent§	not detected	compiles, deadlocks
Lost update	not detected	not detected	not detected	compiles, wrong result
Reference-cycle leak	detected	not detected	not detected	compiles, leaks
Integer overflow (release)	not detected	not detected	detected	compiles, wrong result
Worker-failure blast radius	not detected	not detected	not detected	compiles, shard outage
Double-free across the seam	detected	not detected	not detected	compiles, aborts
Forgotten lock guard‡	not detected	not detected	not detected	compiles, deadlocks

† Several diagnostics fire for these cases. The first is shown (the data race also raises `E0382` and `E0502`, the unguarded mutation `E0524` and `E0597`). ‡ On these rows the instrumented binary was killed by the harness timeout rather than reporting anything. Nothing is corrupted and nothing is undefined, so the tools that detect corruption have nothing to say. The forgotten guard is silent under all three, and the lock-order case is the one TSan sometimes catches, as the next note records. § ThreadSanitizer reports the lock-order inversion on some runs and not others, measured at 5 of 12 repeats, while the program deadlocks on every one. The detector builds its lock-order graph from acquisitions that *succeed*, so a run whose first inversion blocks never completes the second edge of the cycle.

Read as a whole the table makes two things visible at once. Above the boundary rule the entire Rust column is a compile-time rejection, while the C++ columns are a patchwork of runtime detections and outright misses. Below the rule the Rust column joins them, and on two rows it falls behind. Figure **??** draws the same 14 outcomes as a matrix, where that change of character at the rule is the shape of the picture and not something to be read off row by row. The outcomes are generated from the runner's machine-readable log (`faults/results/taxonomy.csv`), the model-checking outcomes from `faults/results/model_checking.csv` and the interpreter's from `faults/results/miri.csv`. Table and figure come from the same log, so neither can drift from the other.

6.4.8 Migration (RQ4)

Incremental adoption is assessed with a working prototype, built in the direction a real migration takes: an existing C++ host keeps a Rust component behind a C ABI. The Rust service is exposed through a thin `extern "C"` seam in a *separate* crate that depends on the unmodified core library, so adopting the component needs no change to the component itself. The C++ host sees only an opaque handle and a set of plain-C entry points (a hand-written header, no generated bindings), and it drives the full UE lifecycle across

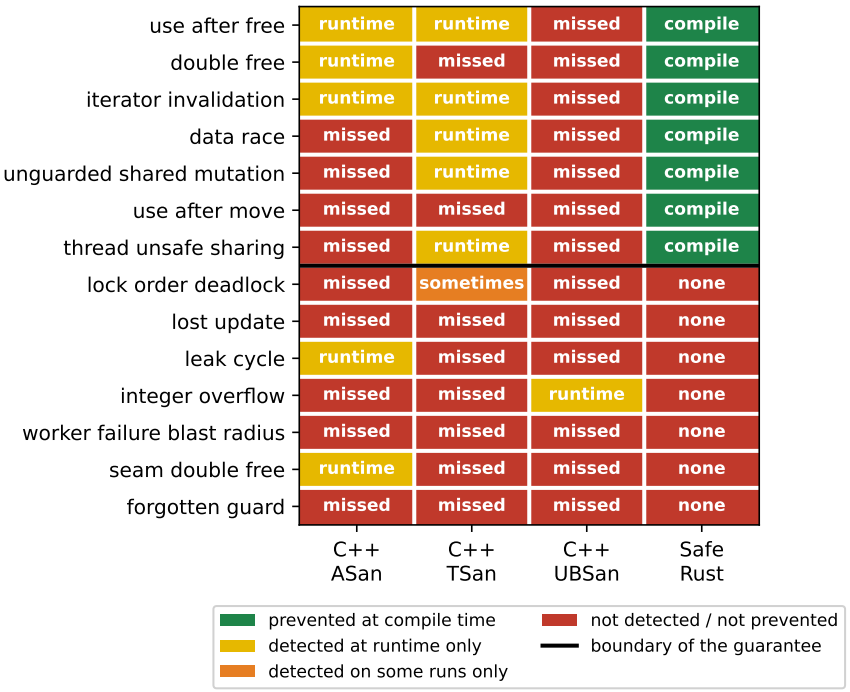


Figure 6.9: The outcomes of Table ?? drawn as a matrix, generated from the same `faults/results/taxonomy.csv`. Above the rule the Rust column is a single colour: every defect is a compile error, so none can be written. The C++ columns there are a patchwork instead, and no one sanitiser covers them. The use-after-move row is the one no sanitiser covers at all. Below the rule the Rust column joins them, and on the reference-cycle leak and the release-build overflow the C++ tooling is ahead. The lock-order deadlock is the one cell where a tool reports intermittently: ThreadSanitizer flags the inversion in 5 of 12 runs while the program deadlocks on every one.

the boundary. The error model crosses intact, so a duplicate connection setup returns `InvalidState` through the seam. Figure ?? shows the resulting structure and where the `unsafe` sits.

The prototype yields concrete interop-cost figures, summarised in Table ??. The seam is small: 11 C-ABI entry points in 231 lines of Rust, plus a 66-line C header, with no third-party FFI dependency. The safety boundary is clean. The core library stays free of `unsafe`, and the raw-pointer handling the C ABI requires is confined to the seam crate, in its 9 `unsafe` functions, under a single documented safety contract. The C++ side never sees a Rust type.

The per-call cost is measured twice, because the two measurements answer different questions and only the second answers RQ4's. The first is the crossing itself. A do-nothing `extern "C"` call costs 2.54 ns, which is an ordinary indirect call and negligible next to the microsecond-scale round-trip of a signalling operation. That bounds the crossing and says nothing about what the seam does to a real operation, because an empty call and a full round-trip are not the same experiment. The round-trip is dominated by a cross-thread handoff, and an argument that the handoff "would be paid identically anyway" is reasoning standing in for a measurement.

The measurement is therefore made directly. The same signalling operation, on the same service configuration and in the same loop, is invoked natively from Rust and then from C++ through the seam, interleaved over 15 repeats so the pair is matched. The paired

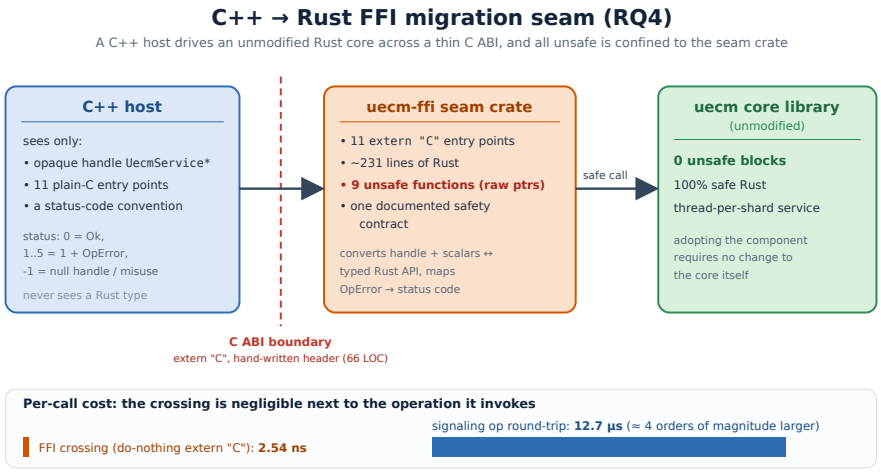


Figure 6.10: The C++-to-Rust FFI migration seam. The C++ host sees only an opaque handle, 11 plain-C entry points, and a status-code convention. A thin `extern "C"` seam crate concentrates all of the raw-pointer `unsafe` (9 functions) and calls the unmodified Rust core, which remains free of `unsafe`. The do-nothing crossing costs 2.54 ns against a 12.7 µs operation. Measured against the same signalling operation called natively, the seam's cost per operation is bounded at 10.91%.

difference is +192 ns per operation, with a bootstrap interval of $[-954, +1341]$ ns and an exact paired p of 0.239. No cost is detected. More usefully, the interval bounds any cost the measurement could have missed at 10.91% of the operation. The interleaving is not optional here. A single pass of each, run while preparing this section, made the seam look *faster* than calling natively by a third, because the run-to-run spread of the handoff on a shared host is far wider than anything a C ABI could contribute. Figure **??** shows the repeats that produced those numbers, and the left panel is that spread. The pairs cross, and the range they cover is an order of magnitude wider than the difference under test.

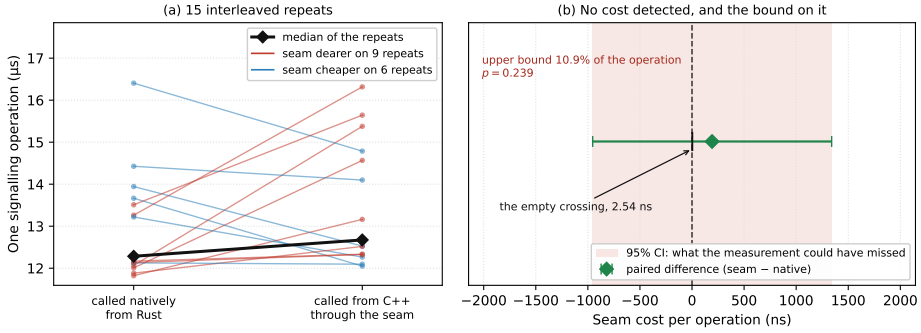


Figure 6.11: The interoperability cost measured on a real operation rather than inferred from an empty call. **(a)** The same signalling operation invoked natively from Rust and from C++ through the seam, one line per interleaved repeat, with the medians in black. **(b)** The pairs reduced to their difference: +192 ns per operation with a bootstrap 95% interval of $[-954, +1341]$ ns and an exact paired p of 0.239. No cost is detected, and the upper end of that interval is what bounds the cost the measurement could have missed at 10.91% of the operation. The empty `extern "C"` crossing, 2.54 ns, is marked for scale rather than as the answer: it is indistinguishable from zero on the axis the operation is measured on.

The main engineering effort therefore goes into *designing* a C-compatible API surface, with an opaque handle, scalar-typed operations, and a status convention. It does not go into runtime overhead. The reverse direction (Rust calling into C++) is symmetric, but it needs a C-ABI wrapper on the C++ side and it brings `unsafe` back to the Rust caller.

The prototype and its measurements are published with the thesis, and they inform the migration guidelines of Chapter **??**.

Table 6.10: FFI migration prototype: seam size, safety boundary, and per-call interop cost (C++ host calling the Rust component across a C ABI, `-O2`). The core library is unmodified and `unsafe`-free. All interop is confined to the seam crate. The last three rows are the direct comparison: one signalling operation on the same service configuration and the same loop, invoked natively and then through the seam, so the difference isolates what the boundary adds rather than what the operation costs.

Metric	Value
C-ABI entry points	11
Seam size (Rust `lib.rs`)	231 LOC
Hand-written C header	66 LOC
`unsafe` functions in core library	0
`unsafe` functions in FFI seam	9
Raw FFI crossing	2.54 ns/call
Signaling op, native caller	12.3 µs/call
Signaling op, through the seam	12.7 µs/call
Seam cost per operation (paired)	+192 ns $[-954, +1341]$
as a fraction of the operation	+10.91% at the upper bound

Three cost axes the seam does not reach. RQ4 asks what incremental adoption costs in effort, interoperability and risk. The seam answers the middle term with a measurement, and it leaves the other two to its own line counts, which describe one component and say nothing about the work of building it. Three further axes are measured here, chosen because a team asks each of them before any of the questions answered so far: how long the toolchain takes, how large the artefact is, and how much code of unknown origin a configuration pulls in. Table **??** collects them.

Table 6.11: Adoption-cost axes beyond the seam. Build time is the median of 15 interleaved repeats at 14 parallel jobs on the local development machine (`g++ 13.3`, `rustc 1.97`), warm-up repeats discarded. "First-party only" rebuilds the same Rust targets with the dependency tree left warm, so the difference above it is what the ecosystem costs and not what the compiler does. Footprint is the `conformance_runner` binary, the tightest matched executable pair in the study, reported at each toolchain's default linkage and then at both matched policies, because the defaults differ and the ordering depends on which policy is held. Crate counts are transitive and come from `Cargo.lock`. The `unsafe` cell is empty for C++ because the language marks no such region, so no count exists to compare.

Measure	C++	Rust
Clean build, core library only (first-party source on both sides)		
Wall clock (s)	1.9	1.2
CPU time (s)	2.6	3.9
Clean build, every default target		
Wall clock (s)	5.7	8.1
CPU time (s)	35.9	64.8
CPU time, first-party only (s)	35.9	29.6
Incremental rebuild after one edit		
Source file changed (s)	1.2	3.6
Core header changed (s)	5.8	3.6
Static footprint, `conformance_runner` (KiB)		
Shipped, each toolchain's default linkage	106	723
Matched: standard library linked statically	1577	723
Matched: standard library resolved dynamically	106	269
Shared standard library left on the host	2531	5295
Third-party code resolved		
Transitive crates, compared component	0	0
Transitive crates, matched-allocator arm	0	8
Transitive crates, benchmark harness	0	17
`unsafe` sites in that third-party code	—	1064

Two of the three are environment-insensitive in the sense Section **??** gives the defect catalogue. Binary sizes and dependency counts are properties of the toolchain and not of the machine hosting it, so they carry from this measurement unchanged. Build time is not, and it is treated accordingly. It was taken on the local development machine over 15 interleaved repeats at 14 parallel jobs, with the first repeats discarded so that a cold page cache is not read as a property of a compiler. The absolute seconds describe that machine. What travels is the paired ratio between two toolchains measured alternately on one host, which is the same reason the campaign interleaves its variants.

Compile time separates the compiler from the ecosystem. On the matched scope, the core library alone with no third-party code on either side, Rust finishes sooner and C++ spends less machine time: 1.9 s against 1.2 s of wall clock, and 2.6 s against 3.9 s of CPU. The two orderings disagree because they measure different things, and here the wall clock is the weaker of the two. The C++ library is two translation units, so at 14 available jobs only two can ever run, while `rustc` splits its one crate into codegen units and kept about three cores busy. Wall clock at this scope therefore rewards the file layout of this codebase as much as either toolchain. CPU time is the comparison that does not depend on it.

The full build is where the practical objection to Rust adoption usually lands, and at first reading it holds. Every default target costs 64.8 s of CPU against 35.9 s, a factor of 1.8. Breaking that figure apart moves it somewhere more useful. Rebuilt with the dependency tree left warm, the same Rust targets cost 29.6 s. So 54% of the clean build is third-party code, and the project's own code compiles in 0.8 times the CPU `g++` needs for its equivalent. On this codebase the compiler is not what makes the build slow. Two qualifications bound that reading. The two target sets are close and not identical. And the C++ dependencies are not absent, they are precompiled into the distribution as shared libraries, which is a real benefit of a mature ecosystem and not an artefact of the accounting.

The incremental loop is quicker in C++ and less predictable. After one edit to an implementation file, C++ rebuilds in 1.2 s, which is 3.1 times quicker than Rust's 3.6 s. After one edit to a core header it takes 5.8 s. That is 1.6 times slower than Rust and, measured in CPU, indistinguishable from building the tree from scratch. The Rust figure does not move. Touching every file in the crate costs 3.8 s against 3.6 s for touching one, because the unit of compilation is the crate and not the file. So the edit-compile loop is faster in C++ where an edit stays inside a translation unit, and slower where it does not. Which case dominates a working day depends on a codebase's header discipline, not on either language. The shape of that result will be familiar from RQ1. The C++ figure is the better one at the median and the worse one in the tail.

The footprint ordering reverses with the linkage policy. Built as each toolchain ships by default, the Rust `conformance_runner` binary is 6.8 times the size of the C++ one, 723 KiB against 106. That comparison is between two linkage policies and not between two languages, because `g++` leaves `libstdc++` to the dynamic loader while `rustc` links its standard library into the binary. Measured at a matched policy the gap narrows and then changes sign. With both standard libraries resolved dynamically, C++ is smaller by 2.5 times. With both linked statically, Rust is smaller by 2.2 times.

The reversal has a measured cause and not an argued one. 47% of the sized symbols in the statically linked C++ binary are locale and stream facets, among them the money, date and wide-character facets this program never instantiates, because a static `libstdc++` link brings the facet table in as one set. Neither dynamic configuration is free either, because each leaves a library on the host to be paid for once: 2531 KiB for `libstdc++` against 5295 KiB for Rust's shared standard library. The conclusion worth carrying is that the question has three answers, and the 6.8× figure is the least informative of them. For a self-contained artefact, which is what a container image or an appliance wants, the ordering favours Rust.

The compared component resolves no third-party code at all. The standard objection to Rust in a regulated codebase is transitive-dependency risk. On the artefact this study compares it does not arise: 0 crates and 0 `unsafe` sites, which turns the dependency-free claim of Section **??** into a measurement instead of an assertion. A surface appears in the configurations around that component. Linking the matched allocator

resolves 8 crates. Of those, 5 reach the artefact and 3 only run while it is being built. That distinction matters, because a build dependency puts no instruction in the binary and still runs code on the build machine. The benchmark harness resolves 17. Counted with one syntactic rule across their sources, those two trees hold 623 and 1064 `unsafe` sites.

Two readings should be kept apart. The count bounds where an audit would have to look, and it is not a risk score, because a crate that concentrates its `unsafe` in one large block scores below one that spreads the same work across many small ones. The C++ column is empty by construction and not by advantage. The language marks no such region, so the whole program stands where Rust's `unsafe` blocks stand, and entering a zero there would be a category error. What the axis does establish is where the unaudited code in this project actually sits. The component that would ship carries none of it, and the largest surface belongs to the apparatus that produced the measurements.

7 Discussion

7.1 Summary of findings

Table **??** collects the headline finding for each research question, together with the strength of the current evidence. The safety results (RQ2) do not depend on the measurement environment at all. The migration results (RQ4) depend on it along one axis only, because build time is a timing measurement, which is why it is reported as a paired ratio. The performance and resource results do depend on the environment, and the honest division now falls inside RQ1 and not between chapters. The saturation knee, the overload comparison, and the resource comparisons are resolved on the host of Section **??**, and they replicate across the allocator control. The far tail below saturation stays unresolved, for reasons that belong to the machine and not to the analysis.

Table 7.1: Summary of findings per research question and the current strength of evidence.

RQ	Headline finding	Evidence status
RQ1	The answer divides at saturation. Below it the variants agree to 2 μs (2.1%) at the median and no tail ordering is resolvable. The end-to-end number is only 0.250% operation, so it cannot bound a language effect in any case, while the operation measured apart is 1.6× faster in Rust. Past saturation they diverge decisively: C++ sustains 1.8× the throughput of Rust at 784 kops/s, in 20/20 repeats under both allocators.	Overload and operation: resolved. Far tail: unresolved on this host. Mechanism: tested. Primitives alone insufficient
RQ2	All seven injected memory-safety and data-race classes are prevented at compile time in safe Rust, where in C++ each compiles and is caught, if at all, only at runtime and by no single sanitiser. The guarantee stops there: the seven boundary classes compile in both languages, and on the leak and the release-build overflow the C++ tooling reports what Rust's does not.	Final
RQ3	A memory penalty that reproduces: 1.9–3.4 MiB more peak RSS in the Rust build under *both* allocators, an order of magnitude less than the allocator's own 32.9–34.4 MiB. CPU inverts across the regimes: Rust draws up to 3.4 percentage points less below saturation and 1.9× more per completed operation above it.	Memory: resolved, unexplained. CPU: resolved in both regimes, opposite signs
RQ4	The seam is small and keeps the core `unsafe`-free at compile time. Measured against the same operation called natively, no cost is detected and any tax is bounded at 10.91% of an operation. The real cost is API design. The core remains exposed at run time to memory errors in the C++ host it shares an address space with. On the adoption axes beyond the seam, 54% of the Rust clean build is third-party code and the project's own code compiles in 0.8 times the CPU `g++` needs, the footprint ordering reverses with the linkage policy, and the compared component resolves 0 crates.	Final. Build-time absolutes are host-bound, the ratio is not

7.2 Discussion on the research questions

RQ1 (predictability). RQ1 has two halves, below saturation and beyond it, and the campaign answers them differently enough that reporting either alone would misrepresent the result.

Below saturation the languages are equivalent in central behaviour, by a margin no deployment would notice. The small paired difference at the median that does clear the correction is a fraction of an 80 μs operation. The far tail in this regime is a different mat-

ter, and one a deployment would also care about. There the result is a null this host cannot turn into an ordering, so the equivalence claimed here does not reach it. The central result is consistent with the wider evidence that Rust's safety checks are largely absorbed by modern out-of-order CPUs [**balasubramanian2017rust**, **emmerich2019drivers**]. It is also weaker than it looks, for the reason Section **??** gives. The operation the languages execute is 0.250% of what the sweep measures. An end-to-end null in a design dominated by a cross-thread wakeup therefore says something about the design and not about the languages. Measured on its own, the operation differs by 1.6×.

Beyond saturation the answer changes in kind, and this is the one result in the study that answers RQ1 on RQ1's own terms. The C++ build sustains 1.8 times the throughput of the Rust one while drawing more CPU per completed operation, reproducibly and at the corrected floor of the whole test family (Section **??**). So the practical form of RQ1's answer is not that the language choice is irrelevant to predictability. It is irrelevant *until the service stops coping*, and at that point it decides how much work still gets done. A comparison that stopped at nominal load, as most in this literature do, would have concluded the opposite with the same apparatus.

That reframes the closest prior work instead of merely agreeing with it. Arvidsson and Kaller measure this quantity for 5G control-plane procedures and find Rust's 99th percentile degrading faster beyond a threshold request rate. They trace it to stackless `async/await`, which permits a context switch immediately before an I/O operation begins [**arvidsson2025async**]. Section **??** accepts that attribution, and this study was built to exclude the confound behind it: one worker per shard over a bounded blocking queue, and no asynchronous runtime on either side.

Section **??** committed in advance to what each outcome would mean, and the second branch is the one that fired. A degradation past a threshold rate appeared with the concurrency model held fixed, so the asynchronous runtime cannot be the only place such an effect starts. The prediction also named where to look first if that happened: bounds and overflow checking [**zhang2022runtime**, **harris2023morello**]. That part of it was wrong. The effect appears only once the handoff path is contended, which points instead at the two standard libraries' blocking primitives recorded in Table **??**. Section **??** then tests whether those primitives alone explain the gap, and they do not reproduce the full-service ordering at significance. That narrows the cause to extra work on the C++ handoff path and not to the safety model. Recording that the prediction resolved against its own first hypothesis, and that the follow-up microbenchmark refined the attribution instead of confirming it, is more useful to the next study than quietly swapping in the explanation that survived.

RQ2 (safety). The result does not depend on the environment. What makes it interesting is not the half that confirms the expectation, but the half that bounds it. Within its scope the guarantee is absolute in one specific sense, and the difference from C++ is one of kind and not of degree. A compiler that refuses to build a program rules the defect out of every execution. A sanitiser rules it out only of the executions an instrumented test happens to take. What is absolute is the execution coverage, not the breadth of the catalogue. The scope itself is still the 14 programs written for it. That fits the CVE

evidence that Rust memory-safety bugs always involve `unsafe` [**xu2021rustcve**], and the caution that unsafety can still spread through dependencies [**evans2020rust**], which is why keeping the core `unsafe`-free matters. The seam double-free is that evidence reproduced under controlled conditions: the one class in this catalogue that safe Rust rejects and unsafe Rust does not.

The one case that reaches past the written programs is the lifecycle defect of Section **??**. It was found in the implementation, not written for the catalogue, and it revises the picture in two directions at once. It strengthens the cost argument. A real defect in code written to be correct survived the entire verification apparatus of this study, meaning tests, parity vectors, and three sanitisers, purely because nothing exercised the path. Runtime detection that depends on test coverage is a weaker proposition in practice than it looks in a table where every row is driven deliberately.

It also narrows what may be credited to Rust, because the language's contribution there was a bounds check and not a proof. Safe Rust turned undefined behaviour into a deterministic panic, which is a different and lesser claim than preventing the error. The honest generalisation is that the guarantee worth budgeting for is *checked by default* and not *ruled out at compile time*. The root cause is worth naming too, because it is a risk of the method and not of either language. A specification that states a prohibition without assigning a mechanism leaves each implementation to fail in its own idiom. The two idioms here are a panic and a use-after-free.

Where the guarantee stops matters just as much to a vendor deciding what Rust buys, and the two boundary classes are instructive for different reasons. The lock-order deadlock shows that Rust reasons about *aliasing* and not about *acquisition order*. A `Mutex` guard is an ordinary value whose lifetime the borrow checker tracks perfectly, and tracking it perfectly says nothing about whether two threads take their locks in compatible orders. The tooling built around the language confirms that this is a boundary and not an artefact of the injected example. Static deadlock detection for Rust is an active research problem in its own right [**zhang2026deadlock**], and model checkers for Rust kernel code are motivated explicitly by the observation that ownership and borrowing leave data races, deadlocks, and livelocks reachable [**tang2025converos**].

The lost update is sharper still. It escapes ThreadSanitizer for precisely the reason it escapes the borrow checker, as Section **??** sets out. With every access correctly synchronised there is no unsynchronised access to find. The distinction it exposes, between a data race and a race condition, is the one the phrase "thread safety" routinely runs together, and Rust removes only the first.

The practical consequence is that adopting Rust retires one category of concurrency defect completely while leaving another untouched. The category left untouched is the one that fails silently, corrupting a counter or a state decision instead of crashing. A migration plan that treats "memory-safe" as "concurrency-safe" would therefore mis-budget its review and testing effort. That is why this boundary is reported as a first-class result and not as a caveat. Figure **??** draws the boundary itself, with the two halves of the guarantee on either side of the line and the mechanism that does the enforcing, or the absence of one, named in each panel. It carries the two concurrency cases rather than all seven outside the guarantee, because they are the pair that makes the distinction, and Table **??** holds

the full catalogue.

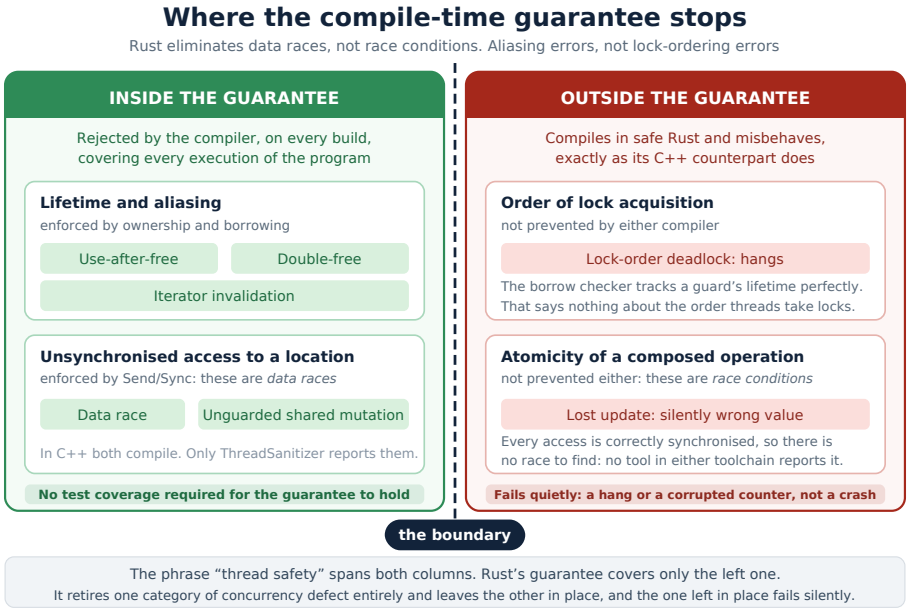


Figure 7.1: Where the compile-time guarantee stops. On the left are the classes safe Rust rejects outright, grouped by the *mechanism* that rejects them rather than by defect family: ownership and borrowing for lifetime and aliasing errors, which is why iterator invalidation sits with them, and the `Send`/`Sync` discipline for unsynchronised access to a location. On the right are the classes the catalogue included to find the edge, which neither compiler prevents. The diagram names one class per mechanism and is not the catalogue, which Table **??** lists in full: the use-after-move belongs with the ownership cases on the left and the shared thread-unsafe handle with the `Send`/`Sync` ones, while the leak, the release-build overflow, the worker-failure case, the seam double-free and the forgotten guard sit alongside the two on the right. The phrase "thread safety" spans both sides of the line, and the guarantee covers only the left. The line itself moves once more than the diagram can show, because a program that writes `unsafe` crosses it voluntarily.

RQ3 (overhead). The answer splits by resource, and the splits do not all point the same way. On memory there is a penalty. The Rust build carries a few MiB more peak resident set than the C++ build, and it keeps that sign under both allocators. Two things make it credible beyond the p-value. Peak RSS is a run-level maximum and not a tail statistic, so the scheduling jitter that destroys the p99 does not touch it. And the effect is an order of magnitude smaller than the allocator's own, which is the useful way to size it. A team choosing between these two languages moves less memory than a team choosing between these two allocators.

On CPU the answer depends on the load, and that dependence is the finding rather than a complication. Below saturation Rust draws slightly *less*, consistently under both allocators. Past saturation the sign flips and the magnitude grows (Section **??**). A single number for "the CPU cost of Rust's safety model" would therefore average two regimes that behave in opposite directions, and neither figure is the safety model in any case. Throughput splits the same way and for the same reason. Below saturation there is nothing to measure, because achieved throughput tracks offered load by construction. Above it sits the largest effect in the study.

Three things bound what the memory result licenses. It is small in absolute terms, and it is a fixed cost rather than a per-context one, so its share of a deployment's footprint falls as the deployment grows. The measured layout difference between the two `UeContext` types contributes almost none of it. It is also unexplained. Splitting it between allocator arena reservation, per-thread caches, and static footprint needs allocation profiles and hardware counters this study did not collect, which is the clearest single gap in the RQ3 instrumentation. And it should not be widened into a claim about Rust's safety model, because nothing here isolates bounds checking or ownership-driven copying as the cause. The mechanisms the literature has localised [**zhang2022runtime**, **harris2023morello**] are candidates to test, not conclusions to import. What can be said is narrower and still useful to a vendor sizing a control-plane node. Adopting Rust here costs a few megabytes of fixed footprint per process, and it buys a faster core operation (Section **??**) that this workload's structure never turns into throughput below saturation, and that turns into a deficit above it.

The sign of that result is worth defending, because one of the closest prior works in the domain reports the opposite sign, while two others agree with it. Agreement first, since it is quickly stated. A second Ericsson-hosted study of RAN node management software found Rust's peak memory usage generally higher than C's [**olsson2023ranmgmt**], and a matched pair of user-space device drivers outside telecom put the Rust version's footprint about 29% above the C version's [**sundvisson2026drivers**]. Neither separates the allocator from the language, which is the distinction Section **??** rests on, so they support the direction of the effect and not the way it splits.

The disagreement is the more instructive case. Karlbäck and Orö measure peak resident set on a RAN workload and find Rust the *lowest* of the three languages and C++ the highest, with the gap widening as concurrency rises. They call the pattern surprising and attribute it tentatively to the initial stack size of the green threads [**karlback2021ran**]. The two findings are compatible once it is clear that they measure different quantities. Their C++ runs on Boost.Fibers, which allocates a stack per fiber, so ten thousand concurrent sequences cost ten thousand stacks. Their Rust runs on Tokio, where a task is a heap-allocated future of a few hundred bytes. What their figures capture is therefore the per-context cost of the concurrency runtime, which is exactly why it grows with the number of contexts.

The design measured here has no green threads on either side. A fixed thread-per-shard pool leaves no per-context allocation to differ, and what remains is the fixed footprint reported above, constant in the number of UEs and not proportional to it. Their disagreement with this result is therefore between two concurrency runtimes and not between two languages, which is as direct a demonstration as the literature offers of why this study holds the concurrency model fixed. It also sharpens the caveat attached to the present number. It licenses nothing about Rust's memory behaviour under a task-per-UE design, which is a different experiment.

RQ4 (migration). The prototype shows that incremental adoption is technically cheap. A C++ host can call an unmodified Rust core across a thin C-ABI seam whose cost on a real operation is bounded at 10.91%, and all `unsafe` stays inside the seam. That bound

comes from the matched comparison of Section **??**, not from setting an empty crossing against a full round-trip. The two are not the same experiment, and on this host a single unpaired pass of each was enough to flip the apparent ordering. The dominant cost is therefore human and not machine: designing a C-compatible surface and, as the adoption studies report [**fulton2021adopting**, **crichton2024learning**], climbing Rust's learning curve. That makes migration risk an organisational concern and not a performance one, and the two largest documented adoptions into existing C and C++ codebases report the same ordering of costs [**li2026rustlinux**, **fan2026android**].

One boundary on that result has to be drawn explicitly, because the natural reading overstates it. "All `unsafe` confined to the seam" is a claim about what the compiler verifies, and it is true as measured. The core crate contains no `unsafe`, and every raw-pointer operation sits in 9 reviewable functions. It is not a claim that the core is protected at run time. Host and component share an address space, and a memory error on the C++ side can corrupt Rust-owned data however carefully the Rust side was checked. That is why the remedies proposed in the literature isolate the two heaps with hardware support instead of relying on interface discipline [**rivera2021galeed**, **kirth2022pkrusafe**], and the defect classes that arise at such boundaries have been catalogued in their own right [**li2022crosslang**]. So a migration of this shape buys less than "the Rust part is safe". It buys a migrated component that stops *contributing* new memory-safety defects, while it stays exposed to those of the host until the host is migrated too. That is still the right direction of travel, and it is a reason to sequence a migration so that the components sharing the most mutable state cross over together instead of one at a time.

The three axes measured beyond the seam sharpen the same conclusion, and they complicate two of its folk versions. Build time is the objection most often raised against Rust in a large codebase, and it survives only in the aggregate. The full build does cost 1.8 times the CPU. But 54% of that is third-party code, so the project's own source compiles for 0.8 times what `g++` spends on its equivalent. An organisation worried about build time should look at its dependency budget before it looks at the compiler. The edit-compile loop is the axis where C++ genuinely leads, and it leads unevenly. Its advantage holds for edits inside a translation unit and reverses for edits to a widely included header.

Binary size behaves worse than an argument would predict. The 6.8× figure that gets quoted compares two linkage policies, and once the policy is held fixed the ordering depends on which one was chosen. The axis that matters most for a regulated codebase is also the one with the cleanest answer. The component compared here resolves no third-party code and contains no `unsafe`, so the transitive-dependency argument against Rust adoption does not reach the artefact under discussion. The unaudited surface this project does carry belongs to its measurement apparatus.

7.3 Revisiting the hypotheses

Section **??** committed to three hypotheses before any measurement was taken. Stating their disposition explicitly, rather than letting the results speak for themselves, is what distinguishes a test from a description.

H1: supported below saturation, refuted above it. The hypothesis had two parts:

that the languages are performance-equivalent below saturation, and that any difference is confined to the far tail. The first is supported. The second is refuted, and not in the direction the wording anticipated, for the reasons Section **??** gives. The far tail below saturation, which is what H1 actually named, remains unresolved on this host. The difference that did appear lies in overload. The hypothesis as written would have led the campaign to look in the wrong place, and the result was found only because the sweep was pushed past saturation on methodological grounds rather than in expectation of it.

A second disposition is that the hypothesis is not well posed. "Performance-equivalent" was written as though it named one quantity. Section **??** shows it names at least three. There is the delivered operation below saturation, where the languages are equivalent. There is the code the compilers emit, where they are not, and where the end-to-end measurement dilutes the difference by more than two orders of magnitude. And there is sustained work under overload, where they are not equivalent either and the difference runs the other way. All three are true, and a hypothesis that cannot tell them apart is one this study should have stated more carefully before measuring.

H2: supported in its precise form, refuted in its loose one. The hypothesis was that safe Rust prevents at compile time "the memory-safety and concurrency defect classes that C++ leaves to best-effort detection". For the seven injected classes of that kind it holds without exception. The fault-injection study still refutes the broader reading the wording invites. The seven boundary classes are exactly the sort C++ leaves to best-effort detection, safe Rust does not prevent them either, and on the leak and the release-build overflow the C++ tooling is the one that reports. H2 is therefore accepted for memory safety and data-race freedom, and rejected for concurrency correctness at large. It also carries an unstated precondition, which the seam case makes explicit: the hypothesis is about *safe* Rust, and the same class it rules out returns as soon as a program writes `unsafe`. The second reading is the one practitioners tend to assume.

H3: supported, and tested the right way on the second attempt. The hypothesis was that interoperability cost is negligible against the operations invoked. Measured directly (Section **??**), no cost is detected and any per-operation tax is bounded at 10.91%. What supports the hypothesis is the bound, not the failure to detect a difference. The first version of this measurement did not deliver that. It set an empty crossing against a full round-trip and inferred the answer, which compares two different experiments and would have given the right conclusion for the wrong reason. The accompanying claim, that the real cost is engineering effort, is supported structurally and not statistically. The seam is small and mechanical, but designing its C-compatible surface was the part of the prototype that took actual design decisions.

7.4 Engineering guidelines for incremental adoption

The results are meant to be usable, and a vendor weighing Rust for a control-plane component needs more than a set of measurements. The following guidelines are the

practical form of the evidence above. Each is tied to the finding that supports it, and the limits of that support are stated and not glossed. A guideline quoted without its limit is how the RQ2 result becomes "the migrated component is safe" and the RQ4 result becomes "migration is cheap", and this study establishes neither. They are given in the order a first migration would run them.

Choose the first component for its interface, not its size. The boundary's runtime cost is bounded at 10.91% of an operation (RQ4), so the criterion for a first Rust component is not how small or how hot it is. It is how narrow and how scalar its interface is. The seam here stayed at 11 entry points because the subsystem's API was already lifecycle-oriented, typed, and free of shared mutable structures crossing the boundary. A component whose interface passes rich C++ objects, templates, or callbacks will need those designed out first, and that redesign is the schedule risk, not the FFI.

Put the ABI in its own crate and keep the core `unsafe`-free. Concentrating every raw-pointer operation in a separate seam crate that depends on an unmodified core (Section **??**) is worth adopting as a rule, not treating as an accident of this prototype. It keeps the dependency-propagation problem Evans et al. describe [**evans2020rust**] away from the safety-critical code, and it makes the reviewable surface of the migration a fixed, countable thing. Two qualifications belong with it. It bounds what has to be *reviewed*, not what can be *corrupted*. While the host remains C++ the two share a heap, so a team needing the stronger property has to look at in-process isolation [**rivera2021galeed**, **kirth2022pkrusafe**] and not at interface discipline alone. And the generated part of a seam is not above suspicion, because binding generators have their own defect history [**cai2026interop**]. Review it as code instead of trusting it as tooling output.

Do not re-budget concurrency review effort on the strength of memory safety. This follows directly from the boundary cases. Adopting Rust removes use-after-free, double-free, iterator invalidation, and data races from the defect budget entirely, and those are the classes that dominate the security statistics [**msrc2019memory**, **google2022android**]. It removes nothing from the deadlock and atomicity-violation budget. Lock-ordering discipline, invariants about which locks may be held simultaneously, and review for composed-but-not-atomic operations remain exactly as necessary as they were in C++, and they should stay in the code-review checklist and the design-review agenda after migration.

Prefer a shared-nothing structure to a shared-and-locked one, in either language. The strongest concurrency result in this study belongs to neither language. The thread-per-shard design removes locks from the hot path altogether, which is why the boundary defects had to be built as standalone programs instead of found in the implementations. Where state can be partitioned by a natural key, as UE state can be partitioned by `ue_id`, partitioning it turns a class of concurrency defects into a structural impossibility for both languages, and it does so without either compiler's help.

Budget for the learning curve, and measure it. The dominant cost identified here is human, which matches what adoption studies report [**fulton2021adopting**]. This thesis cannot quantify that cost, because it is a single-author study, so the guideline is correspondingly modest. Treat the first component as a training investment with a deliberately generous schedule, and instrument it, recording review time and defect density, so the second component can be planned on measured effort instead of assumed effort.

Measure the component, not the round-trip, and do not generalise either. The performance findings carry a methodological warning that travels further than the numbers do. An end-to-end request/response measurement of a design like this one is 0.250% language and the rest scheduler (Section **??**). Below saturation it will therefore return "no difference" between almost any two competent implementations, and reading that as evidence about the languages is a mistake this study made before it measured the components apart. A vendor running its own comparison should budget for both: the end-to-end measurement, which says what a deployment will feel, and the component measurement, which says what the language is doing.

Sweep past saturation, because that is where the languages stopped agreeing. This is the guideline the study would most like to have followed from the start instead of arriving at. Every conclusion it drew about performance while the sweep stopped at nominal load was a null. Each was correct at that operating point and silent about the one where the two implementations differ by 1.8× (Section **??**). The knee is not a detail at the edge of the graph. For a control plane it is the region that decides whether a signalling storm degrades service or ends it, and it is where a bounded, blocking design has no graceful behaviour left. A comparison that stops while the system is still meeting its offered load has measured the easy half of the question. Sweeping past the knee costs one more afternoon of machine time.

Beyond both of these, the reusable artifact is the benchmarking framework and not its current output. The decision it should inform is "measure this on your component, and measure it past the point where your component stops coping", not "Rust is fast enough, proceed".

7.5 Related work

Chapter **??** set out the ground this work stands on, in four strands. What follows positions the thesis against them and then treats at length the two studies it has to answer to. What matters here is the property the four share.

Cross-language measurement puts compiled systems languages close on time and energy [**bugden2022rust**, **pereira2017energy**], and it finds Rust's safety cheap at run time [**balasubramanian2017rust**, **levy2017kernel**]. That holds where timing is tightest: embedded hypervisors [**mo2023rustshyper**], production cloud infrastructure [**agache2020firecracker**], and packet processing within a few percent of a C reference [**emmerich2019drivers**, **xiao2024rpkt**]. Where a cost does appear it has been pinned down, to bounds, overflow and division checks [**zhang2022runtime**], to that

same bounds check set against hardware-enforced capabilities [**harris2023morello**], and to recurring anti-patterns [**cui2025perfbugs**]. A like-for-like port of one suite from C++ to Rust puts that bounds-check cost at 34% of a 46% geometric-mean overhead. It also cautions that where the compiler is credited with removing such checks cheaply, the measured code has sometimes declared its hot functions `unsafe` [**naylor2026cherisimt**]. That caveat does not apply to the `unsafe`-free core compared here. All of this work is average-case, and none of it reports behaviour past a saturation point, which is the regime this thesis targets.

The safety evidence is settled the same partial way. There are machine-checked foundations for the safe fragment and for the aliasing discipline `unsafe` code has to respect [**jung2018rustbelt**, **jung2021safe**, **jung2020stacked**]. There is confirmation that `unsafe` is used sparingly and that memory-safety CVEs always involve it [**qin2020rust**, **astrauskas2020unsafe**, **xu2021rustcve**]. There is auditing that finds defects at scale even so [**bae2021rudra**, **zheng2024security**, **qin2024safety**], and the caution that unsafety spreads through dependency chains [**evans2020rust**]. And there are the reports motivating the shift, set against a long record of memory corruption resisting runtime mitigation [**msrc2019memory**, **google2022android**, **google2024securebydesign**, **cisa2023memorysafe**, **oncd2024building**, **szekeres2013sok**].

The third strand qualifies the adoption argument. Section **??** states its central limit. Confining `unsafe` to a seam does not protect the Rust side, because foreign code in the same address space can corrupt Rust-owned data anyway [**rivera2021galeed**, **kirth2022pkrusafe**]. The seam has its own defect classes [**li2022crosslang**], and its tooling has its own bugs [**cai2026interop**]. Lighter crossing models exist for embedded use [**schuermann2023encapsulated**]. What that section does not cover is reach. Aliasing patterns expressible in C but not in safe Rust bound translation in principle [**emre2023aliasing**], interoperation is the central open problem in embedded Rust [**sharma2024embedded**], and `unsafe` concentrates at exactly these boundaries [**mccormack2026unsafe**]. The two largest documented adoptions into existing C and C++ report the resulting friction empirically [**li2026rustlinux**, **fan2026android**]. Automating the rewrite is an active line of work and not a solved problem, since current translators still emit code whose safety the compiler cannot validate [**hong2025ctorust**]. That is why the migration cost measured here is the cost of a hand-written seam.

The fourth strand is the domain and the method: the tail-latency and microsecond-scale literature [**dean2013tail**, **barroso2017microseconds**, **li2014tales**, **huang2017predictability**], thread-per-core structuring moving the tail in its own right [**enberg2019threadpercore**], O-RAN disaggregation [**polese2023oran**], control-plane latency as an active target in the core network too [**liu2023l25gc**, **park2025ctrlbench**], adoption evidence pointing at the learning curve rather than run-time cost [**fulton2021adopting**, **crichton2024learning**, **ardito2021verbosity**], and standard guidance on repeat-based evaluation [**georges2007rigorous**] and fault injection [**natella2016fault**]. Rust has mobile-network precedent in the datapath [**qazi2017pepc**] and, on the control-plane side, an open-source gNB-CU with a separated CU-CP and CU-UP over E1. That last one bears on the gap rather than

on the method. It establishes that the ASN.1 and SCTP protocol surface is no barrier in principle, but it is an existence proof and not a comparison, reporting no latency and carrying no C++ counterpart built to the same specification [**alsoran**].

Four systems studies come closest in method, though not in domain, because they too build the same component twice and attribute the difference. One compares IPC mechanisms implementing a single network function in both C++ and Rust [**li2021ipc**]. RedLeaf is evaluated against C baselines built for the purpose [**narayanan2020redleaf**]. A manual translation of the STAMP transactional benchmark suite into Rust is the closest published match to this study's design, because its workload is concurrent and both versions implement one specification [**suchert2023stamp**]. The fourth measures a Rust UDP tunnelling driver in the Linux kernel against an equivalent C driver [**gonzalez2023rustudp**]. That last one is worth a second look for its method and not its result. It reports Rust slower by 0.19% at $p = 1.5 \times 10^{-15}$, a difference overwhelming in statistical terms and negligible in operational ones. Keeping those two apart is what the pre-committed interval-versus-p rule of Section **??** exists to do. Like the wider literature, none of the four measures past saturation.

Five master's theses, all carried out at Ericsson, are the closest prior work by domain and deserve separate treatment. None of them is peer-reviewed, and none is indexed by any citation database, so they are cited here as the record of what a vendor has actually attempted rather than as established results.

Karlbäck and Orö compare Rust and Go against C++ for RAN applications on x86 and ARM, combining runtime measurements with development-efficiency metrics, and conclude that C++ remains the choice where performance is critical [**karlback2021ran**]. Their control deserves to be quoted and not paraphrased, because it is the clearest statement in this literature of the confound the present study is built to remove. The authors record that they deliberately did not write the same program three times. They "aimed for providing the best solution in each language, instead of trying to make the code as identical as possible between the languages", and they "stayed away from using the exact same solution in each language" [**karlback2021ran**]. For the question they ask, which is what a team would get by writing idiomatic code in each language, that is the right choice. It is the wrong control for the question asked here. The concurrency runtime then varies with the language (Boost.Fibers over libevent, goroutines, and Tokio respectively), so no measured difference can be divided between the two. Section **??** returns to this with a concrete case, where their resource results and the ones reported here have opposite signs, and the disagreement resolves into a difference between runtimes and not between languages.

Olsson asks a narrower version of the same question two years later, comparing Rust against C for RAN node management software, and finds Rust's execution speed on par with or better than C's, its peak memory usage generally higher, and its safety materially improved [**olsson2023ranmgmt**]. Two of those three findings agree with the ones reported here, including the resource result on which Karlbäck and Orö disagree with this study, which is taken up in Section **??**. What the study does not do is report percentiles, or drive the software past its capacity: the comparison is of execution time and peak memory at the loads the applications were run at.

Olsson and Stenström are the closest prior work on the concurrency model this thesis adopts, and their result is the sharpest illustration in the literature of why the model has to be held fixed. They build a thread-per-core packet processor on asynchronous Rust over the Glommio runtime, mirror two existing DPDK Eventdev applications with it, and measure throughput between 20% and 50% of the DPDK versions with latency they describe as far worse [**olsson2023asyncpackets**]. Read as a language comparison the result is damning. Read as what it is, a comparison between a young asynchronous runtime and a kernel-bypass framework tuned over a decade, it says almost nothing about Rust and a great deal about the runtime: an effect size of two to five times is not the scale at which code generation operates. It is also the reason the design measured here uses one thread per shard and no asynchronous runtime on either side, and Chapter **??** records the comparison that would separate the two properly.

Arvidsson and Kaller are closer still. They evaluate asynchronous programming frameworks, C++ with Boost.Fibers, Rust with Tokio and Monoio, Go, and Erlang, on 5G control-plane procedures, and report both median and 99th-percentile latency across a load sweep [**arvidsson2025async**]. Their result is the most direct challenge in the literature to the expectation this thesis set out with. Boost.Fibers shows the lowest latency at every load, and while the Rust runtimes track it at the median, their 99th percentile rises more steeply beyond a certain request rate. That is a tail divergence in the right domain, and it would be dishonest to omit it. It is also, however, a comparison in which the concurrency runtime co-varies with the language: the authors attribute the effect to stackless `async/await` permitting a context switch immediately before an I/O operation is initiated, which is a property of Tokio and Monoio rather than of Rust.

That attribution does not have to be taken on trust, because the runtimes have since been examined directly in a real-time setting. Škoudlil et al. dissect the execution model of asynchronous Rust against a thread-based C++ equivalent. They report that the runtimes' scheduling policies are "complex and difficult to understand due to the use of abstraction layers", and that Tokio specifically is "designed to maximize throughput rather than time determinism". Restructuring the same application onto prioritised threads over a different runtime yields response times that match classical response-time analysis, which the idiomatic work-stealing configuration does not [**skoudlil2025asyncrust**]. The tooling being built around those runtimes points the same way. Priority scheduling is something an asynchronous Rust runtime has to be given by a library, not something it already provides [**zhuang2026flowprioritize**]. The tail behaviour at issue is therefore a property of the runtime, and it changes when the runtime does. That confound is exactly what the frozen specification and the shared thread-per-shard model in this thesis are designed to remove. The two studies are complementary and not contradictory. Theirs establishes that a tail difference is observable in this domain. This one is built to determine whether the language is what causes it.

Emberg and Wadsten are the closest of the five in method, and the honest way to position this thesis is against them rather than against the framework comparisons. They build two implementations of one ASN.1 decompiler, in C++ and in Rust, have both call the same third-party C++ code, reach it from Rust by wrapping it in C, and compare the two with a rank-sum test. They also interview developers about the experience of writing

the Rust side [**emberg2023asn1**]. That is a matched pair, an FFI seam, a statistical comparison and an effort measurement, which is four of the things this thesis does, in the same industry, two years earlier. Three differences remain, and they are the ones that carry the gap. The object is a stateless transformation rather than a stateful, concurrent subsystem, so there is no concurrency model to hold fixed and no shared state to contend for. The measurements are taken at the load the benchmark happens to generate rather than swept to saturation and beyond, so no tail behaviour under overload is reported. And the safety side is argued from developer interviews rather than from a defect catalogue, so nothing establishes which classes the compiler actually rejects.

Their study also supplies something more useful than a comparison, which is a warning. Their two benchmarking sessions reversed the verdict. The first, after a code review of the kind normally performed in practice, found the two implementations comparable. The second, after both sides had been optimised further, found the Rust version worse by 85–91%. The authors reason that the first session is the more representative of software in general, which is a defensible reading. The more important observation is that a single-author matched pair can be made to yield either answer, depending on where the optimisation effort lands. Section **??** treats that as the principal threat to this study's own construct validity and describes what is done about it.

Two 2026 theses from outside the vendor set apply parts of this thesis's method to smaller objects, and both are worth acknowledging because they arrive at compatible answers independently. One builds functionally equivalent C and Rust user-space device drivers and subjects them to fault injection, fuzzing and equivalence testing, concluding that safety costs nothing measurable in run time for that class of driver but carries a fixed memory overhead [**sundvisson2026drivers**]. The other reproduces a set of CWE classes in C++, Rust and Python to establish at which stage each fault is caught, which is the staging question RQ2 asks [**andersson2026stdmemsafety**]. Neither involves concurrency, a telecom workload, or a load sweep, and both are bachelor's work. They are cited as convergent evidence on the smaller questions, not as substitutes for the comparison attempted here.

Table **??** sets this study beside the five directly, and two things are worth reading off it. The middle columns show that the earlier work is not weaker, only aimed differently. Four of the five measure a language together with the framework it was written on, and the fifth measures a matched pair on a stateless object at a fixed load. The last column shows what follows from that. Rust has the lowest peak memory in one study and higher peak memory in another. It is on par with C in one and at a fifth of DPDK's throughput in another. It tracks C++ at the median in one, and it is comparable and then much worse across two sessions of a single study. Those findings cannot be reconciled with each other, because no two of them hold the same things fixed. That is the signature of a literature in which the independent variable is not controlled, and it is why this thesis builds its object of study instead of finding one.

Table 7.2: This study beside the five vendor theses, all carried out at Ericsson and none peer-reviewed. The rows are ordered by year. The last column reports each study's own verdict in its own terms, which is why the entries do not line up: they are answers to different questions, measured on different objects in different regimes. The verdict recorded for this thesis is the one Section **??** defends, including the part the measurement host cannot resolve. Figure **??** places the same five studies on the axes of the gap instead, alongside the peer-reviewed literature.

Study	Object measured	Regime reported	Reported verdict on Rust
Karlbäck & Orö, 2021 [**karlback2021x86**]	RAN applications on x86 and ARM, one idiomatic implementation per language	Runtime and resource use at the loads exercised	C++ preferred where performance is critical. Rust's peak memory the lowest of the three languages
Olsson, 2023 [**olsson2023rannggmt**]	RAN node management software, Rust against C	Execution time and peak memory	Speed on par with or better than C, peak memory higher, safety materially improved
Olsson & Stenström, 2023 [**olsson2023asynchpackets**]	Packet processing: asynchronous Rust on Glommio against DPDK Eventdev in C	Throughput and latency against core count	20–50% of DPDK's throughput with far worse latency. Asynchronous Rust judged not currently viable
Emberg & Wadsten, 2023 [**emberg2023asn1**]	An ASN.1 decompiler built twice, the Rust side reaching C++ through a C wrapper	Execution time in two benchmarking sessions	Comparable in the first session, 85–91% worse in the second. Preferred in practice on non-performance grounds
Arvidsson & Kaller, 2025 [**arvidsson2025async**]	5G control-plane procedures across five asynchronous frameworks	Median and 99th percentile across a load sweep	Median tracks C++. The 99th percentile rises more steeply past a threshold request rate
This thesis, 2026	A UE Context Management service built twice from one frozen specification, with the concurrency model, allocator and build settings fixed	Median, p99, p99.9 and maximum across a sweep to and beyond saturation	Equivalent at the median below saturation. The far tail there is unresolved on this host. Past saturation C++ sustains 1.8× the throughput of Rust

Section **??** recorded that all five come from one vendor. Read against the table, that concentration has a sharper consequence. "C++ remains the choice where performance is critical" and "asynchronous Rust is not currently viable" are findings about particular software on particular infrastructure, and repetition inside one organisation does not establish their generality. The question is therefore not settled elsewhere and imported. It is open, unexamined in this setting, and the answer given here is one an independent

organisation can check, because the specification, both implementations and the harness are published with it.

Not all of this evidence points the same way, and the qualifying findings are taken as seriously here as the favourable ones. Even the positive networking result records a real cost. The idiomatic Rust driver issues substantially more instructions per packet than its C reference and is measurably slower in absolute terms. Its safety checks are absorbed by out-of-order execution, not literally free [**emmerich2019drivers**]. Two further matched comparisons point the same way on different workloads. Idiomatic safe Rust implementations of the STAMP transactional benchmarks run slower than both the C originals and machine-transpiled Rust [**suchert2023stamp**], and a general study of Rust against C reports it significantly slower in some cases [**okawa2024ruststudy**]. The largest study of Rust inside an existing C codebase reaches a similar conclusion from the other direction. It finds that Rust's safety features introduce runtime overhead and development complexity when they are not carefully managed [**li2026rustlinux**].

The evidence does not run one way, though. Safe low-level code without runtime overhead has been demonstrated for network device drivers, the same class of program as Emmerich's, when the safety obligations are discharged through type invariants rather than checks [**pirelli2023safelowlevel**]. The "zero-cost" framing [**balasubramanian2017rust**] is therefore best read as neither established nor refuted in general, but as a property of a particular program on a particular workload. That is why tail behaviour must be measured on the target workload and not assumed.

The safety story is likewise not absolute. Empirical studies of real-world Rust find that memory- and thread-safety bugs still occur in practice, with blocking bugs and misused synchronisation among them [**qin2020rust**, **qin2024safety**]. They also find that a substantial fraction of the ecosystem cannot be fully statically checked, because `unsafe` propagates through dependency chains [**evans2020rust**].

These qualifications do not overturn the case for Rust, but they motivate two of the thesis's design choices. The first is measuring predictability and defect prevention directly on the subsystem instead of importing general-purpose, average-case conclusions. The second is keeping the safety-critical core `unsafe`-free, so that the dependency-propagation caveat does not apply to it.

Three axes separate this thesis from all of it. The first is whether the comparison controls for design by building both systems from one specification. The second is whether it reports tail behaviour under overload instead of average-case performance. The third is whether it measures the cost of adoption instead of describing it. None of the three is unoccupied on its own, as the matched implementations [**emmerich2019drivers**, **li2021ipc**, **narayanan2020redleaf**, **suchert2023stamp**, **gonzalez2023rustudp**, **pirelli2023safelowlevel**, **emberg2023asn1**], the percentile sweep stopping at a latency requirement [**arvidsson2025async**], and the effort measurements [**karlback2021ran**, **fulton2021adopting**] above all show. No prior study combines the three for a stateful, concurrent control-plane workload, and that is the narrower gap this work occupies.

7.6 Threats to validity and mitigations

Every conclusion here attributes a difference to the implementation language. The credibility of the study therefore rests on the controls that hold everything else equal, and on an honest account of where those controls stop. Table **??** maps the threats first, by the kind of inference each endangers, so the remaining risks can be read as a set instead of gathered one at a time from the prose that follows. Three are load-bearing. The study cannot rule out single-author skill asymmetry. Its end-to-end instrument is the wrong one for part of what RQ1 and RQ3 ask. And its strongest single finding, the overload divergence, rests on an attribution it did not measure directly.

Table 7.3: Threats to validity by the kind of inference each endangers, the control applied, and what that control leaves behind. Only the entries in the last column are live risks. The middle column is what has already been done about each. The one row for which no control was available at all, the missing hardware counters behind the RQ3 result, says so instead of softening it.

Threat	Control applied	What remains
Internal: is the language really the only difference?		
Implementation bias between the two versions	Frozen specification, shared allocator and optimisation level, pinned hash functions, forced-idiom log, transcript-level parity testing	Single-author skill asymmetry. Parity bounds behaviour, not performance
Construct: do the metrics measure what the questions ask?		
The sweep measures the design, not the language	The operation is measured apart from the handoff and both are reported (§**??**)	One operation on one data structure is narrower than the sweep it supplements
The overload result is attributed but not localised	Narrowed to two entries in the forced-idiom log (§**??**). The contended-primitive microbenchmark (§**??**) rules out those two alone as the full explanation	The full handoff path under load still includes variant dispatch and larger requests on the C++ side
A defect catalogue is a sample the author chose	Seven classes drawn from outside the guarantee, with expectations recorded in advance	Fourteen programs remain fourteen programs
RQ3 captures resource cost but not its origin	None available on this host	The memory result is established but unexplained
RQ4 measures the seam, not the safety of the whole	The distinction is stated wherever the result is quoted	One address space: a compile-time property of the Rust side bounds nothing at run time
Conclusion: are the differences real or noise?		
Run-to-run non-determinism	20 interleaved repeats, bootstrap intervals, exact paired permutation test, Holm correction, floors quoted	Pairing cancels only what a block shared, so the far tail below saturation stays unresolved
A corrected p-value that does not replicate	The allocator control runs every comparison twice. Replication is followed over the p-value	It has cost a result before and would again. Here the tail rows are the ones it withholds
A contaminated run defeating a mean-based statistic	The reading rule of §**??**, fixed in advance, plus the per-repeat agreement count	Peak RSS under `jemalloc` is read from its interval against its own p-value
External: how far do the results travel?		
One subsystem, one workload, one machine	RQ2 rests on the compilers, RQ4 on the compilers, the C ABI and the dependency graph	The performance results do not travel, and the overload result travels least of all. RQ4's build-time seconds describe one machine, its ratio does not
A virtualised measurement host	Interleaving, pairing, and 20 repeats. The guest's limits recorded in the data (§**??**)	Frequency unobservable and CPU steal non-zero, neither removable from inside the guest

Internal validity: is the language really the only difference? The central threat is implementation bias between the two versions, mitigated structurally by the controlled-variable strategy of Section **??** and Table **??** and by the transcript-level parity testing of Section **??**. What remains uncontrolled is single-author skill asymmetry. One person wrote both implementations and is not equally expert in both, so an advantage to whichever language that person knows better cannot be ruled out. The parity suite bounds this threat for *behaviour* and not for *performance*.

That the threat is real and not merely conceivable is documented in the same industry, by the two benchmarking sessions of Emberg and Wadsten that reversed their own verdict [**emberg2023asn1**]. One matched pair supported two different conclusions, depending on where the optimisation effort landed. The controls here are chosen against that failure mode. The specification is frozen before either implementation is written, not negotiated between them. The optimisation level and allocator are matched, not tuned per side. And forced divergences are logged, not resolved in whichever direction the author finds easier. None of that reduces the remaining risk to zero. It makes that risk visible, which is why the forced-idiom log is a deliverable and not an appendix.

Construct validity: do the metrics measure what the questions ask? RQ1 is answered with high percentiles and the maximum instead of the mean, which is the standard way of measuring predictability in this literature [**dean2013tail**]. Two risks remain. The metric list was narrowed to those four during the work, because inter-operation jitter and p99.99 are instrumented but postdate the reported campaign (Section **??**). And p99.9 on a synthetic workload is not the same quantity as a missed procedure deadline in a live network.

A second and larger construct gap in RQ1 was found rather than anticipated. The end-to-end latency the sweep reports is 0.250% operation and the rest cross-thread handoff (Section **??**), so it measures the delivered cost of the design faithfully and the cost of the language barely at all. Which of those RQ1 asks about depends on the reader. A vendor sizing a node wants the first, a language comparison the second. The study reports both and keeps them apart, but the question as originally worded did not tell them apart. Two remedies are applied, and no amount of statistical care substitutes for either. The operation is measured apart from the handoff, and the sweep is pushed into the regime where the design stops absorbing the difference. What remains is that a microbenchmark of one operation is a narrower object than the sweep it supplements.

The remaining three are stated in Table **??** and bounded here. RQ2's catalogue is a sample the author chose. The seven boundary classes mitigate that, because they force it to contain cases the expected conclusion does not cover, and two of them returned the opposite result. Fourteen programs are still fourteen programs. RQ3 captures resource cost but not its origin, because no hardware counters or allocation profiles were collected, so the study can say that the memory penalty appeared and not why. The mechanisms most likely responsible have been isolated elsewhere [**zhang2022runtime**, **harris2023morello**]. RQ4 measures the size of the seam, the `unsafe` it concentrates, and the per-call cost. Those are the right quantities for the question as posed, and they are not a measure of how safe the composed system is. Host and component share an

address space, so a compile-time property of the Rust side bounds nothing the C++ side can corrupt at run time [**rivera2021galeed**, **kirth2022pkrusafe**].

Conclusion validity: are the differences real or noise? The statistical treatment is set out in Section **??** and its outcomes in Table **??**. The repeat count is chosen so that the design is not the binding constraint: an exact test on 20 pairs floors at 0.0000019, and at 0.000027 after correcting 14 comparisons, so a null here reflects the data rather than the arithmetic. An earlier pass at seven repeats did not have that property, and its corrected floor above 0.05 made every null in the table uninterpretable.

Three limitations remain. First, the measurements were taken inside a virtual machine that does not expose frequency or turbo control and that records non-zero CPU steal (Section **??**). The far tail therefore contains a contribution from the hypervisor that cannot be separated from the software under test. That is why the far tail below saturation is reported as unresolved and not as a null. It interacts with the pairing in a way worth naming. Pairing removes the interference that is *common* to a repeat, which is why run-level statistics such as peak RSS, CPU, and sustained throughput come through cleanly. It cannot remove interference that lands on one run and not its partner, and that is exactly the regime the tail percentiles live in. It is also why the overload comparison is made on throughput and not on latency. A run-level average is the statistic pairing can rescue on a host like this one.

Second, a corrected p-value below 0.05 is necessary here and visibly not enough, and the converse also holds. On an earlier campaign, before the two `jemalloc` builds were held to one configuration, CPU utilisation cleared the Holm correction under one allocator and reversed sign under the other. No threshold would have caught that. It was visible only because the design runs every comparison twice. In the present campaign the control withholds nothing a threshold would have promoted, though the tail rows still fail it. The opposite direction has a case too, in the peak resident set size under `jemalloc` described in Section **??**. There an interval far from zero and 18 of 20 repeats in agreement sit against a p-value near unity, which a handful of low outliers are enough to produce. That row is read from its interval, under the rule fixed in Section **??** and not chosen once the disagreement appeared. The bimodality behind it is a defect in the measurement and not in either allocator, because it affects both languages in equal numbers and appears at every load level. Diagnosing it is left as instrumentation work.

Third, the overload divergence is measured and replicated, and its mechanism is now tested and not only argued. Section **??** isolates the bounded queue and completion primitives under the same pattern of 7 producers. They do not reproduce the full-service ordering at significance ($p = 0.226$). That rules out those two constructs alone as the whole explanation, even though they are where the forced-idiom log predicted contention would matter. The cause is not unknowable. It likely includes extra work on the C++ handoff path, in the larger tagged `Request`, in variant dispatch, and in the map operations the microbenchmark strips away.

External validity: how far do the results travel? The study covers one subsystem, one workload, one concurrency structure, and one machine, and each of those bounds the reach of the conclusions differently. The safety and migration results (RQ2, RQ4) travel most easily, because they are properties of the compilers and of the C ABI and not of this subsystem. Any C++ codebase writing these defect classes gets the same compile-time outcome, and any narrow, scalar-typed interface gets a similarly cheap seam. Two of RQ4's adoption axes carry the same way, because a binary's size and a dependency graph's shape follow from the toolchain. The third does not travel as an absolute, because build seconds belong to the machine that measured them, and what carries there is the ratio between two toolchains timed alternately. The build-time *shape* is specific too, in a way worth naming. A two-file C++ library and a single Rust crate are both small, and the incremental result in particular would move on a codebase with a different header structure.

The performance results travel least. UECM was deliberately chosen to be small and partitionable, and its shared-nothing design removes lock contention from the hot path. A subsystem with genuinely shared mutable state, deep call graphs, or heavy allocation churn could therefore exercise the two languages' abstraction and allocation costs quite differently. The workload is synthetic and spec-justified, not replayed from production traffic, so its operation mix is plausible but unvalidated. Nothing here speaks to PHY or L2 processing, to the data-heavy user plane, or to the operational concerns (upgrade, observability, multi-tenancy) that a production deployment would raise.

The defensible claim is therefore narrow and carries a condition. For stateful, concurrent, partitionable control-plane services of this kind, and while they are meeting their offered load, the language choice is not a performance decision. Once they are not meeting it, the language choice is one. And the overload result is the least transferable finding in the study rather than the most, because it depends on the queue bound, on one request being outstanding per producer, and on two particular standard-library primitives. Extending either claim beyond this class means rerunning the framework, which is why the framework and not the numbers is offered as the reusable contribution.

A final limit on reach is temporal and not structural, and it applies to the comparison itself and not to any one result. One of the two objects being compared is moving. The C++ side is pinned to C++23 as specified and shipped, and to the compiler versions recorded in Section **??**, while the standards committee is actively pursuing enforced safety profiles (Section **??**). This does not qualify anything measured here, because none of that work is standardised or generally implemented, and the concurrency profile that would reach the boundary classes of RQ2 comes after an initial set that is itself still to be specified [**vandevoorde2026profiles**]. It does mean the RQ2 result should be read as a statement about these languages as they stand, not as a permanent one. An implemented and enforced lifetime profile would narrow the memory-safety half of the catalogue. An implemented concurrency profile would bear on the half deliberately placed outside Rust's guarantee. Once again the framework is what survives. The catalogue is designed to be re-run against a compiler that enforces profiles, and the answer would then be measured instead of predicted.

7.7 Ethical and professional reflection

The conduct of the study raised no data-protection questions. It processes no personal, subscriber, or operator-confidential data. All traffic is synthetic and generated by the deterministic driver, the contexts carry fabricated identifiers and a fixed dummy security field, and the subsystem is modelled on production code rather than copied from it. No privacy, consent, or data-retention concern arises, so the artifacts can be released openly for reproduction. The *use* of the study is a different matter and deserves stating explicitly, because a language-selection recommendation in this domain is not a neutral technical output.

Two responsibilities follow. The first concerns over-claiming. A vendor could reasonably read the RQ2 result as licence to describe a migrated component as "safe", and the boundary cases are why that reading would be wrong. A deadlocked control plane is an outage, a silently lost counter update can misinform a mobility decision, and nothing demonstrated here prevents either. Reporting the boundary as prominently as the guarantee is a professional obligation and not a stylistic choice, precisely because the audience for this work is making a decision that affects networks people depend on.

The second concerns the weight placed on evidence the host cannot fully support. Tail-latency numbers measured inside a virtual machine with non-zero CPU steal are not a sound basis for a production commitment. Saying so in the text only works if the qualification travels with the numbers when they are quoted elsewhere. That is part of why the summary table records an evidence status per research question and not a finding alone. Beyond the immediate results, a migration recommendation also carries organisational consequences that fall on people and not on systems: retraining, hiring, and a period of reduced velocity. The honest framing of the RQ4 finding is that the cost was moved from run time to human effort, not removed.

8 Conclusion and Future Work

This thesis compared modern C++ and Rust on a UE Context Management service modelled on a real Nokia gNB-CU-CP component, along four dimensions: tail-latency predictability, compile-time defect prevention, safety-model overhead, and incremental-adoption cost. Built against a single frozen specification and driven through identical code paths, the two implementations let language effects be isolated from design and skill differences.

Three conclusions follow from the evidence. First, on safety the difference is one of *kind*, and it has a definite edge. Safe Rust rejects all seven injected memory-safety and data-race classes at compile time, and one of those seven is a class no C++ sanitiser reports at all. Runtime hazards that C++ can only hope to catch with sanitiser-instrumented tests become errors that cannot be written at all. The edge stops there. All seven boundary classes compile and misbehave in safe Rust exactly as they do in C++, and on two of them the C++ tooling is the one that reports. The guarantee is therefore best described precisely. Rust eliminates data races, not race conditions, and aliasing errors, not lock-ordering errors, leaks, wrapping arithmetic or fault containment, and it eliminates none of them in a region marked `unsafe`. Where the boundary defects are caught, they are caught at test time, in two of the three cases only after being rewritten into a model checker's primitives, which is a real cost and a different one from the sanitiser's.

Second, on migration the cost is dominated by API design and the learning curve, not by any run-time tax. An unmodified Rust core can be driven from C++ across a thin, `unsafe`-isolated seam whose cost on a real operation, measured against the same operation called natively, is bounded at 10.91%. The costs a build engineer would raise first do not change that ordering. Most of the Rust build's extra expense is compiling dependencies and not the compiler's own work, the binary-size gap is a linkage policy and reverses when the policy is matched, and the compared component resolves no third-party code at all. Where C++ leads is the edit-compile loop, and it leads only for edits that stay inside a translation unit.

Third, on performance the evidence divides twice before it divides by resource. It divides first by *what is being measured*, and then by *whether the service is coping*. Getting both divisions right was the study's most consequential correction to itself, and each was found rather than anticipated. An end-to-end operation in this design is 0.250% the work the language performs, and the rest is a cross-thread handoff the operating system performs, so an end-to-end null cannot bound a language effect at all. Measured on the operation alone the languages are not equivalent. Rust is the faster of the two, by 1.6×, for reasons that trace to the standard library's choice of hash-map layout and not to anything about safety.

The second division is one of regime, and it is the more consequential. Below saturation the two languages are equivalent at the median for any purpose a deployment would care about, and that is a real result about deployments of this shape. The equivalence is practical rather than statistical, since the paired test does resolve 1.6 μs in Rust's favour

under both allocators. Their far tail stays unresolvable on a virtualised host, which is an admission about the instrument. Past saturation they are not equivalent. The C++ build sustains 1.8× the throughput of the Rust build, in 20/20 interleaved repeats under both allocators, while drawing 1.9× the CPU per completed operation, and the isolated primitives do not reproduce that ordering at significance (Section **??**). On memory the Rust build carries a few MiB more fixed footprint under both allocators, an order of magnitude less than the allocator choice itself costs.

Taken together these support a narrow claim, narrower than the one this thesis expected to make. For stateful, concurrent, partitionable control-plane services of this kind, and *at nominal load*, the choice between modern C++ and Rust is not a performance decision. It is a decision about which classes of defect an organisation wants to make unwriteable, and about whether it will pay the engineering and learning cost of getting them. Under overload it becomes a performance decision again, and one that currently favours C++. The reasons lie in the concurrency plumbing and not in either language's guarantees, and a service required to degrade gracefully cannot treat them as incidental. What this thesis adds is therefore an instrument and not a verdict: a frozen specification, a parity-tested pair of implementations, a reproducible measurement framework that reaches past the saturation knee, and a defect catalogue that maps both what the guarantee covers and where it stops.

Several directions extend the work, each of them the direct remedy to a limitation named above. The immediate priority is executing the bare-metal campaign, whose target is now narrow: the saturation knee and the overload comparison are already resolved, and what bare metal would buy specifically is the far tail below saturation, together with hardware counters and allocation profiles on the same visit. Beyond that:

- Replay captured, anonymised production traces in place of the synthetic workload, so that the operation mix is validated rather than merely spec-justified.
- Scale the comparison to further control-plane subsystems, or to the data-heavy L2/PHY path, where shared mutable state and allocation churn would exercise the two languages differently than a shared-nothing design does.
- Instrument the comparison with hardware performance counters and allocation profiles. This is what would turn the memory result of Section **??** from an observation into an explanation, and what would confirm or refute the cache-locality account offered in Section **??** for the operation-level gap. It is the clearest instrumentation gap in the study as it stands.
- Extend the fault catalogue further along the boundary mapped here. Leaks, wrapping arithmetic and worker failure are now in it. Misuse of `unsafe` and the behaviour of the boundary classes under a whole-program model checker rather than a hand-shrunk model are the obvious next steps. A checker that covers a Rust program together with its foreign dependencies [**pearce2026rustmc**] would extend the same treatment to the seam of Section **??**, whose cost this study measures but whose interleavings it does not explore.

- Evaluate Rust async runtimes against the thread-per-shard model used here. That would separate the tail divergence Arvidsson and Kaller attribute to stackless `async`/`await` [**arvidsson2025async**] from the language itself, under the controls this study imposes. The same experiment would settle a second question at the same time. Olsson and Stenström measured asynchronous Rust at a fraction of DPDK's throughput on a thread-per-core packet processor [**olsson2023asyncpackets**], and nothing in the literature yet separates how much of that gap belongs to the runtime and how much to the framework it was measured against.
- Measure what in-process isolation of the two heaps costs at the seam, since that is what an `unsafe`-free core would need to be protected as well as verified [**rivera2021galeed**, **kirth2022pkrusafe**].
- Track defect density and review effort in a longer-term case study, as Rust is adopted incrementally in a production RAN codebase.

Statement on the Use of AI Tools

Tool. Anthropic's Claude Opus 5, used through the Cursor development environment, is the only generative AI system used in preparing this thesis, and it was used throughout, from June to November 2026. It is named neither as an author, which University of Helsinki policy forbids, nor as a source, which is the author's own choice.

What the assistance covered. It was extensive. The model drafted and revised the running prose and proposed the order in which the arguments are made. It wrote and refactored both implementations of the UE Context Management service, the load generator, the fault-injection catalogue, the interoperability prototype, the analysis code behind the bootstrap intervals and the paired permutation tests, and the figure-generating, build and LaTeX sources. It also suggested candidate literature. The abstract is the one part of the thesis it must not touch, because University of Helsinki policy never permits a language model in a maturity test, and in this programme the maturity test is the abstract. In this draft it has not yet been rewritten unaided, and this statement will say so until it has been.

What the model did not settle. The research problem, the four research questions, the three hypotheses, and the control on which the study's value rests are the author's. That control is one frozen language-neutral specification and one concurrency model, so that language remains the only systematic difference between the two implementations. The author also chose to carry defect classes from outside Rust's compile-time guarantee, which is what keeps RQ2 falsifiable. Every measurement was run by the author on the hosts documented here, and the measured quantities the thesis quotes reach the page as macros regenerated from the raw logs rather than as figures typed into a paragraph. The readings are the author's too, including the two restraints the evidence imposes: an unresolved far tail is not equivalence, and a sweep dominated by cross-thread handoff cannot bound an effect of the language.

How the output was checked. Model output was treated as a draft to be falsified. Every bibliography entry was resolved against Crossref, DataCite, or the issuing body and traced to the page or clause carrying the claim made of it. That audit found four passages that overstated or misplaced their source, one of them a quotation absent from the work it was attributed to, and all four were corrected, with the audit kept beside the artefacts. Code was checked by running it, against a conformance suite whose transcripts are diffed line by line and a fault catalogue that fixes each expected outcome before the defect is injected.

What the author took from it. The model is dependable at mechanical work, at transposing an idiom between two languages or keeping a build honest. It is undependable in one predictable way: it will supply a plausible citation for a claim and a confident sentence where the data supports a hedge. Its fluency is not evidence. The effort saved in drafting is spent again in checking, and the defensible parts of this thesis are the parts checked against a source, a specification, or a measurement.

Responsibility. The author is responsible for the whole of this thesis. Where it is wrong,

the error is the author's.

Appendix A Abbreviations and key terms

The abbreviations and terms used throughout the thesis, for readers outside the telecom domain.

Table A.1: Abbreviations and key terms used throughout the thesis.

Term	Meaning
RAN	Radio Access Network: the part of the mobile network connecting devices to the core
NG-RAN	Next-Generation RAN, the 5G radio-access-network architecture
IMT-2030	The ITU framework for the generation after 5G, commonly called 6G
AI-RAN	An architecture in which AI and RAN workloads share one accelerated infrastructure
gNB	5G base station (“next-generation NodeB”)
gNB-CU / DU	Central Unit / Distributed Unit: the disaggregated halves of a gNB
gNB-CU-CP	Control-plane part of the central unit: the system class studied here
UE	User Equipment: an end device (phone, modem) attached to the network
UECM	UE Context Management: the per-UE state-management subsystem reimplemented here
RRC	Radio Resource Control: the control-plane signalling protocol between UE and gNB
DRB	Data Radio Bearer: a per-UE user-plane data channel set up during a session
Control plane	The signalling and management path, as opposed to the user-plane data path
Tail latency	High-percentile latency (p99, p99.9), the worst-case-oriented timing metric
Jitter	How much latency varies from one operation to the next, as opposed to its typical value
Saturation	The offered load beyond which the service can no longer keep up and requests start queueing
Shard	One of several independent slices of the UE state, each owned by exactly one worker thread
FFI	Foreign Function Interface: calling Rust from C++ across a C ABI
HDR histogram	High-Dynamic-Range histogram, used for accurate latency capture

Appendix B Literature search protocol

Section **??** reports the two findings this search produced. The protocol is recorded here so that the gap the study claims can be checked rather than taken on trust.

Candidate literature was gathered from the ACM Digital Library, IEEE Xplore, USENIX, and Google Scholar, using combinations of the terms *Rust*, *C++*, *memory safety*, *tail latency*, *systems programming*, *performance comparison*, and *RAN / 5G control plane*. The search was restricted to peer-reviewed venues, except where an industry or government report is itself the object of study, as the memory-safety mandates and vendor vulnerability statistics are. The result set was then extended by backward and forward citation chaining from the core Rust-safety papers, and filtered to work that either measures a systems language empirically, formalises Rust's guarantees, or reports adoption experience. The domain literature (3GPP specifications and the O-RAN survey) was added separately to fix the architectural context.

Those sources alone would have missed the closest prior work, which is why two more were added. The vendor-hosted theses that come nearest to this study (Section **??**) appear in none of the citation databases: not in Crossref, OpenAlex, DBLP, or Semantic Scholar. Neither forward nor backward chaining can reach them, because nothing indexed cites them and their own reference lists are not deposited. They were found instead by searching the institutional repositories directly, DiVA for the Nordic universities and Finna for the Finnish ones. The same two searches establish the absence on the other side. Finna, which federates the Finnish university repositories, returns no thesis on Rust in a radio-access or telecom context. An author-affiliation search of Nokia's indexed publications in OpenAlex returns none either. Its entities hold on the order of thirty thousand indexed works, several hundred of which concern the radio access network, and one mentions Rust, in an unrelated preprint on trusted-execution monitoring.

Two practical notes belong with the protocol, since both distort a naive search. Free-text queries for *Rust* in the general indexes rank plant pathology and corrosion chemistry above the programming language, so field-restricted or discipline-restricted queries are necessary. And the theses are grey literature, so they are cited as evidence of what has been attempted and found rather than as peer-reviewed results.

Appendix C Statistical design: family membership and resolving power

Section **??** states two commitments: which comparisons enter the corrected family, and how far into a difference the design can see at all. The reasoning behind them is recorded here, so that neither can be read as a choice made once the p-values were known.

Which comparisons enter the family. The representative operating point is chosen below saturation, where every variant completes work as fast as it is offered. Achieved throughput there is therefore pinned to the offered load by construction, and it cannot express a language effect whatever the languages do. Testing it would re-measure the operating point and not the code. It would also not be free. Holm divides its budget across the whole family, so two comparisons that cannot resolve anything would make the correction harsher on the ones that can. Below saturation, throughput is therefore reported with its interval in the resource table, where its job is to confirm that the operating point is sustained and not merely nominal.

Above saturation that reasoning reverses, and the design has to follow it. Once the offered rate is no longer being met, what each variant still sustains is pinned to nothing and becomes the one quantity that can separate them. Excluding it there would discard the only observable the regime offers. The second operating point is fixed in the overload region, defined as the deepest swept load at which *every* variant has stopped meeting the offered rate, and sustained throughput and CPU utilisation are tested at it (Section **??**). Those comparisons join the same Holm family instead of forming a second one, so the correction covers everything the thesis reports. No latency percentile is tested above saturation at all. With a fixed intended send schedule, a producer that cannot keep up falls behind cumulatively, so the percentiles there measure accumulated backlog against a measurement window that cuts it off. That makes them a lower bound and not an estimate, and attaching a p-value to a lower bound would test the harness and not the software.

The floors the design can reach. An exact test on n pairs cannot return a two-sided p below $2/2^n$, and after correcting a family of m comparisons the smallest attainable adjusted value is m times that. With 20 repeats and 14 comparisons those two floors are 0.0000019 and 0.000027, both quoted in Section **??**. A campaign whose corrected floor lies above the threshold being applied cannot demonstrate significance however large the effect, so reporting "not significant" from such a campaign would say more about its repeat count than about the languages.

Appendix D Host and campaign configuration

Table ?? itemises the measurement host of Section ?? against the bare-metal protocol that testbed-grade tail figures require. Table ?? gathers the configuration of the service, the workload and the campaign, whose derived values Section ?? quotes.

Table D.1: The host the reported results were measured on, against the protocol that testbed-grade tail figures require (`bench/TESTBED.md`). The rows where the two columns agree are properties of the source and the compilers. The rows where they differ are what a bare-metal campaign would still buy. Only the last of those differences now costs a result, because the saturation knee and the resource comparisons were resolved on this host: what remains is the far tail below saturation.

Aspect	Used for the results reported here	Required for testbed-grade tail figures
Host	Dedicated dual-socket server, 2× Xeon E5-2680 v4, 16 vCPUs, one NUMA node	Same class of machine, without the hypervisor
Virtualisation	KVM guest: frequency and turbo host-controlled and unobservable	None. Governor and turbo set and verified
CPU steal	Non-zero, order 10 ms of vCPU time per second	None
Kernel	Rocky Linux 8.10, distribution kernel	Same, with `isolcpus` and `nohz_full` on the benchmark cores
Toolchain	`g++ 15.1.1`, `rustc 1.97`	`g++ 15.1.1`, `rustc 1.97`
CPU pinning	Cores 2–15 via `taskset`, 7 shards, 7 producers	Same, on cores also removed from the scheduler
Simultaneous multithreading	Absent from the topology the guest is shown. Whether its vCPUs share thread siblings is decided on the host	Absent or disabled, and verified on the machine itself
Warm-up	Initial fraction of each run discarded	Initial fraction of each run discarded
Repeats and interleaving	20 repeats, four variants interleaved per repeat	20 repeats, four variants interleaved per repeat
Environment capture	`bench/capture_env.sh`, stored with each result	`bench/capture_env.sh`, stored with each result
Hardware counters	Unavailable in the guest	Available: the clearest instrumentation gap
Adequate for	Parity, fault injection, FFI structure, the saturation knee and the overload comparison, and the resource comparisons	Additionally the far tail below saturation, and the mechanism behind the resource and operation results

Table D.2: Configuration of the service under test, the workload driving it, and the measurement campaign. Service parameters are fixed by the frozen specification, with one exception: the shard count is set to match the cores the host offers (Section **??**). All of them, that one included, are identical in both languages. Campaign figures are generated from the measurement logs together with the tables that report them. The representative load is not chosen by eye but derived, as the busiest swept load at which all four build variants still hold their p99 below 5 ms.

Parameter	Value	Note
Service under test, identical in both languages		
Shards, and worker threads	7	one thread owns one shard, one of each per pinned core
Queue bound per shard	1 024 requests	bounded and blocking, so overload is observable
Initial map capacity	4 096 entries	reserved so steady state does not rehash
Bearers per context	at most 8	
Signaling payload	64 bytes	
Context ceiling per shard	200 000	
Shard selection	SplitMix64, then modulo	pinned: diffuses sequential identifiers
Intra-shard hash	FNV-1a, 64-bit	pinned on both sides and checked bit-for-bit
Workload		
Workloads	signal, mixed	mixed walks the full RAN lifecycle
Active UEs	5 000	
Offered-load levels	14	up to 784 kops/s
Representative load	56 kops/s	derived, see caption
Measured saturation	335 kops/s	tracks the offered rate to 336 kops/s
Load model	open loop, fixed intended schedule	latency measured from intended send time
Campaign		
Build variants	4	two languages × two allocators
Repeats per sweep point	20	variants interleaved within each repeat
Measured runs	1120	
Latency capture	HDR histogram, [1 ns, 3.6 s], 3 s.f.	same algorithm and configuration on both sides
Summary statistic	median with bootstrap 95% CI	intervals reported in full, not as half-widths
Language comparison	exact paired permutation test	14 tests, Holm-corrected
Attainable p floors	0.0000019 / 0.000027	uncorrected / after correction

Appendix E Fault-injection example

As an illustration of the fault-injection study (Chapter **??**, RQ2), the use-after-free defect is shown in both languages. In C++ the alias into the context map survives the `erase` that frees it, so the following program compiles cleanly and is flagged only at runtime by AddressSanitizer:

```
std::unordered_map<unsigned long, UeContext> ctxs;
ctxs[1001] = UeContext{1001, 0};
UeContext* p = &ctxs.at(1001);  // alias into the container
ctxs.erase(1001);               // frees the node that p refers to
p->messages_handled += 1;       // use-after-free
```

The analogous safe-Rust program does not compile: the borrow checker forbids mutating the map (`remove`) while the reference `p` into it is still live, so the defect cannot be written in the first place (error `E0499`):

```
let mut ctxs: HashMap<u64, UeContext> = HashMap::new();
ctxs.insert(1001, UeContext { messages_handled: 0 });
let p = ctxs.get_mut(&1001).unwrap(); // mutable borrow starts
ctxs.remove(&1001);                   // borrow while p live -> rejected
p.messages_handled += 1;              // p used after the remove
```